

CS329A 学习笔记

Self-Improving AI Agents · Stanford 2025 秋

非官方个人笔记 · 根据公开课视频字幕整理 · 图为自绘示意

2026-09-24

CS329A 学习笔记

Stanford CS329A: Self-Improving AI Agents (2025 秋) · 讲者 Aakanksha Chowdhery、Azalia Mirhoseini 非官方个人笔记，根据 YouTube 公开课视频的字幕整理 · [播放列表](#) · [课程主页与阅读清单](#)

一句话：整门课讲的是一个回路——让模型在推理时多花算力（采样、搜索、长思考、用工具），用验证器把好结果挑出来，再把这些结果训回模型；然后把同一套思路从"答题"推广到"完成任务"的 agent，并追问怎么评测它、下一步往哪走。

课程地图

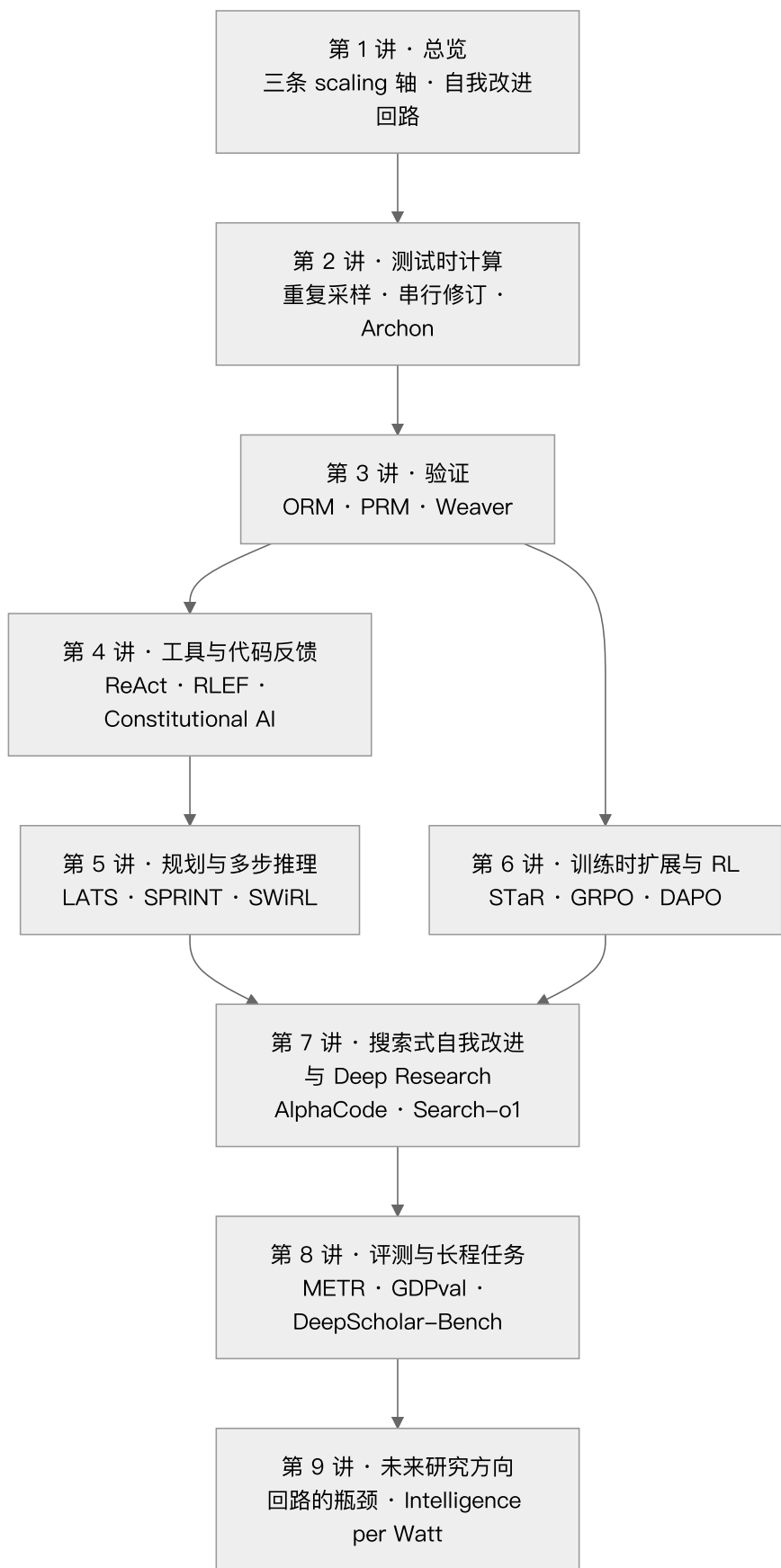


图 0-1 | 九讲之间的依赖关系（自绘示意）：第 2、3 讲是"推理时多算 + 挑得出来"这对基本矛盾；往右一支把它用在带工具的多步任务上，往左一支把它训回模型；两支在第 7 讲汇合成能自己做研究、自己改进的 agent

九讲索引

讲	主题	时长	视频
第 1 讲	课程总览：三条 scaling 轴、推理模型、从 LLM 到 agent	1:09:42	YouTube
第 2 讲	测试时计算扩展：重复采样的幂律、并行 vs 串行、Archon	1:03:21	YouTube
第 3 讲	稳健的验证：ORM、PRM、Math-Shepherd、Weaver	1:12:59	YouTube
第 4 讲	用工具与代码反馈来学习：ReAct、RLEF、Constitutional AI	1:11:13	YouTube
第 5 讲	规划与多步推理：LATS 树搜索、SPRINT 并行推理、SWiRL 分步 RL	1:14:56	YouTube
第 6 讲	训练时扩展与 RL 的规模化：STaR、DeepSeekMath 与 GRPO、DAPO	1:12:39	YouTube
第 7 讲	搜索式自我改进与 Deep Research：AlphaCode 2、Search-o1	1:12:27	YouTube
第 8 讲	智能体评测与长程任务：METR time horizon、GDPval、DeepScholar-Bench	1:15:18	YouTube
第 9 讲	未来研究方向：回路的三处瓶颈（多样性、验证、出题）与 Intelligence per Watt	1:07:42	YouTube

怎么用这份笔记

- **每讲的结构是固定的**：一句话 → 时间轴 → 核心内容（带图）→ 关键图表速查 → 提到的工作 → 术语对照 → 字幕勘误 → 带走的问题。赶时间就只看"一句话"、图和"带走的问题"。
- **时间戳都能点**，直接跳到视频里对应的位置。图下面黄色的「▶ 看原幻灯片」会跳到老师讲那一页 PPT 的时刻。
- **图是我重新画的示意图**，用来解释机制，不是课件截图。课上的原图大多出自论文，图注和"关键图表速查"里给了论文链接。
- **想对照原文**：视频自带人工校对的英文 CC；在播放器「设置 → 字幕 → 自动翻译 → 中文（简体）」可以开中文字幕；登录 YouTube 后，描述区的「内容转文字」能看全文并点句跳转。

- "小注" 是我补充的课外事实或对口误的更正；"应为 / 应出自" 表示这是我根据内容推断的，老师没有明说。
- 字幕对人名、论文名、模型名的识别错误不少，每讲末尾的"字幕勘误"列了会影响理解的那些。

YouTube 上没有放出来的课次

全课共 20 次，公开视频只有 9 讲。按课程主页的课表，未公开的包括：Melvin Johnson (Google DeepMind) 讲后训练从聊天机器人到 agent 的演进；软件工程的 agentic 框架 (CodeMonkeys、KernelBench)；Junchen Jiang 讲 agent 的记忆 (Cartridges、MemGPT、CacheBlend)；Denny Zhou 讲 LLM 推理；Thang Luong 讲 AlphaProof、AlphaGeometry 与 Gemini 的 IMO 金牌；Misha Laskin (Reflection AI) 讲自主 agent 系统；Danny Driess (Physical Intelligence) 讲机器人里的多模态 agent；以及三次期中展示。这些课次的阅读清单在课程主页上都有链接。

CS329A 第 1 讲 | 课程总览 (Course Overview)

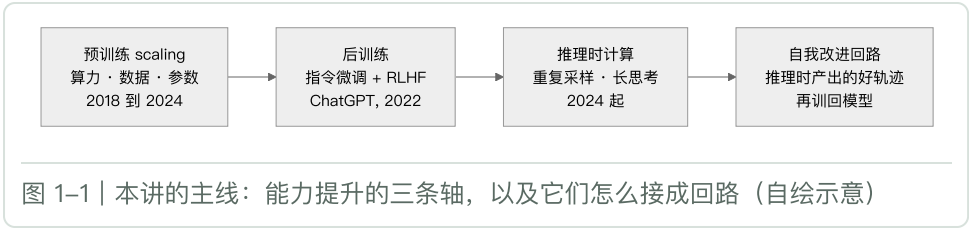
Stanford CS329A: Self-Improving AI Agents (2025 秋) 视频：
<https://www.youtube.com/watch?v=6YnLB0XbTnI> (1:09:42, 自带人工校对英文 CC) 讲者：Aakanksha Chowdhery (Stanford 兼职教授 / Reflection AI, PaLM 一作)、Azalia Mirhoseini (Stanford CS 助理教授, 此前在 Google Brain / DeepMind、Anthropic) 课程主页：
<https://cs329a.stanford.edu>

一句话：LLM 的进步先后来自三条轴——预训练规模、后训练（指令微调 + RLHF）、推理时计算；把"推理时多算"换来的高质量轨迹再训回模型，就是课程名里的"自我改进"。Agent 是把这种能力放进"目标—行动—反馈—停止"的循环，而**可靠的验证**是整条链路的瓶颈。

时间轴

时间	内容
0:06	开场、讲者介绍、本讲安排
2:23	Scaling laws：算力 / 数据 / 参数三条轴
4:11	模型规模的指数增长；few-shot / zero-shot
6:50	涌现能力与 Chain-of-Thought
10:50	ChatGPT 的关键增量；预训练 → 精选数据 → 指令微调 → RLHF
19:26	推理成为新前沿：Large Language Monkeys（重复采样）
24:32	问答：搜索空间、延迟、无验证器领域、温度
27:50	DeepSeek / o1 / Gemini Thinking：测试时扩展 × 微调 = 自我改进回路
30:05	推理模型：o1 的 test-time 曲线、"思考"的几个环节
34:46	o1 vs GPT-4o；问答：pass@1 vs pass@k、模型偏爱自己的轨迹、ORM/PRM
40:50	从 LLM 到 Agent：定义、agentic workflow、编排模式
47:56	Coding agent 循环；为什么今年变可靠；generator—verifier gap
52:50	澄清用户意图；落地场景：代码、客服、研究报告、AI Scientist
58:07	问答：CoT 是涌现的还是训练出来的
1:00:20	课程安排：作业、项目、评分、时间点

核心内容



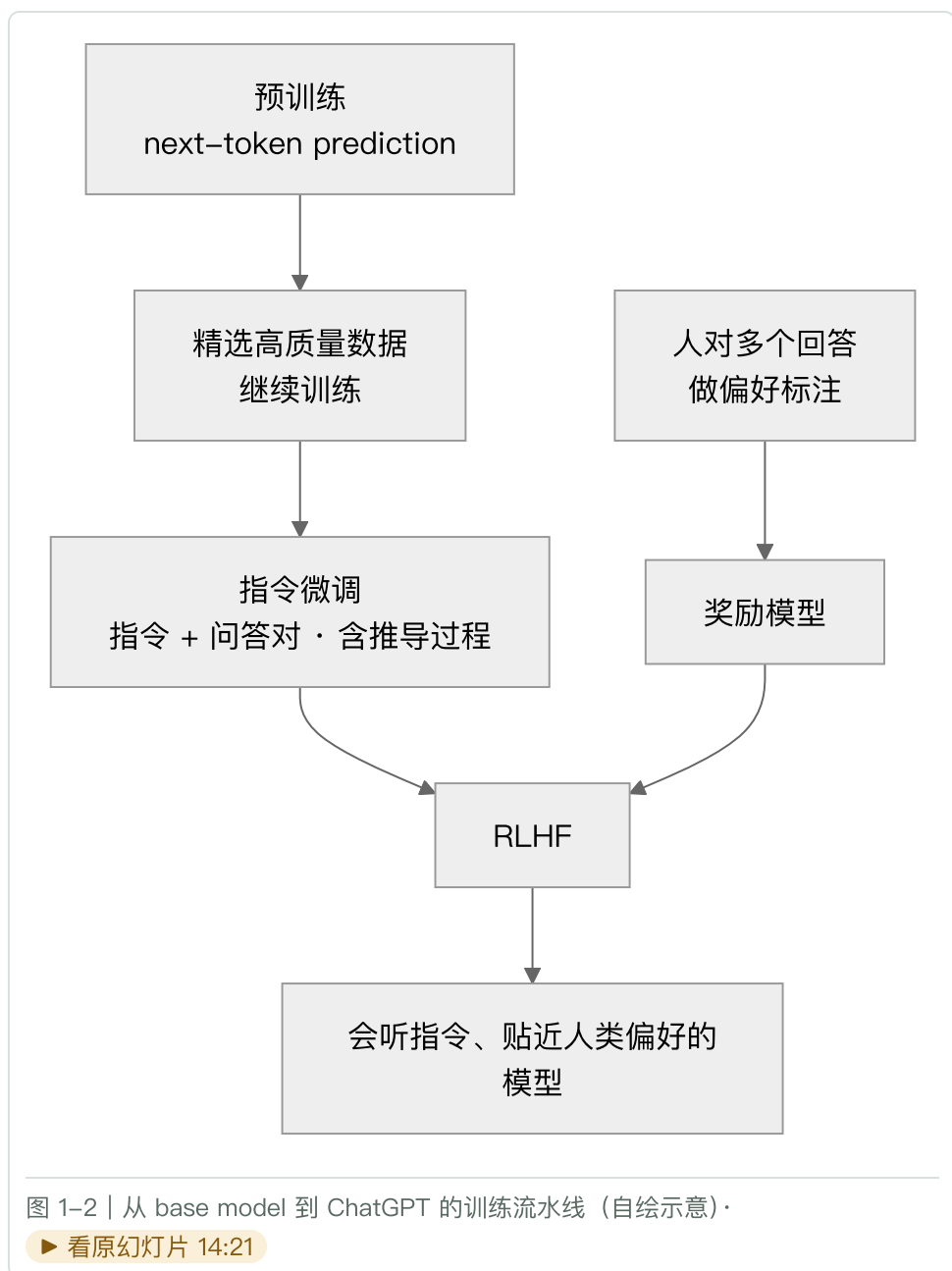
1. 预训练时代：三条 scaling 轴

- 增加算力、数据量、参数量，测试 loss 都按可预测的规律下降——这是 GPT-3、PaLM、Gemini 一路做大的依据。
- 规模演进：BERT 3.4 亿 → GPT-2 15 亿 → GPT-3 1750 亿 → PaLM 5400 亿 → GPT-4（外界估计万亿级）。
- 讲者的判断：这条路从 2018 年一直有效，到 2024 年开始出现饱和迹象。这是后面转向后训练和推理时扩展的背景。
- 做大带来三件事：基准分数持续提升；few-shot / zero-shot（给任务描述或几个例子就能照做，不必逐任务微调，原型成本骤降）；涌现能力（到一定规模才出现，无法从 loss 曲线事先预测）。

2. Chain-of-Thought：最重要的涌现能力

- 做法：few-shot 示例里不只给答案，还给出推导过程。
- 规模依赖：LaMDA / GPT / PaLM 的对比中，约 8B 以下的模型几乎吃不到 CoT 的好处，大模型才明显受益。模运算、单词重排等任务同样呈现"到某个规模突然会了"。
- 为什么重要：今天的推理模型本质是 CoT 的延伸，区别在于思考链由模型自己生成，而不是人写进 prompt。
- 课末问答的补充：CoT 最初是被"发现"而非"设计"出来的（GSM8K 上先见苗头，PaLM 上变得显著，当时还能解释笑话）；但现在的推理模型是被专门训练去思考的，不能再简单归为涌现。趋势是模型自己判断什么时候需要长思考。

3. 从 GPT-3 到 ChatGPT：后训练流水线

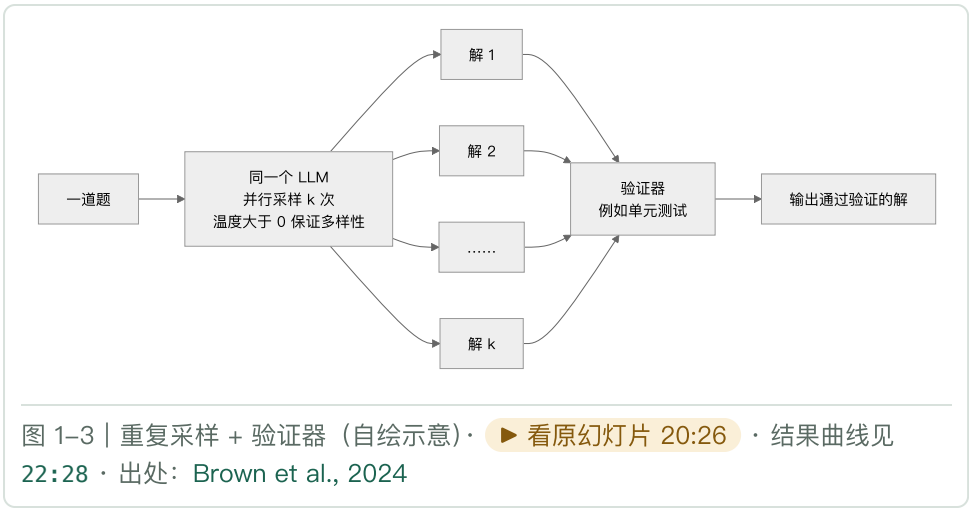


1. **预训练**：海量文本上做 next-token prediction。模型只有统计意义上的世界知识，不会听指令，也没有对错观。
2. **精选数据继续训练**：目标函数不变，数据换成高质量来源（书籍、优质文章等，厂商为此付出高额成本）。
3. **指令微调**：指令 + 问答对，含带推导过程的样本；数据是人工、模板、合成的混合。数据质量和覆盖面直接决定这一步的产出。
4. **RLHF**：人对模型的多个回答做偏好标注 → 训练 reward model 替代人 → 用 RL 把模型往 RM 认可的方向推。Reward 可以是多维的（正确性、有用性、具体性、无害性）并按需要加权。

对齐（让模型遵循人的目标、偏好、价值）至今没有被真正解决；课上的图显示，针对 sensibleness / safety 的精选数据微调能明显拉高对应指标。

4. 推理时计算：第三条轴

Large Language Monkeys (Azalia 实验室, 2024)：同一道题并行采样 $1 \rightarrow 10,000$ 次，用验证器（例如单元测试）挑出正确解。



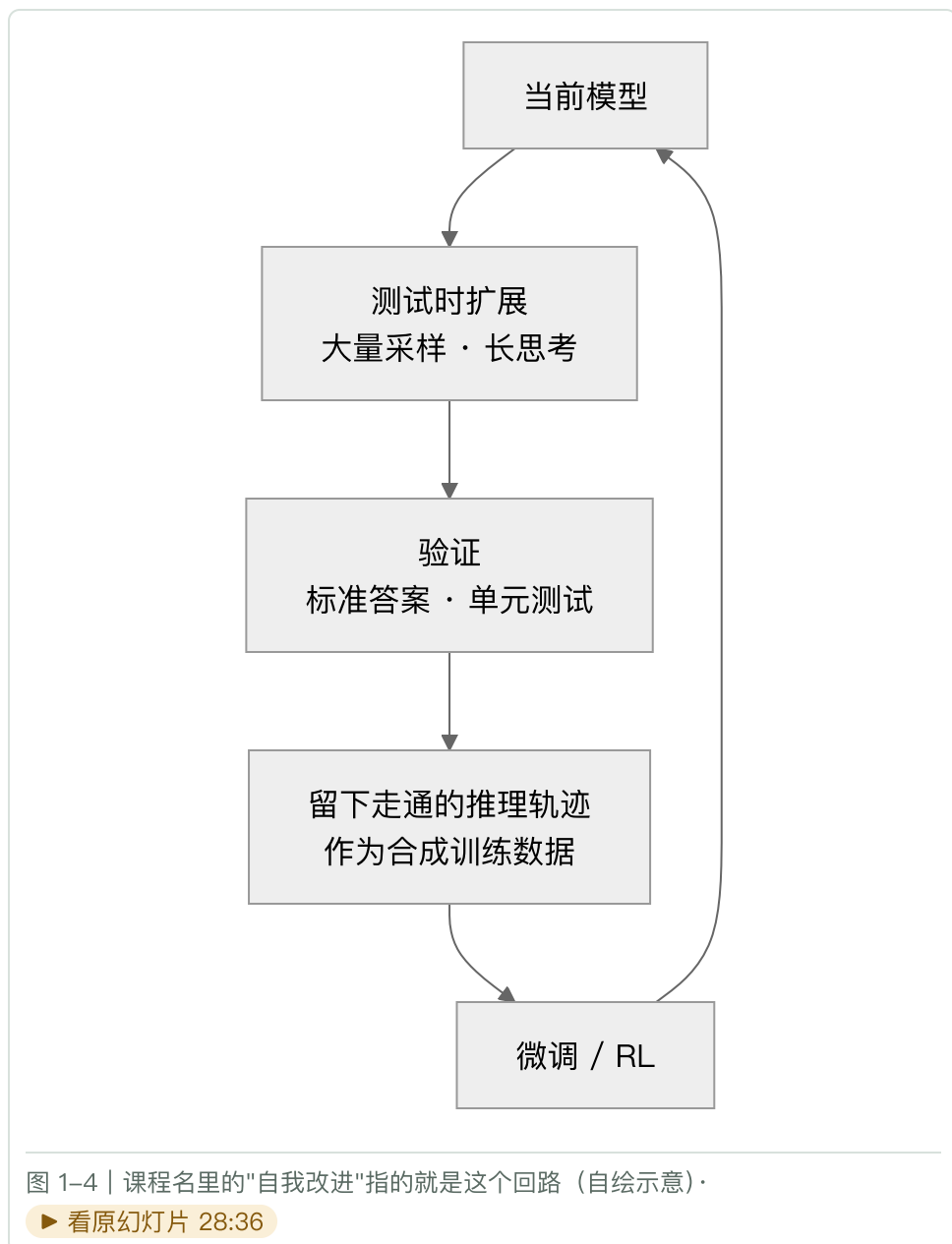
- 指标是 coverage：至少有一个样本做对的题目比例（即 pass@k）。
- 结果：Llama-3 8B / 70B 单次采样不如 GPT-4o，但采样数上去后在数学、代码基准上反超。有的题 10,000 个样本里只有三四个是对的。
- 含义：模型“会的”远多于单次采样表现出来的。不动参数、只改推理方式就能换来能力，所以叫 inference scaling。Coverage 随样本数呈对数线性增长。

问答要点：

- **为什么不是暴力搜索**：候选解由模型生成，样本效率远高于在全空间随机搜索或树搜索；MiniF2F 里有 IMO 难度的题，小模型一万次采样就能碰到正解。
- **延迟**：并行采样对延迟影响小，真正的权衡是算力成本，并且随问题类型和难度变化。
- **没有验证器的领域**：要靠 LLM-as-judge、训练 reward model、工具辅助验证——第 3 讲专门讲。
- **温度**：不能无限调高，约 1.2 以上输出开始崩坏；增加多样性另有技巧。

- **按难度动态分配采样数**：当时没有直接对应的已发表工作，可以用 reward model 的信号来引导——被点名为值得做的方向。

5. 自我改进回路 = 测试时扩展 × 微调



- DeepSeek、o1 系列、Gemini Thinking 的共同点：把测试时扩展当**数据引擎**。对答案可验证的题（数学有标准答案、代码有测试）大量采样，留下走通的推理轨迹，作为合成数据再微调 / RL 回模型。
- 这个回路没有明显的天花板，是课程名 "self-improving" 的所指。
- o1 的图：AIME 上 pass@1 准确率随测试时算力（对数轴）线性上升——这种关系此前只在训练算力上见过。

- **pass@k 与 pass@1** (Aakanksha 的解释): base model 重复采样时能出对的, 但不知道哪个对 (pass@k 高); train-time / test-time scaling 做的事, 本质是让模型学会选中对的那条, 把 pass@k 兑现成 pass@1。

小注: 课上说 DeepSeek 是 2024 年 12 月发布, 对应的是 V3; 推理模型 R1 发布于 2025 年 1 月。

6. 推理模型在"想"什么

- 环节: 问题分析 → 任务分解 → 试探并吸收反馈 (跑测试、用计算器、自评) → 自我纠错 → 走不通就回溯换方案。
- 例子: 让 o1 写一个做矩阵转置的 bash 脚本, 思考轨迹里依次出现需求分析、输入输出格式确认、实现步骤分解、中途发现问题并修正。
- 能力来源: 一部分来自人工策划的数据 (bootstrapping, 如指令微调里带推导的样本), 更大一部分是在合成数据上微调 / RL 的过程中习得并泛化的。
- 能力边界: o1 在数学、数据分析、编程上明显强于 GPT-4o, 个人写作、文本编辑并不占优。

问答要点:

- 思考 token 虽然贵, 但确实在帮助解题; 起作用的不是"说出来"这个动作, 而是训练把模型变成了通用的思考者。
- 能否用别的模型来生成推理轨迹: 目前模型明显更偏好自己的轨迹, 即使对方更强; 但让另一个模型做评估和反馈是可行的 (后面讲 SWiRL 时展开)。
- 怎么教会推理: 没有公开的完整配方; 课程会讲 ORM (结果奖励) 与 PRM (过程奖励)。

7. 从 LLM 到 Agent

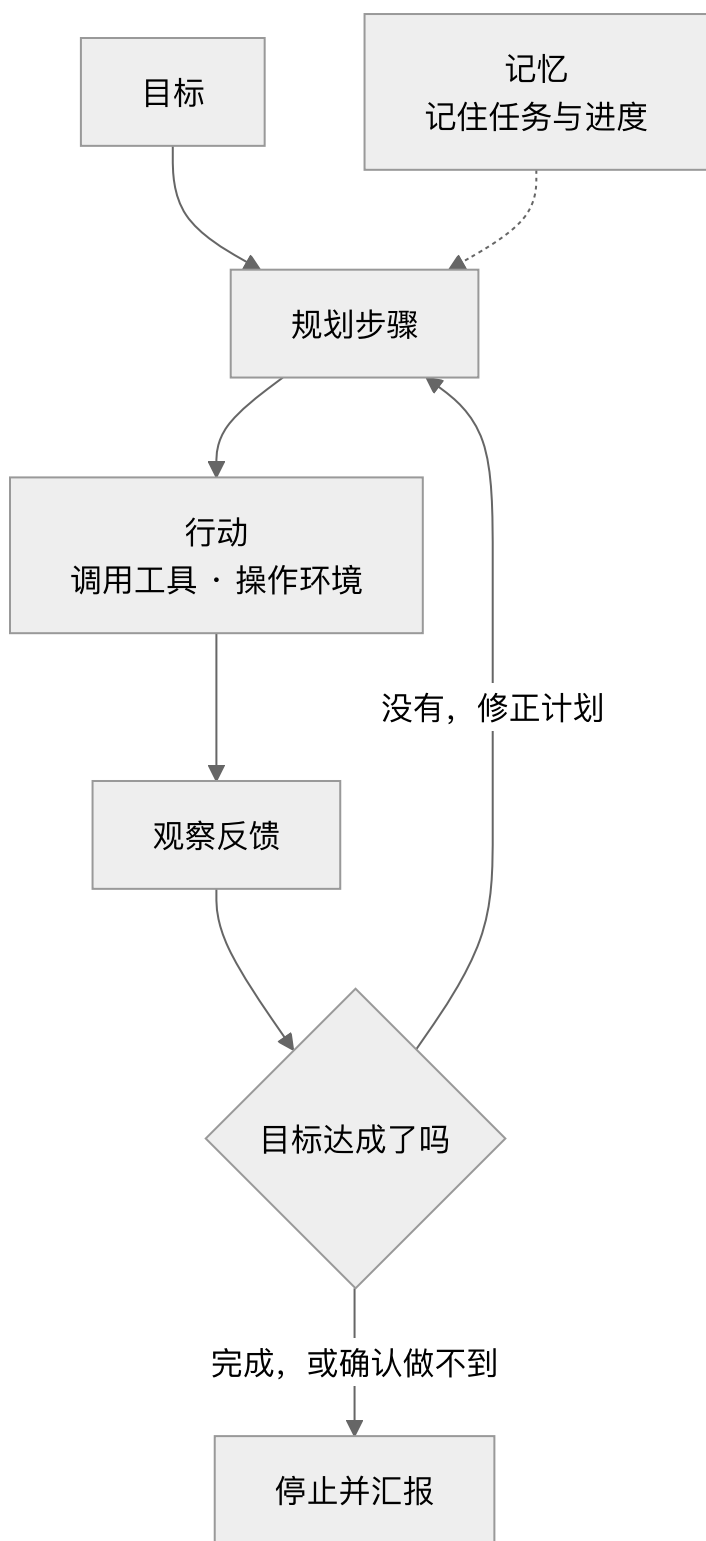


图 1-5 | Agent 与聊天模型的区别：带反馈、自己决定何时停的循环（自绘示意）·

► 看原幻灯片 42:20

- 聊天模型和推理模型仍是“回答问题”，并不替你完成任务。2025 年的变化：Claude Code、Codex、Deep Research 让端到端的真实 workflow 变得可用。

- Agent 的要素：有目标 → 自己规划步骤 → 与环境交互（调用工具）→ 根据反馈修正 → 自己决定何时停止（完成，或承认做不到）；长任务需要记忆来保持不跑偏。
- 现实：多数落地系统是**人手画好图的 agentic workflow**，而不是完全开放的循环；开放循环目前只在 coding 和 research 两个领域看到可用的迹象。
- 组件：LLM 调用、verifier（可执行的检查，如单元测试）、critic / judge（LLM-as-judge）、工具调用（搜索等）。
- 编排模式：prompt chaining（串行分解）、routing（按难度或类型分流）、parallelization（并行后聚合，Deep Research 的形态）、orchestrator（中心 LLM 先做计划再派发，如 Claude Code 的 plan）、evaluator（生成—评审循环）。这套分类和 Anthropic 《Building Effective Agents》基本一致。
- 对模型的新要求：规划、多步推理、自我纠错——正是后续各讲的主题。

8. Coding agent 为什么今年才可靠

- 循环本身（读仓库、搜文件、改代码、跑命令、看输出、再改）和去年相比没怎么变。
- 讲者给的原因：更强的基座 + RL with verifiable rewards 起效；模型变强后还会触发自我改进回路——能自己生成测试，测试更可靠，验证也就更可靠（CodeMonkeys 的思路）。
- **Generator—verifier gap**：生成容易，判断生成物好不好难。创意写作这类领域只能靠人给反馈，人就成了瓶颈；有可靠反馈的领域才能持续改进。稳健验证仍是整个方向的瓶颈之一（第 3 讲，含 Azalia 实验室组合多个弱验证器的工作）。
- 一个没有共识的问题（学生提出）：如果预训练后的模型多采样总能出一个对的，RL 只是把 pass@k 变成 pass@1，为什么提升会这么大？讲者的回答：RL 到底是“激发”还是“增加”能力，目前没有定论；至少有迹象表明持续做 RL 能持续提升。
- 容易被忽略的一环：**澄清用户意图**。用户通常说不清需求，agent 得先弄清楚要找什么、用什么标准验证，才谈得上“完成”。

9. 落地场景

- **代码**：迁移、版本升级、重构、数据工程（抽取 / 清洗 / 数仓迁移）、补单元测试——重复性高、容易验证。
- **客服**：实时转写、知识库辅助、智能回复、通话总结；端到端自动化刚起步。
- **研究报告**：选参考资料 → 拟大纲 → 逐篇摘要 → 合成长文（课上的例子应出自 STORM 一类系统，作业会用到）。
- **AI Scientist**：想法生成 → 实验迭代 → 论文写作。即使有幻觉，作为跳出个人经验的头脑风暴来源也有价值。

关键图表速查（点时间戳跳到原幻灯片）

图	看什么	跳转	出处
Scaling laws 三联图	横轴分别是算力、数据量、参数量，纵轴 test loss 一路下降	3:02	应出自 Kaplan et al., 2020
模型规模增长	2018—2024 参数量的指数增长	4:11	—
CoT 随规模涌现	LaMDA / GPT / PaLM 三组：小模型吃不到 CoT 的好处	9:13	应出自 Wei et al., 2022
重复采样的 coverage 曲线	红色虚线是 GPT-4o 单次采样；小模型的线在采样数上去后穿过它	22:28	Brown et al., 2024
o1 的 test-time compute 曲线	AIME 上 pass@1 对测试时算力（对数轴）近似线性	30:39	OpenAI o1 发布博客
o1 的思考轨迹	需求分析 → 分解 → 自我纠错的实际文本	32:40	同上
o1 vs GPT-4o 分领域对比	推理类任务占优，写作类不占优	34:46	同上
两种 agentic workflow 图	生成 + 评审；并行 + 聚合	43:52	—
Coding agent 循环	agent、终端、仓库之间的交互	47:56	—
研究报告 agent 流程	选资料 → 大纲 → 摘要 → 成文	56:05	应出自 STORM
AI Scientist 三阶段	想法生成 / 实验迭代 / 论文写作	57:05	Lu et al., 2024

课程安排（对自学有用的部分）

- 3 次作业（50%）+ 课程项目（50%）；项目 1—4 人，提供 API 额度。

- 认可的项目：新评测集 / benchmark；现有 agentic 系统的可靠性研究；在某个 benchmark 上爬分；改进或质疑课上某篇论文的设计决策。不接受综述，也不接受只拼一个 app——必须有假设、有问题、有实验。
- 节奏：10 月初 proposal → 两周后 midterm（要有实验进展）→ 期末报告（权重大）→ 12 月 12 日海报展。
- 每讲配套论文列在课程主页，建议配合阅读；上一届有项目最终发表成论文。

提到的工作

名称	在本讲里的作用
Scaling laws（三联图应出自 Kaplan et al., 2020）	预训练三轴的可预测性
BERT / GPT-2 / GPT-3 / PaLM / GPT-4	规模演进、few-shot、涌现
Chain-of-Thought prompting（对比图应出自 Wei et al., 2022）	随规模涌现的推理能力
GSM8K	最早显示推理链有效的数据集
Large Language Monkeys（2024）	重复采样的 inference scaling
OpenAI o1 / DeepSeek / Gemini Thinking	推理模型；测试时扩展 + 合成数据训练
SWiRL	多步推理的合成数据 + 分步 RL（后续展开）
CodeMonkeys	自生成测试来做验证
The AI Scientist	研究全流程 agent
Claude Code / Codex / Deep Research	2025 年可用的端到端 agent

术语对照

English	中文
scaling laws	扩展定律
emergent behavior	涌现行为
few-shot / zero-shot learning	少样本 / 零样本学习
chain of thought (CoT)	思维链
instruction tuning	指令微调
RLHF	基于人类反馈的强化学习
reward model	奖励模型
inference-time / test-time scaling	推理时（测试时）扩展

English	中文
repeated sampling	重复采样
coverage, pass@k	覆盖率 (k 次中至少一次做对)
pass@1	单次通过率
verifier	验证器
LLM-as-a-judge	LLM 评审
ORM / PRM	结果奖励模型 / 过程奖励模型
task decomposition	任务分解
self-correction / backtracking	自我纠错 / 回溯
agentic workflow	智能体工作流
prompt chaining / routing / parallelization	提示链 / 路由 / 并行化
orchestrator / evaluator	编排器 / 评估器
generator-verifier gap	生成-验证差距
RL with verifiable rewards	可验证奖励的强化学习

字幕勘误（开 CC 看时会遇到）

"scaling loss" → scaling laws；"Cloud Code" → Claude Code；"normal 3HP and 7AB" → Llama-3 8B / 70B；"math f to f" → MiniF2F；"aiming benchmark" → AIME；"Swirl" → SWiRL；"white coding" → vibe coding；"Edson" → Ed（课程论坛）；"cs239a" → cs329a。

带走的问题

1. RL 是在"激发"预训练里已有的能力，还是在"增加"新能力？ pass@k → pass@1 的解释能覆盖多少现象？
2. 没有验证器的领域（开放式对话、创意、消费级产品体验）怎么搭自我改进回路？ 反馈从哪来、怎么不被人力卡住？
3. "澄清用户意图"在 agent 循环里应该放在哪一步、用什么信号判断已经澄清够了？

CS329A 第 2 讲 | 测试时计算扩展 (Test-Time Compute Scaling)

Stanford CS329A: Self-Improving AI Agents (2025 秋) · 对应课表第 2 次课 (9 月 26 日) 视频: https://www.youtube.com/watch?v=-Ggc37xLj_Y (1:03:21, 自带人工英文 CC; 学生提问收音差, 字幕里有不少 [INAUDIBLE]) 讲者: Azalia Mirhoseini (推断: 本讲没有自我介绍, 但她把 Large Language Monkeys、KernelBench、Weaver、Archon 都称作自己实验室的工作) 配套阅读: Large Language Monkeys (Brown et al., 2024) · Scaling LLM Test-Time Compute Optimally... (Snell et al., 2024) · Archon (Saad-Falcon et al., 2024) · How Do Large Language Monkeys Get Their Power (Laws)? (Schaeffer et al., 2025; 讲者未点名, 按内容推断)

一句话: 不改参数、不做微调, 只在推理侧多花算力, 就能可预测地换来能力——重复采样的 coverage 随样本数走幂律, 根源是题集里难题的长尾; 真正的瓶颈是把对的样本挑出来 (generation-verification gap)。并行采样之外, 还有串行修订、PRM 引导的搜索, 以及 Archon 这种把各类推理时算子当成"层"来做架构搜索的做法。

时间轴

时间	内容
0:06	开场: 预训练 / 微调 / 推理三阶段, 本讲只动推理侧
1:06	Large Language Monkeys 回顾: 重复采样 + 验证器
3:06	SWE-bench 上同样成立; 单元测试让整条链路全自动
5:15	推理侧的 scaling law: coverage 与样本数的指数化幂律
7:19	为什么是幂律: 单题指数、全集幂律, 条件是难题长尾
11:30	算力账本的变化: 推理可以大笔投入, 还能离线花
12:30	自动验证: 形式化证明、单元测试、AI as compiler (KernelBench)、语言互译
15:34	没有验证器的领域: majority voting / reward model 与 coverage 的差距
19:42	讨论一: 验证器质量、先自学再采样、只筛错、集成验证器 (Weaver)、coverage 是否虚高
26:55	Snell et al.: 并行采样 vs 串行修订
28:57	ORM 与 PRM; best-of-N; PRM 引导的 beam search

时间	内容
33:02	并行 × 串行的组合；现成 PRM；实验设置（MATH、PaLM、难度五档）
36:04	结果：voting < ORM < PRM < compute-optimal；难度 × 串并配比
38:08	测试时算力 vs 预训练：易题、中等题换得动，最难的题换不动
39:09	问答：推理每次都要花钱、比例的含义、小模型能否练成专家
42:22	讨论二：tree search、易题偏串行 / 难题偏并行的直觉
45:24	Archon：把推理时扩展当成架构设计问题
47:26	算子：generator、fuser、critic、ranker、verifier
50:38	Fusion 实验：融合超过 oracle 选择；多模型集成
54:47	Unit test 的生成与"由模型来评估"
56:49	搜出来的架构；搜索空间的裁剪
58:53	叠层有收益；Bayesian optimization 与约束
1:00:53	结果：pass@1 平均超过 GPT-4o / Claude 3.5 Sonnet 14.1%

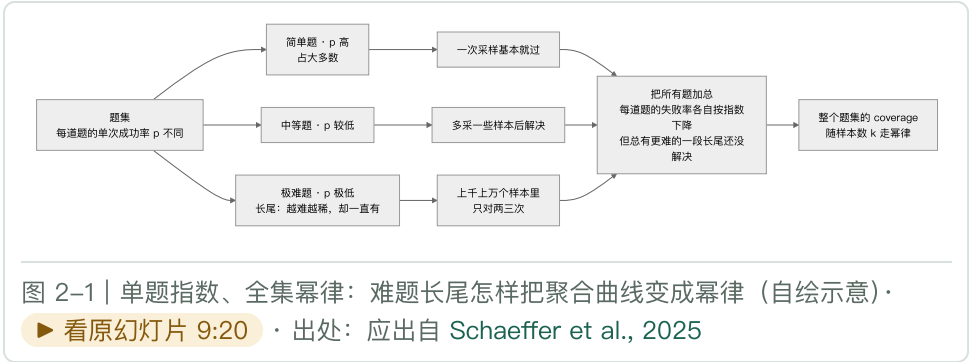
核心内容

1. 出发点：只动推理侧

- LLM 的三个阶段：预训练（数月、海量 GPU、万亿级 token）、微调（数据量小几个数量级）、推理。本讲的所有方法都不改参数、不做微调，只改"怎么用模型"。
- 回顾 Large Language Monkeys：同一道题反复采样（10 次、100 次……），验证器挑出对的那个作为系统输出。Llama-3 8B / 70B 单次不如 GPT-4o，采样数上去后在高难数学、代码和多类问答上反超。
- 讲者的解读：小模型其实已经"会"这些题，只是第一次不说；重复采样是在把答案引出来。
- 这一范式在他们试过的领域几乎都成立，包括 SWE-bench 这类 agentic 基准：样本数 1 → 1,000，DeepSeek（讲者印象中是 V3）的 coverage 超过 Claude 3.5 和 o1-preview。代码任务有单元测试自动选样本，于是"开源模型 + 采样 + 测试"就是一条全自动造出更强系统的路。

小注：Large Language Monkeys 原论文的 SWE-bench Lite 实验用的是 DeepSeek-Coder-V2-Instruct、250 个样本（15.9% → 56%）；课上这张 1,000 样本、对比 o1-preview 的图应是后来更新的版本，讲者自己对模型版本也不确定。

2. 推理侧也有 scaling law，根源是难题的长尾



- 预训练 scaling law 让人按数据、算力、参数量预测 test loss。推理侧的对应物：coverage 与并行样本数 k 服从 exponentiated power law，两个系数靠曲线拟合得到。
- 验证范围：Llama-3、Gemma、Pythia 等模型族，70M—70B 参数，多个领域；拟合曲线大体贴合实测，连 70M 的模型也不例外。
- 用处是做预算：要达到某个 coverage，需要多少样本、多少资源，可以事先估。

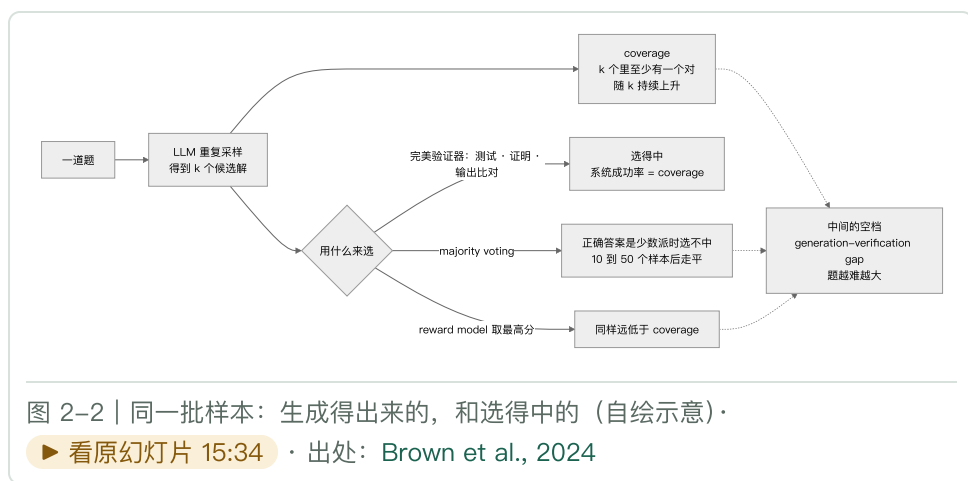
为什么是幂律（应出自后续工作 *How Do Large Language Monkeys Get Their Power (Laws)?*, Schaeffer et al., 2025；见图 2-1）

- 单看一道题：单次成功率 p_i ，则 $\text{pass}_i@k = 1 - (1 - p_i)^k$ ，失败率随 k 指数下降。
- 整个题集的聚合曲线却是幂律。两者要对得上，充要条件是各题 $\text{pass}@1$ 的分布在极难一端拖着一条长尾：多数题一次就过，但总有一批 $\text{pass}@1$ 极低的题，越难越稀、却一直有。
- 实测的 $\text{pass}@1$ 分布确实如此。换句话说，幂律是“题目难度分布”的性质，不是“单题行为”的性质。

小注：Monkeys 论文里的形式是 $c \approx \exp(a \cdot k^b)$ ，即 $\log c$ 对 k 呈幂律。直觉推导：聚合失败率 $= E[(1 - p)^k] \approx \int f(p) \cdot e^{(-kp)} dp$ ；若密度 $f(p)$ 在 $p \rightarrow 0$ 附近像 $p^{(\beta-1)}$ ，积分就按 $k^{(-\beta)}$ 衰减。

含义：过去预训练花上亿到几十亿美元、微调少得多、单次推理几乎不计成本（一问一答）。现在推理成了可以大笔投入的第三项，而且能离线花——把 agent 放出去围着一个问题持续生成、持续改进。

3. 验证：采样之后的另一半问题



容易验证的领域

- 数学：形式化证明，证明工具逐步检查。
- 代码：单元测试。写测试通常比写完整程序容易得多，人来写也行。
- AI as compiler（讲者实验室在做、她本人很看好的方向）：让 LLM 从 PyTorch 源码生成 CUDA 这类贴近硬件的底层代码。验证天然存在——同样的输入，生成的 kernel 与 PyTorch 参考实现输出一致即可。KernelBench 就是这样的基准，coverage 同样随样本数稳定上升。
- 推广：任何两种语言之间的移植（Python / C++ ↔ Java）都能用“行为等价”来验。

小注：对比输出是在有限输入上做数值等价检查，工程上很强，但严格说不是完备证明；KernelBench 除正确性外还看相对 PyTorch 的加速。

没有验证器的领域：generation—verification gap（图 2-2 的下半部分）

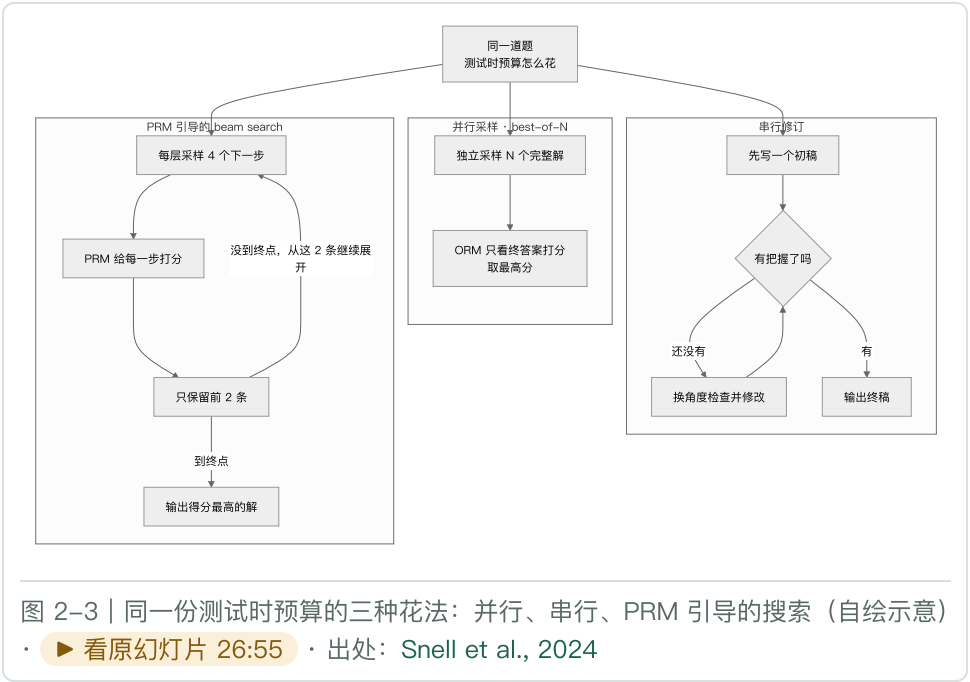
- 对比图：横轴样本数，纵轴系统最终成功率。Majority voting（取出现最多的答案）在 10—50 个样本后就走平；假设有完美选择器的 coverage 曲线还在涨。两线之间的空档就是 generation—verification gap——模型生成得出正确答案，系统却捞不出来。第 1 讲叫它 generator—verifier gap。
- 题越难空档越大：MATH 比 GSM8K 明显。
- 换成 reward model（给每个答案打分的 LLM）做 best-of-N，或把 RM 与投票结合，离 coverage 仍差一大截。
- 投票失效的原因很直接：最难的题在 1,000 甚至 10,000 个样本里只对一两次、两三次，正确答案是少数派。GSM8K 上多数题投票管用，

最难的那批照样如此。

4. 讨论一里值得留的点

- **Coverage 是不是虚高？** 数学题他们人工抽查过，真对的比例在 90% 以上（字幕此处数字含糊）；代码侧如果单元测试覆盖不全，“过了测试但其实不对”是真实的失败模式——又回到验证器质量。
- **验不出对，但筛得出错：**有些领域证明正确很贵，排除错误却容易；也有验证本身带噪声的（物理、分子动力学模拟）。讲者：视领域用模拟、工具或另一个模型来过滤。
- **很多验证器再投票：**正是 Weaver 的思路——弱监督地集成验证器，规模是 10–20 个而不是上千个，算力代价高。第 3 讲展开。
- **先自学再采样：**采样前先让模型探索问题域、沉淀成上下文，再进并行采样。讲者：self-study、搜索、工具使用都是重复采样之外的扩展手段。
- 每题 10,000 个样本的数据已放在 Hugging Face；“怎么缩小 generation—verification gap”被点名为合适的课程项目。

5. 生成方式 × 选择方式 (Snell et al., 2024)



论文：Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters。讲者只取了与测试时扩展最相关的部分，可以整理成两个正交的维度。

怎么生成

- 并行采样：同一问题独立出 N 个答案。
- 串行修订 (sequential revisions)：先出初稿，再反复修改、换角度检查，有把握了才输出。课上假设靠 prompt 驱动；推理模型则已经被训练成自己会这样做。

怎么选

- ORM：只看最终答案打分。它是学出来的，换到新领域准确率有限。
- PRM：对解答的每一步打分 (0-1)。粒度是"步"不是 token——一个有意义的推理块，或者干脆按句子切；训练数据可以来自人工逐步标注好 / 坏。PRM 一般从语言模型微调而来，域内最好用（在目标任务的子集上训练最有效），但因为底座是 LM，跨任务也有一定泛化。

组合

- Best-of-N：并行采 N 个，ORM 取最高分。
- PRM 引导的 beam search：每层预算 4 个样本，PRM 打分留前 2，从这两条继续展开；也可以改用分数阈值。本质是逐层剪枝，只有在希望的分支上花算力。
- 并行 × 串行：例如开两条修订链，各得一个终稿，再用 PRM / ORM 选；链内部也可以用 PRM 引导。
- 实用提示：现成的开源 PRM 很多，挑一个和任务最接近的直接用，或者用自己的数据训 PRM / ORM。

6. Snell et al. 的实验结论

- 设置：MATH (12k 训练 / 500 测试)，PaLM 系模型；自训 PRM，并专门微调了一个修订模型（讲者：如今只要是指令微调过的模型，让它改它就会改）。
- 难度定义：对每题多次采样，按模型自己的通过率分 5 档——难度是相对模型而言的。
- 预算—准确率曲线：majority voting < ORM < PRM < 精心混合的 compute-optimal 策略。目标是"达到某个准确率所需的最小生成预算"；怎么混合修订与并行，至今是开放问题。
- 串行：并行的配比：容易的题几乎全押串行最好；难题上最优配比说不清，从第 4 档到第 5 档都会变。
- 测试时算力 vs 预训练：按"推理 token : 预训练 token"的比例来比，易题和中等题把算力花在推理上更划算；最难的题仍是更大、预训练更多的模型占优。讲者说这与实验室的经验一致——小的开源模型配

上测试时算力越来越能打，但极难的问题上，前沿模型在任何合理的推理预算下都还是更好。

问答要点：

- 预训练只花一次、推理每次都花，这样比公平吗？讲者承认这一点，但认为这组结果的价值在于回答“预训练是不是做到头了”——没有；同时不是谁都训得起大模型，对多数人、多数问题，测试时扩展是够得着的手段。
- 图上的比例是预训练算力与推理算力的相对比值，不是“每个任务训一次”的一比一。
- 小模型能不能练成难题专家？针对特定题集微调总是可以的，但这里比的是通用数据、通用配方。
- 讨论二：tree search 就是并行与串行的混合，树结构还能复用计算，对应前面的 PRM 逐层剪枝。学生的直觉——易题条条路都通，顺着一条改就行；难题可行路径稀少，需要更多并行探索——讲者认可。她的抽象：生成的 token 是一个调质量的旋钮，问题是怎么分配，以及怎么把这些生成引出来。

7. Archon：把推理时技术当成“层”来搜

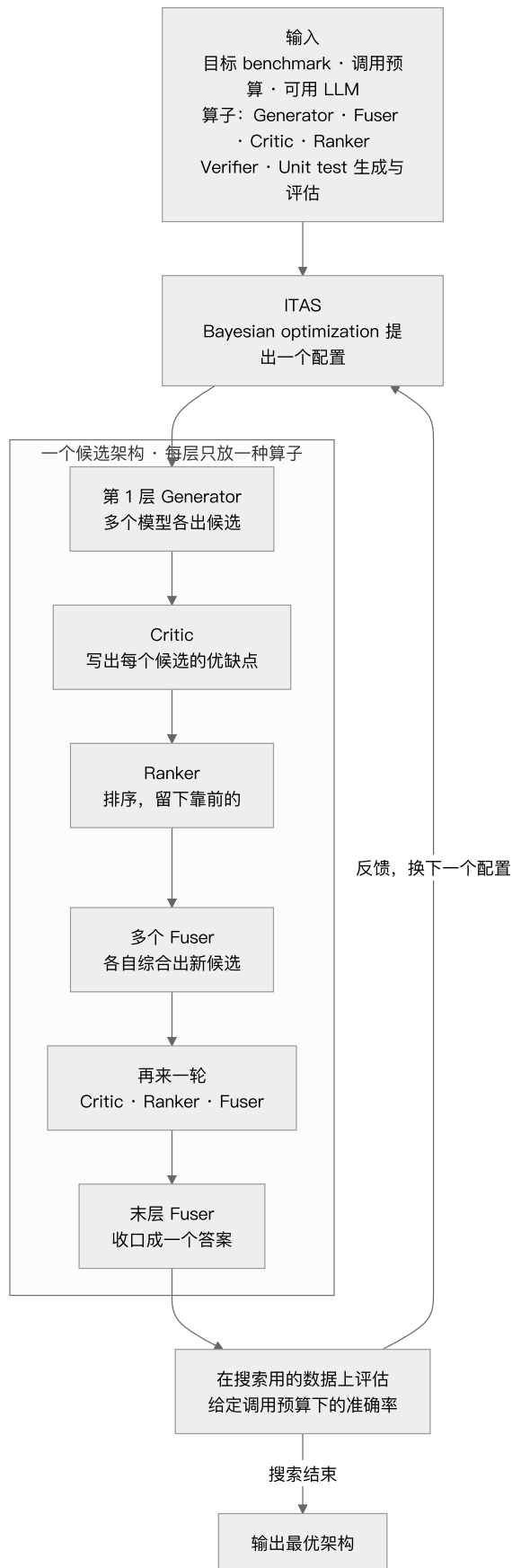


图 2-4 | Archon: 外圈是 ITAS 的搜索循环, 里面是被搜索的分层架构 (自绘示意)

· ▶ 看原幻灯片 46:24 · 出处: Saad-Falcon et al., 2024

论文：Archon: An Architecture Search Framework for Inference-Time Techniques（讲者实验室，课程 TA 是共同作者之一）。

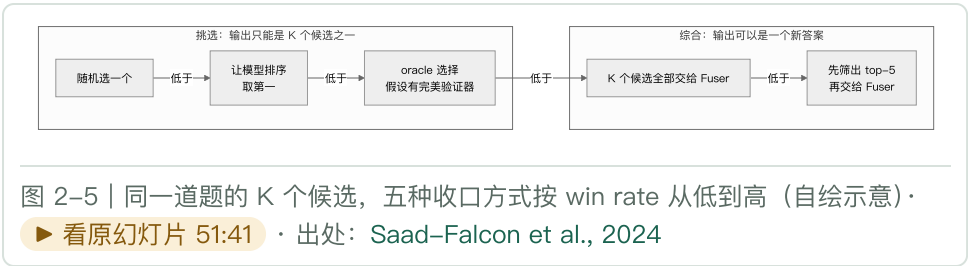
- 问题：各种推理时技术怎么混搭，才能把"正确率—成本"前沿推到最好，而不浪费 token？思路是把它当成架构设计问题。
- 输入：目标 benchmark、推理调用预算、可用 LLM 集合、推理时技术集合。优化器：ITAS（Inference-Time Architecture Search）。输出：一套把模型和技术组合起来的分层架构。

算子全部靠 prompt 实现，没有为这些角色训练过模型：

算子	做什么
Generator	从模型采样，n 次生成就是 n 个候选
Fuser	把问题和 K 个候选一起交给 LLM，让它综合出一个新答案（讲者归为串行更新）
Critic	列出某个候选的优点和缺陷
Ranker	给候选按质量排序
Verifier	判断候选是否成立，并给出理由
Unit test generator / evaluator	让模型为题目写检验条件；再让模型（而不是执行器）逐条对照候选

Unit test 的例子（括号匹配题）：奇数个括号必须输出 no；闭括号必须与最近一个尚未匹配的开括号配对。也可以让模型把测试代码一并写出来。

8. Fusion 实验：综合比挑选更强



实验在某个问答 / 推理基准上比 win rate（讲者当场没想起是哪个）。

- 单模型采 1—10 个样本：随机选 < 模型自己排序取第一 < oracle 选择 < 全部 fuse < 先筛出 top-5 再 fuse。
- 最反直觉的一点：fusion 超过了 oracle selection。挑选类方法的上限是"最好的那个候选"，综合却可以写出比任何单个候选都好的答案；先筛后融更好，说明过滤仍有价值。

- 换成多模型集成（按强弱依次加入，每个模型出一个答案）：随机选会被弱模型拖低，但各方法的相对次序不变。负责融合的是最强的那个模型（讲者印象）。

9. Archon 搜出的架构、搜索约束与结果

- 通用任务上搜出的结构（见图 2-4）：多模型 generator 层 → critic → ranker → 多个 fuser → 再来一轮 critic / ranker / fuser → 最后一个 fuser 收口。代码任务则是：大量采样 → 生成 unit test → 评估。
- **深度有用**：多模型 + 三层 critic / fuser + 末层 fuser，在多数任务上明显好于“最强模型跑一次”和“最强模型采 8 次 + 一层 fusion”。讲者的类比：像深度学习里加层。
- 搜索很贵（每个配置都要跑大量推理调用），所以先离线做实验把空间砍小：每层只放一种算子；第一层固定为 generator；critic 必须排在 ranker 或 fuser 之前；unit test generator 后面必须跟 evaluator。
- 在专门划出一份数据上，以“给定调用预算下的准确率”为目标，用 Bayesian optimization 搜；比贪心和随机搜索更省配置数。目标可以换（可用模型、调用预算等）。
- 结果：Archon 最后只输出一个答案，所以比的是 pass@1。只用开源模型的组合就能追平或超过当时的闭源前沿模型；在 instruction following、推理、数学、代码几类任务上，pass@1 平均比 GPT-4o、Claude 3.5 Sonnet 高 14.1%。
- 既可以针对单个任务搜，也可以搜一个通用架构；通用版同样超过前沿模型——说明这类架构能泛化到调优任务之外。

小注：论文初版摘要里，14.1 个百分点对应开源 + 闭源模型都可用（all-source）的设置，只用开源模型是 10.3 个百分点，后续版本数字略有更新；课上的口径与此略有出入，引用时以论文为准。论文评测集包括 MT-Bench、Arena-Hard-Auto、AlpacaEval 2.0、MixEval、MATH、CodeContests。

关键图表速查（点时间戳跳到原幻灯片）

图	看什么	跳转	出处
重复采样的 coverage 曲线	横轴样本数、纵轴 coverage: Llama-3 8B / 70B 多采样后越过 GPT-4o 的单次成绩; SWE-bench 那张横轴到 1,000, DeepSeek 越过 Claude 3.5 和 o1-preview	2:06 · 3:06	Brown et al.
scaling law 拟合	预测曲线与实测 coverage 基本重合; Llama-3、Gemma、Pythia, 70M-70B 都成立	6:17	同上
单题 pass@1 的分布	按 pass@1 统计题目数: 一次就过的简单题占大头, pass@1 越低的方向拖着一条长尾——幂律的来源	10:22	应出自 Schaeffer et al.
voting vs coverage	绿 = majority voting, 10-50 个样本后走平; 蓝 = coverage (完美选择器) 继续上升; MATH 的空档比 GSM8K 大, reward model 的线同样远在蓝线之下	15:34	Brown et al.
PRM 引导的 beam search 示意	每层 4 个样本、PRM 留前 2 再展开的树; 对比 best-of-N 只在终点打一次分	29:59	Snell et al.
预算—准确率曲线	majority voting 是基线; 紫 = ORM, 绿 = PRM, 蓝 = 精心混合的 compute-optimal 策略	36:04	同上
难度 × 串并配比	1-5 档难度 (5 最难); 颜色 = 串行 : 并行配比, 最深的紫 = 几乎全串行, 在易题上最好; 难题上最优配比不固定	37:07	同上
测试时算力 vs 预训练	绿 = 易、蓝 = 中、橙 = 难; 按推理 token 与预训练 token 的比例来比: 易 / 中题推理侧划算, 最难的题相反	38:08	同上
Fusion win rate	橙 = 随机选, 绿 = ranker 取第一, 蓝 = oracle 选择, 红 = 全部 fuse, 紫 = 先取 top-5 再 fuse; 红线压过蓝线是看点; 右图换成多模型集成, 次序不变	51:41	Saad-Falcon et al.
Archon 搜出的架构	逐层看: 多模型 generator → critic → ranker → 多个 fuser → 再一轮 → 末层 fuser; 原图字很小, 对照图 2-4 看	56:49	同上
叠层消融	最右的粉色 = 多模型 + 三层 critic / fuser + 末层 fuser, 对比"最强模型一次"和"最强模型 8 次 + 一层 fusion"	58:53	同上
Archon vs 闭源前沿	只用开源模型的 Archon 对比当时的闭源前沿模型; task-specific 与 general-purpose 两版; 平均 pass@1 高 14.1%	1:00:53	同上

提到的工作

名称	在本讲里的作用
Large Language Monkeys (Brown et al., 2024)	重复采样、coverage、推理侧 scaling law
<i>How Do Large Language Monkeys Get Their Power (Laws)?</i> (应出自, Schaeffer et al., 2025)	单题指数 vs 全集幂律; 难题长尾是充要条件
预训练 scaling laws (第 1 讲)	推理侧 scaling law 的类比对象
SWE-bench	agentic 代码基准, 重复采样同样有效
Llama-3 8B / 70B、Gemma、Pythia、GPT-4o、DeepSeek、Claude 3.5、o1-preview	重复采样实验涉及的模型
KernelBench	PyTorch → CUDA kernel 生成; 对比输出即可验证
GSM8K / MATH	voting 与 coverage 的差距; MATH 也是 Snell et al. 的实验集
Weaver	弱监督集成多个弱验证器 (第 3 讲)
<i>Scaling LLM Test-Time Compute Optimally...</i> (Snell et al., 2024)	并行 vs 串行、ORM vs PRM、难度分档、与预训练的对比
PaLM (小注: 论文用的是 PaLM 2-S*)	Snell et al. 的基座模型
Archon (Saad-Falcon et al., 2024)	推理时架构搜索 (ITAS)
Bayesian optimization	ITAS 的搜索算法

术语对照

English	中文
test-time / inference-time compute	测试时 / 推理时计算
repeated sampling	重复采样
coverage (pass@k)	覆盖率
exponentiated power law	指数化幂律
long tail of hard problems	难题长尾
automated verification	自动验证
formal proof	形式化证明
majority voting	多数投票
best-of-N	N 选一
oracle verifier / oracle selection	理想验证器 / 理想选择
generation-verification gap	生成-验证差距
parallel sampling	并行采样

English	中文
sequential revision	串行修订
outcome reward model (ORM)	结果奖励模型
process reward model (PRM)	过程奖励模型
beam search	束搜索
compute-optimal	算力最优
inference-time architecture search (ITAS)	推理时架构搜索
generator / fuser / critic / ranker / verifier	生成器 / 融合器 / 批评器 / 排序器 / 验证器
ensemble	集成
inference call budget	推理调用预算
Bayesian optimization	贝叶斯优化
weakly supervised	弱监督

字幕勘误（开 CC 看时会遇到）

"test laws" → test loss; "exponential power law" → exponentiated power law（论文用词）; "Llama 3 8B, 7, 8B" → Llama-3 8B / 70B; "pass at one / past at one / paths at one / passage one" → pass@1; "RAC" → RAG; "weaver ... a week supervised ensembling" → Weaver, weakly supervised ensembling; "the palm model" → PaLM; "itest" → ITAS; "users" ("a bunch of users"、"ranker and user") → fusers / fuser; "Oracle" → oracle（理想选择器，不是公司）; "inference called budget / inference, core budget" → inference call budget; "GPT-4.0" → GPT-4o; "math encoding problems" → math and coding; "above like a 90s, 7%, 8%" 数字含糊，只能确定是 90% 以上。另: "capital MATH" 不是误识别，是讲者口头强调全大写的 MATH 数据集。

带走的问题

- 幂律来自难题长尾，而长尾上的正确样本恰恰是"万里挑三"的少数派——验证器不完美时，coverage 的可预测增长有多少能兑现成 pass@1?
- 串并配比随难度变，而且第 4 档到第 5 档就不一样：能不能在线估计难度、动态分配预算？（第 1 讲也点名过这个方向。）

3. Fusion 为什么能超过 oracle selection? 在有唯一标准答案的任务（数学、代码）上还成立吗，还是主要发生在开放式问答的 win rate 上？
4. 推理模型已经把串行修订内化进权重，Archon 式的外部编排还剩多少增益？架构搜索本身的推理开销该不该算进预算？

CS329A 第 3 讲 | 稳健的验证 (Robust Verification)

Stanford CS329A: Self-Improving AI Agents (2025 秋) · 对应课表第 3 次课 (9 月 29 日) 视频: <https://www.youtube.com/watch?v=p7TdPUcPoik> (1:12:59, 自带人工校对英文 CC) 讲者: Azalia Mirhoseini (最后一篇 Weaver 出自她的实验室) 配套阅读: Training Verifiers to Solve Math Word Problems (Cobbe et al., 2021) · Let's Verify Step by Step (Lightman et al., 2023) · Math-Shepherd (Wang et al., 2023) · Shrinking the Generation-Verification Gap with Weak Verifiers (Weaver, NeurIPS 2025)

一句话: 重复采样说明模型"做得出来", 难的是"挑得出来"。四篇论文是一条清晰的演进线: 结果级验证器 (ORM) → 人工标注的过程级验证器 (PRM) → 自动标注的 PRM 并反过来用于 RL → 不再训练新验证器, 而是用弱监督把一堆不完美的验证器组合成一个强的, 再蒸馏成小模型。

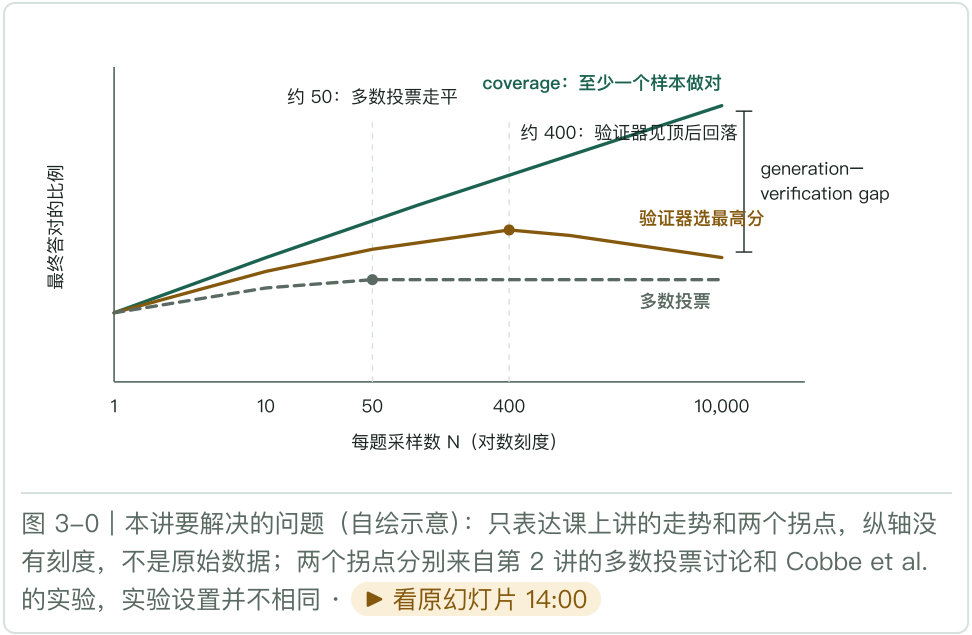
时间轴

时间	内容
0:06	引子: 从 generation-verification gap 到验证
1:08	论文一: Training Verifiers (OpenAI 2021) 与 GSM8K
3:14	验证器怎么训: 采样 100 个解、按终答案打标签、双目标、逐 token 打分
10:50	结果: 验证 vs 纯微调; 生成器与验证器的大小搭配
14:00	采样数到 400 之后反而下降; 问答
21:20	论文二: Let's Verify Step by Step——ORM vs PRM
25:52	PRM800K 与主动学习式的数据收集
29:00	结果: PRM > ORM > 多数投票; 标注效率; 分布外泛化
31:50	问答: PRM 会不会帮倒忙、跳步骤式的 reward hacking
37:50	论文三: Math-Shepherd——不用人的过程标注
40:50	课堂讨论: rollout 式标注的三个缺陷
43:25	用于验证和 PPO 的结果; 问答
51:48	论文四: Weaver——组合弱验证器
54:50	score → weight → select; 弱监督估计每个验证器的准确率
1:00:04	验证算力的几种扩法; 8B ≈ 70B、70B ≈ o3-mini
1:04:11	把整个验证器池蒸馏进 400M 小模型

核心内容

1. 为什么需要验证器

上一讲的结论是 coverage 随采样数一路上升，但多数投票这种"免费"的选择方法在几十个样本以内就不再进步——能生成的和能选中的之间的差距就是 generation-verification gap。验证要解决两件事：生成之后自动选出对的答案；或者在生成过程中给模型引导。



2. 论文一：Training Verifiers to Solve Math Word Problems (OpenAI, 2021)

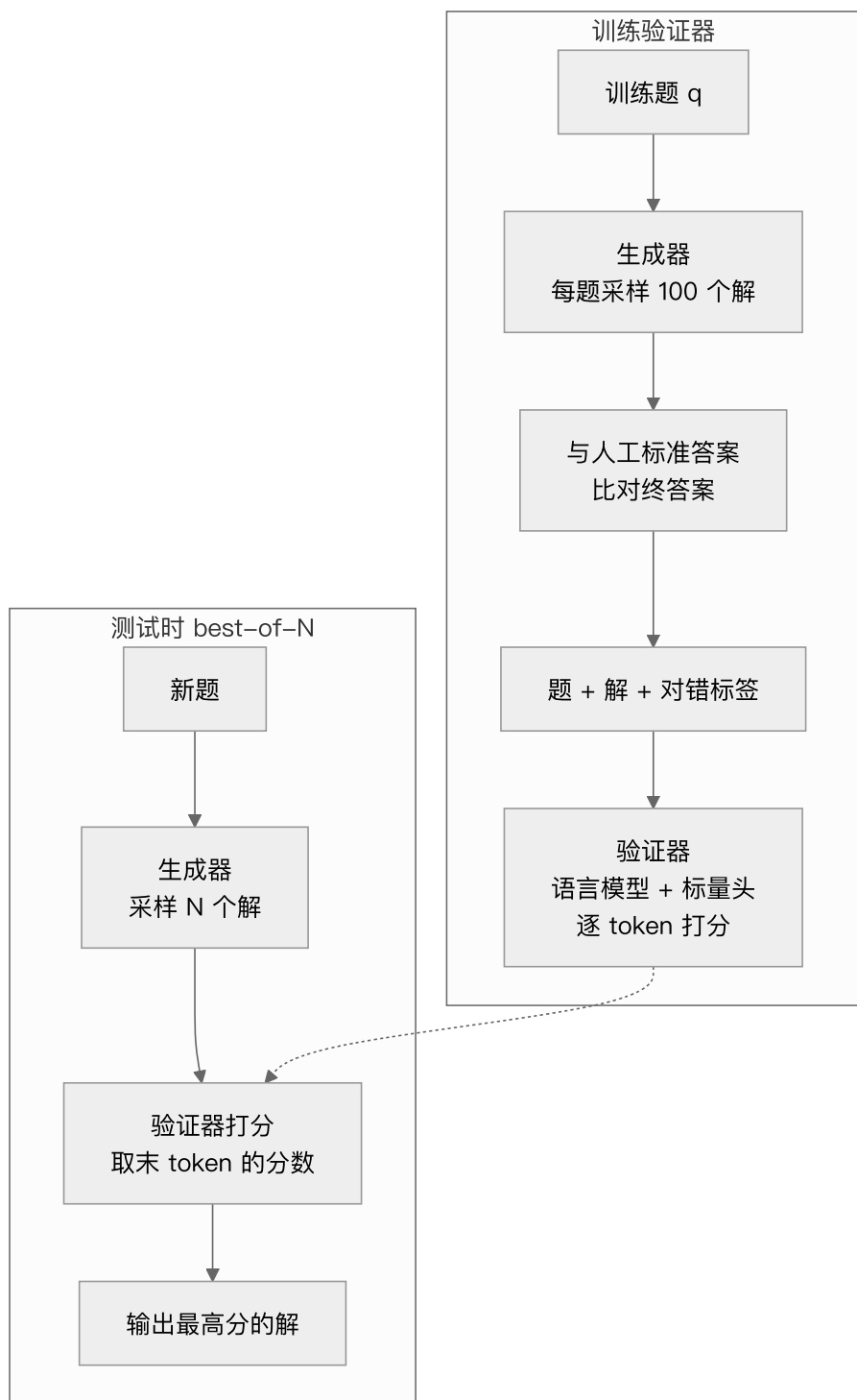
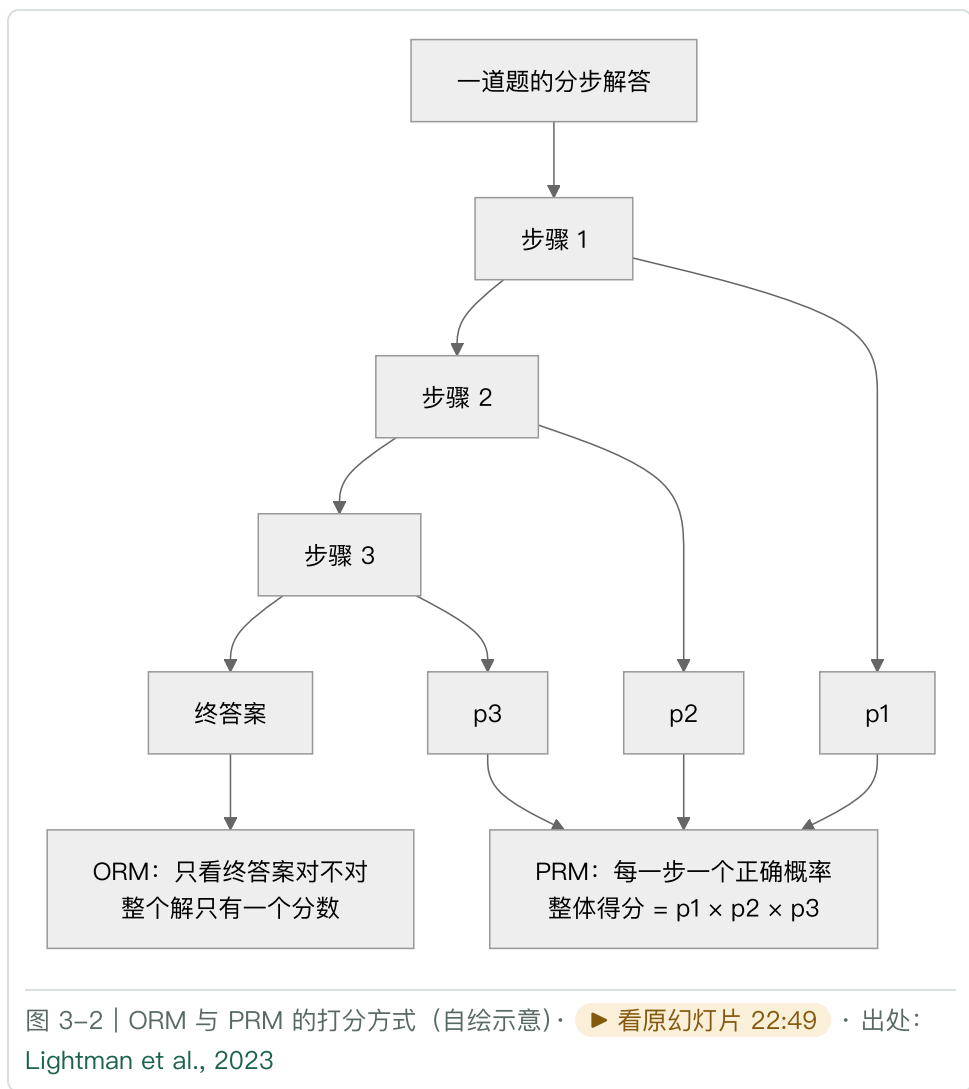


图 3-1 | 验证器的训练与使用（自绘示意）· [▶ 看原幻灯片 3:14](#) · 出处: Cobbe et al., 2021

- **GSM8K**: 约 8,500 道小学数学应用题，题目不难，但都需要几步推理，解答用自然语言写。至今仍是小模型常用的评测，做项目时可以直接拿来当 eval。
- **验证器**: 输入题目 + 解答，输出"这个解正确"的概率。

- **训练流程**：先把生成器在数据集上微调 2 个 epoch → 每题采样 100 个解 → 和人工标准答案比对终答案，得到对 / 错标签 → 用这批数据训练验证器 1 个 epoch。讲者的评论：今天的模型已经很会做数学和遵循指令，前面那步生成器微调多半可以省掉，新的验证器工作通常直接训练。
- **结构与目标**：验证器本身是语言模型，外加一个标量输出头，对解答的每个 token 都预测一次；题目部分的 token 不计损失。训练时同时用正确性预测和语言建模两个目标，论文报告两者一起用更好。推理时只取最后一个 token 的分数作为整个解的得分。可视化里能看到分数在出错的位置由绿转红。
- **结果**
 - 6B 和 175B 两个规模上，"采样 100 个 + 验证器选最高分"都优于"只做监督微调"，而且训练数据越多优势越大；数据很少时（175B 在约 1,000 题以下）验证器帮不上忙。
 - **大生成器 + 小验证器** 优于 **小生成器 + 大验证器**——符合"生成比验证难"的直觉。两者大小的 Pareto 最优配比被点名为值得做的研究题，尤其是现在基座强了很多、公开的 reward model 也很多（Hugging Face 上有排行榜）。
 - **采样数不是越多越好**：增加到约 400 个时收益最大，再多准确率反而下降。原因是样本越多，越容易出现"错误但验证器给高分"的解，验证器的分辨率跟不上。论文实际使用 100 个。对比：多数投票在 50 个样本以内就到头了。
- **问答**
 - 数据足够多时，直接微调和训练验证器会趋同；验证器的好处是不动基座，生成器保持通用。
 - 能不能对验证器本身做 test-time scaling：可以，Weaver 就是一种形式。

3. 论文二：Let's Verify Step by Step (OpenAI, 2023)

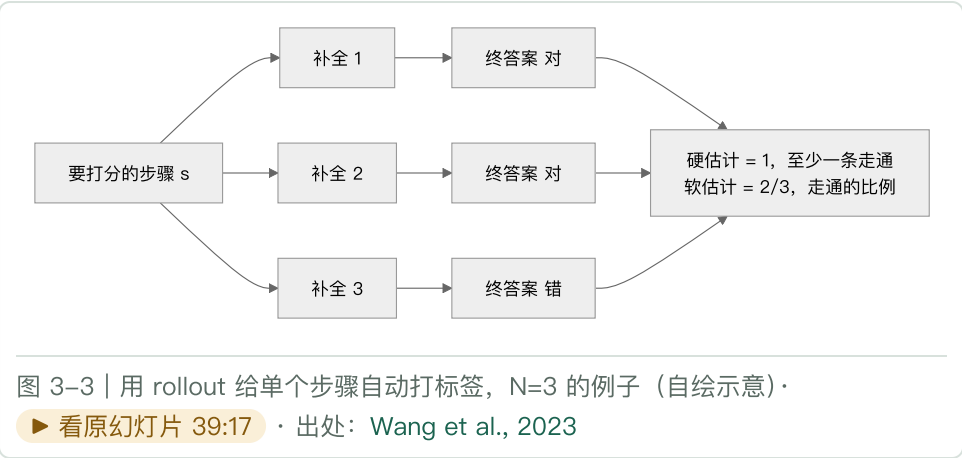


- **动机**: 分步解题时, 前面一步走错, 后面全盘皆错。
- **ORM vs PRM**: ORM 对整个解给一个奖励, 标签来自终答案比对; PRM 对每一步给奖励, 标签来自人工逐步标注 (正确 / 错误 / 中性)。PRM 的整体得分取各步正确概率的乘积; ORM 仍取最后一个 token 的分数。
- **过程监督的好处**: 信用分配更精确; 能压住 false positive——模型有时过程是错的, 终答案却碰对了, 只看结果会把这种解当成正例; 推理过程可解释, 并且是"人认可的"解法。讲者特别提醒, false positive 是 test-time scaling 里容易踩的坑。
- **PRM800K**: 开源的 80 万条步骤级标签。收集方式是迭代的主动学习: 用当前 PRM 挑出最值得标注的样本给人标, 再重训 PRM, 循环往复, 数据效率约为随机挑选的 2.6 倍。

小注: 课上对"convincing wrong-answer"的描述是"终答案对、中间步骤错"; 论文原文的定义是"被当前 PRM 打了高分、但终答案错"的解。两种都是 PRM 最容易判断失误的样本, 但方向相反, 以论文为准。

- **训练**：以 GPT-4 为基座，ORM 和 PRM 都先在数学数据上微调过。
- **结果**：best-of-N 下 PRM > ORM > 多数投票，N 越大差距越大；多数投票在 100 个样本左右失效。PRM 能从正确率不到 5% 的样本分布里把对的解捞出来。按标注量算 PRM 也更省：量级上，ORM 每题标 100 个解的效果，大约相当于 PRM 每题标 1 个解。分布外测试中，多数投票反而比 ORM 稳，PRM 最好，对分布偏移的容忍度最高。
- **问答**
 - PRM 会不会帮倒忙（某一步看着好，其实对答案没贡献）：较新的做法是 PRM 和 ORM 一起用；另外 PRM 的阈值是新引入的超参，需要调。
 - 两者的标注量很难对齐比较：ORM 每题 k 个标签，PRM 是 $k \times$ 步数，而步数不好控制。
 - 模型跳过推理直接写答案，能骗过 PRM 吗：只在测试时当选择器用，生成器没被改动，可以通过 prompt 要求它写出步骤，问题不大。真正的风险出现在拿 **PRM 当奖励去训练生成器** 的时候——模型可能学会少写步骤来讨好 PRM。人工标注可以把"跳步"标成差；自动标注时这一点要格外小心。

4. 论文三：Math-Shepherd (2023) ——不用人的过程标注

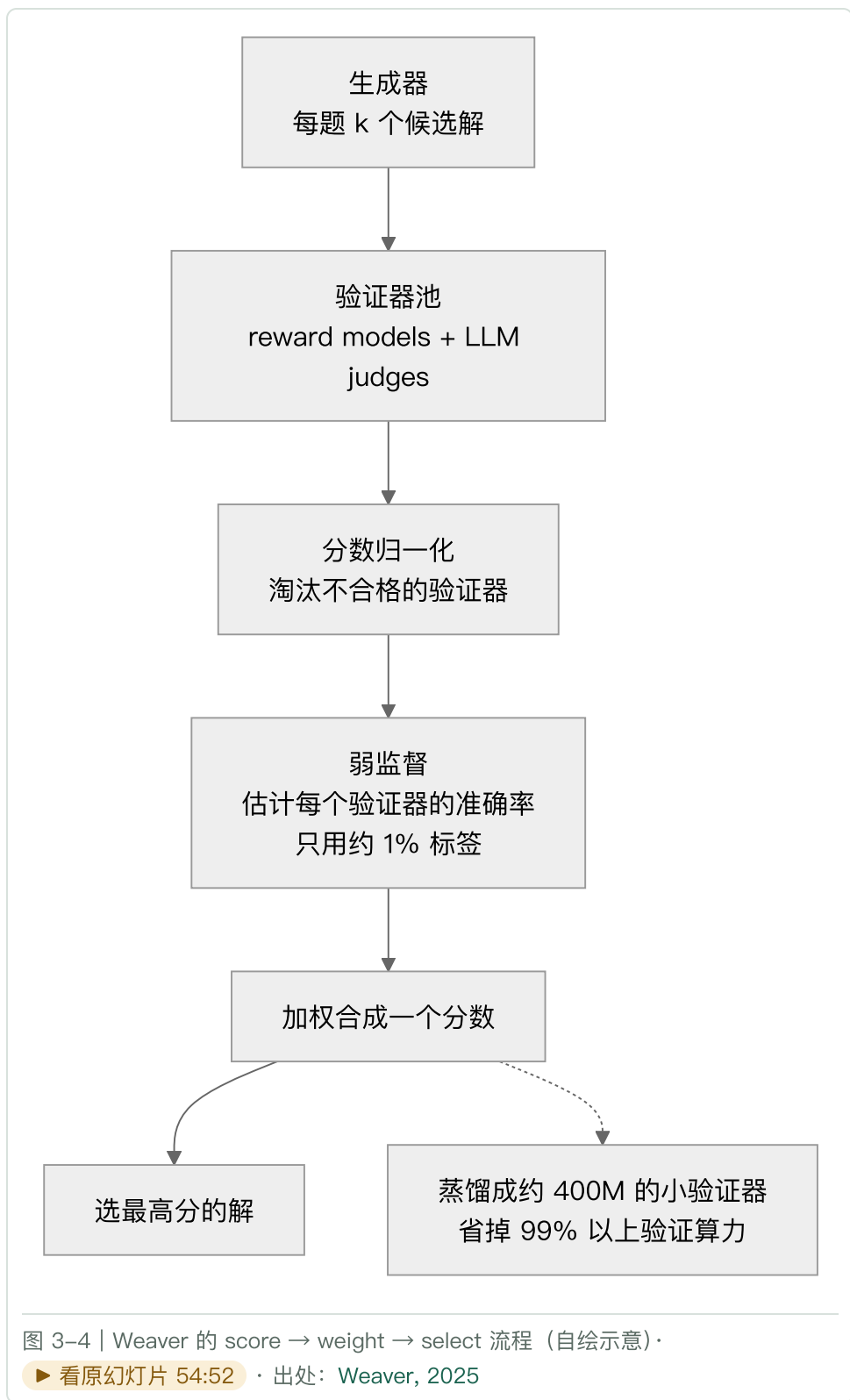


- **思路**：一个步骤好不好，用"从这一步出发，最终走到正确答案的潜力"来定义。从该步往后采样 N 条补全：
 - Hard estimation：只要有一条到达正确答案，这步记 1；
 - Soft estimation：到达正确答案的比例，就是这步的得分。
- **课堂讨论出的三个缺陷**：N 太小会漏掉少见但正确的路径，非常规解法会被低估；难题上怎么 rollout 都是错的，得不到任何信号；中间走

错、但最后碰对答案的路径，其错误步骤也会被标成正确。

- **两种用法**：测试时对 N 个候选解用 PRM 打分取最高；训练时把 PRM 当奖励，用 PPO 逐步强化生成器。
- **结果**
 - Hard 和 soft 两种估计在 N=4 时差别不大，最终选了更简单的 hard。
 - Math-Shepherd > ORM > self-consistency（多数投票的另一个名字）；在更难的 MATH 上优势比 GSM8K 更大，并超过了用 PRM800K 训练的验证器。结论在 LLaMA2-70B、LLemma-34B、DeepSeek-67B 上都成立。
 - Mistral-7B 用该 PRM 做 PPO，明显好于用 ORM 做 RL；RL 之后再叠一层验证还能涨，但已经开始趋于平缓。
- **和课程主题的关系**：模型自己生成标注 → 训练出 PRM → PRM 在测试时和训练时两头改进生成器，是多层的自我改进。
- **问答**
 - 怎么让 PRM 鼓励自我纠错：给打分步骤配 rubric，或者让它调用工具（计算器、SymPy）核对算式——验证本身也可以是 agentic 的、可以吃 test-time compute 的。
 - 固定总算力时，多采样生成器还是多花在验证器上：论文没有控制这个变量，是可以做的研究题。ORM 和 PRM 要公平比较，必须来自同一份数据。
 - 一个延伸想法：从 RL 之后的模型重新生成标签、再训一轮验证器，把循环继续转下去。

5. 论文四：Weaver (Stanford, NeurIPS 2025) ——组合弱验证器



- **出发点：**不训练新验证器，用推理时算力缩小差距。"弱"不是故意挑差的，而是承认现有最好的验证器也都不完美。候选池包括各家训练的 reward model (ORM / PRM) 和 LLM-as-judge (可带 rubric、可用工具)。

- **观察：**直接取 top-1 / top-5 / top-10 个验证器做平均，有帮助但不单调；一旦有标签、能给每个验证器学一个权重（朴素贝叶斯或逻辑回归，每个验证器只有一个参数），提升就很稳定。
- **Weaver 的流程 (score → weight → select)**
 1. 让所有验证器打分并归一化到同一尺度；
 2. 用极少量标签把明显不合格的验证器**过滤掉**——讲者强调这一步很关键，池子要有准入门槛；
 3. 用弱监督（思路来自 Snorkel）估计每个验证器的准确率。核心假设是各验证器在给定真实标签时相互独立，信号来自它们之间一致与不一致的模式；如果所有验证器永远给出同样的分数，就学不到任何东西；
 4. 按估计出的权重合成一个分数，选最高的解。
- **结果**
 - 相比朴素平均，在 GPQA Diamond、MATH、MMLU-Pro 这些难任务上提升更明显。
 - 曲线从上到下：Pass@K oracle（上限）> Weaver（全量标签）> Weaver（每个数据集只用约 1% 标签）> 朴素集成 > 多数投票 / Multi-Agent Verification。后者只靠 prompt 让 LLM 从多个角度打分，有两个数据集上还不如多数投票。
 - Llama 3.1 8B Instruct 做生成器 + 一池 8B 及以下的验证器，平均约 70%，追平 70B 模型的多数投票；换成 70B 级生成器 + Weaver，平均 86.2%，接近 o3-mini。这里报告的是整个系统最终选中答案的准确率，不是 coverage。
- **验证算力的扩法：**更多样本、更大的生成器或验证器、更多验证器。
- **蒸馏：**验证器池的开销很大（每个样本要过一遍所有模型）。把 Weaver 的合成分数蒸馏进一个约 400M 的小模型，能保住约 97% 的准确率，验证算力省掉 99% 以上；按“准确率—总推理算力”衡量，连未蒸馏的 Weaver 也比朴素集成和多数投票划算。权重已开源。

小注：论文摘要给的数字是保留 98.7% 的准确率、验证算力最多减少 99.97%。

6. 总结与最后的问答

- 验证在训练和推理两头都能提升质量；过程奖励总体比结果奖励有效，最好两者结合；验证器吃数据，数据越多越好，训好之后还可以

反过来用于 RL；Weaver 是另一条路——把 test-time scaling 花在"更多验证器"上，而不是"对单个验证器多采样"。

- **代码领域：**除了训练验证器，还可以让模型自己生成单元测试来当验证器（CodeMonkeys，后面会讲）。
- **推理模型还需要这些吗：**仍然受益。推理模型的训练本身就是"生成推理轨迹 → 奖励 → RL → 把正样本轨迹用于下一轮训练"，test-time scaling 已经内化成了造数据的环节；推理时多采样依然能探索解空间的不同区域。
- **重复采样最终会不会只用于造训练数据、测试时只问一次：**讲者希望如此（pass@1 足够强、效率最高）。但把分布压尖的代价是可能损失解法的多样性和创造性，而这恰恰是希望保留的性质。
- **生成器和验证器同源好还是异源好：**模型普遍更偏好自己的生成；讲者没见过系统比较"同家族同尺寸 vs 不同家族不同尺寸"的研究——又一个开放题。

关键图表速查（点时间戳跳到原幻灯片）

图	看什么	跳转	出处
验证 vs 纯微调	6B / 175B 两条线随训练集变大的走势；数据少时验证器不占优	10:50	Cobbe et al.
生成器与验证器的大小搭配	大生成器 + 小验证器那条线在上面	12:50	同上
采样数 vs 准确率	约 400 处见顶、之后回落	14:00	同上
逐 token 打分可视化	绿 → 红的位置就是验证器认为出错的地方	9:28	同上
PRM vs ORM vs 多数投票	三条 best-of-N 曲线随 N 拉开的差距	29:00	Lightman et al.
分布外泛化	多数投票 > ORM、PRM 最好	31:01	同上
Math-Shepherd 对比基线	红 = self-consistency，蓝 = ORM，绿 = Math-Shepherd	44:25	Wang et al.
PPO 结果表	PRM 做奖励 vs ORM 做奖励；RL 后再叠验证	46:29	同上
Weaver 扩展曲线	oracle、Weaver、朴素集成、多数投票、MAV 的相对位置	1:00:04	Weaver
8B ≈ 70B、70B ≈ o3-mini	跨模型档位的对比柱状图	1:02:09	同上
蒸馏后的算力效率	浅蓝 = 蒸馏版，深蓝 = 原版 Weaver	1:05:12	同上

提到的工作

名称	在本讲里的作用
Training Verifiers to Solve Math Word Problems (Cobbe et al., 2021)	第一个系统的 ORM + best-of-N；提出 GSM8K
Let's Verify Step by Step (Lightman et al., 2023)	ORM vs PRM；PRM800K；主动学习式标注
Math-Shepherd (Wang et al., 2023)	rollout 自动标注过程标签；PRM 用于验证和 PPO
Weaver (NeurIPS 2025)	弱监督组合验证器池并蒸馏
Snorkel (Ratner et al.)	Weaver 所用弱监督思路的来源
Multi-Agent Verification (MAV)	多角度 prompt 打分的基线
Self-consistency	多数投票基线
GSM8K / MATH / MATH500 / GPQA Diamond / MMLU-Pro	评测集
CodeMonkeys	自生成单元测试作验证器（后续展开）
Hugging Face 上的 reward model 排行榜（应为 RewardBench）	现成验证器的来源

术语对照

English	中文
verifier	验证器
generation–verification gap	生成–验证差距
outcome reward model (ORM)	结果奖励模型
process reward model (PRM)	过程奖励模型
outcome / process supervision	结果监督 / 过程监督
credit assignment	信用分配
false positive	假阳性（过程错、答案碰对）
best-of-N	N 选最优
majority voting, self-consistency	多数投票，自洽性
coverage, pass@k oracle	覆盖率，pass@k 上限
active learning	主动学习
hard / soft estimation	硬估计 / 软估计
rollout, completion	展开，补全
reward hacking	奖励投机

English	中文
rubric	评分细则
LLM-as-a-judge	LLM 评审
weak supervision	弱监督
ensemble	集成
conditional independence	条件独立
distillation	蒸馏
distribution shift, out-of-distribution	分布偏移, 分布外
greedy decoding	贪心解码

字幕勘误

"GSM 8-K" → GSM8K; "PSI / QI" → 解 s_i / 题 q_i; "MLU Pro" → MMLU-Pro; "LLema-34B" → Llemma-34B; "code monkeys" → CodeMonkeys; "weak to strong supervision" 在这里指 weak supervision (Snorkel 一脉), 不是 OpenAI 的 weak-to-strong generalization; 论文消融部分说的 "sentence level", 论文原文是整解级 (solution-level) 对 token 级。

带走的问题

1. 固定总推理算力, 应该怎么在"生成器多采样""验证器做大""验证器加多"之间分配? 生成器和验证器的大小有没有 Pareto 最优配比?
2. 把验证器当 RL 奖励时, 怎么防止生成器学会跳步骤、写讨好验证器的过程? 自动标注的 PRM 在这一点上比人工标注脆弱多少?
3. Weaver 的"验证器相互独立"假设在实践中多大程度成立? 如果池子里的 judge 大多来自同一基座, 相关性会不会让权重估计失真?
4. 数学和代码之外、没有标准答案的任务, rollout 式的过程标注还能定义吗? "走到正确答案的潜力"换成什么?

CS329A 第 4 讲 | 用工具与代码反馈来学习

(Learning from Feedback with Tools/Code)

Stanford CS329A: Self-Improving AI Agents (2025 秋) · 对应课表第 4 次课 (10 月 3 日; 所讲三篇论文与该次课的指定阅读完全一致) 视频: <https://www.youtube.com/watch?v=Lxh9RF5S-K0> (1:11:13, 自带英文 CC, 专名仍有识别错误) 讲者: Aakanksha Chowdhery (推断: 没有自我介绍; 她称前两讲由 Azalia 讲授, 提到自己在 Google 与机器人团队的合作, ReAct 实验所用的 PaLM 也是她主导的模型) 配套阅读: [ReAct: Synergizing Reasoning and Acting in Language Models](#) (Yao et al., 2022) · [RLEF: Grounding Code LLMs in Execution Feedback with Reinforcement Learning](#) (Gehring et al., 2024) · [Constitutional AI: Harmlessness from AI Feedback](#) (Bai et al., 2022)

一句话: 三篇论文回答同一个问题——让模型变好的反馈从哪来。ReAct 的反馈是工具返回的观察, 只写回上下文; RLEF 的反馈是代码执行结果, 先写回上下文供修复, 再变成奖励更新权重; Constitutional AI 的反馈是模型依据人写的原则对自己的批评, 用来造微调数据和偏好标签。回路里的信号够强, 模型就能超出训练数据给它的水平。

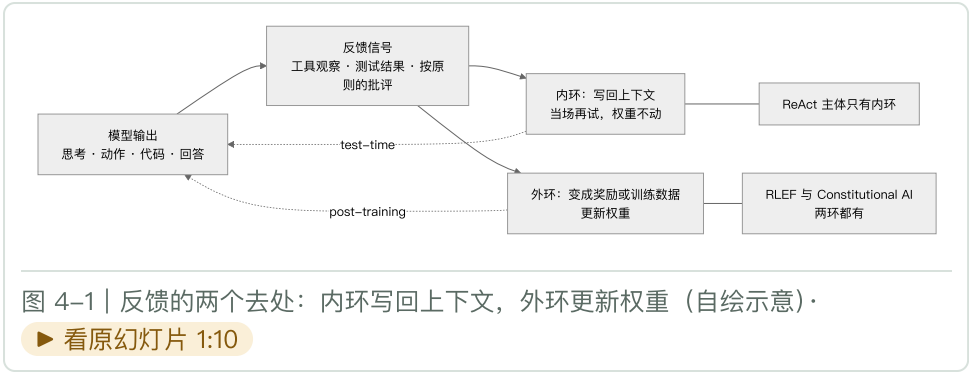
时间轴

时间	内容
0:06	引子: LLM 要落到真实任务, 得会用工具、跑代码并从结果里学; 三篇论文与"反馈从哪来"
1:41	ReAct 的动机: 人的"想—做—看"循环; 没有 grounding 的模型只能幻觉
3:47	回顾两条割裂的路线: CoT (只推理) 与 WebGPT (只行动)
4:49	ReAct 的做法: 纯 prompting; 三个任务 HotpotQA / FEVER / WebShop
6:52	左环推理、右环行动; 思考只发生在语言空间; 怎样保证动作合法
8:59	实现 (冻结的 PaLM + few-shot) 与 Apple Remote 例题
12:30	问答: 模型知不知道该不该搜、是否交错、搜索结果矛盾、为什么要显式 thought
17:19	知识任务的结果: 基线、两种 fallback 组合、失败模式分析
20:23	决策任务 WebShop: 对比 IL、IL + RL 与人类专家
22:05	ReAct 小结与局限; 课堂讨论 (噪声反馈、缺失的认知机制、overthinking)
27:25	论文二 RLEF: 动机与设定 (动作 = 代码, 观察 = 执行反馈, 二元奖励, PPO)

时间	内容
29:16	框架图：公开测试内环 + 隐藏测试奖励外环；回文子串例题
31:20	两级测试的用意；token 级策略 + turn 级价值；问答：两组测试从哪来
35:26	结果：solve rate 对采样预算；为什么有效——逐轮错误分析
37:50	问答：二元奖励够不够、ORM 还是 PRM、对比 SFT、两级测试的消融、超时为何变多
43:44	小结与课堂讨论：代码库放不进上下文怎么办（Claude Code、SWE-bench、CodeMonkeys）
46:25	论文三 Constitutional AI：RLHF 回顾与人工标注的瓶颈
47:56	宪法 = 人写的原则；两阶段总览；宪法示例
50:29	SL 阶段：critique → revision → 微调；修订次数的影响
51:29	RL 阶段：AI 偏好 → preference model → RL；问答：宪法修订与遗忘
53:33	结果：Elo 曲线；问答：CoT 版本更不 helpful、SL 阶段没有显式反馈、AI 反馈准不准
58:12	Pareto 图；推广与后续工作（RLAIF、Self-Refine、SCoRe）
1:00:20	全课回顾
1:03:30	结尾问答：认知科学类比、手工框架会不会过时、context 即 state、数据过滤

核心内容

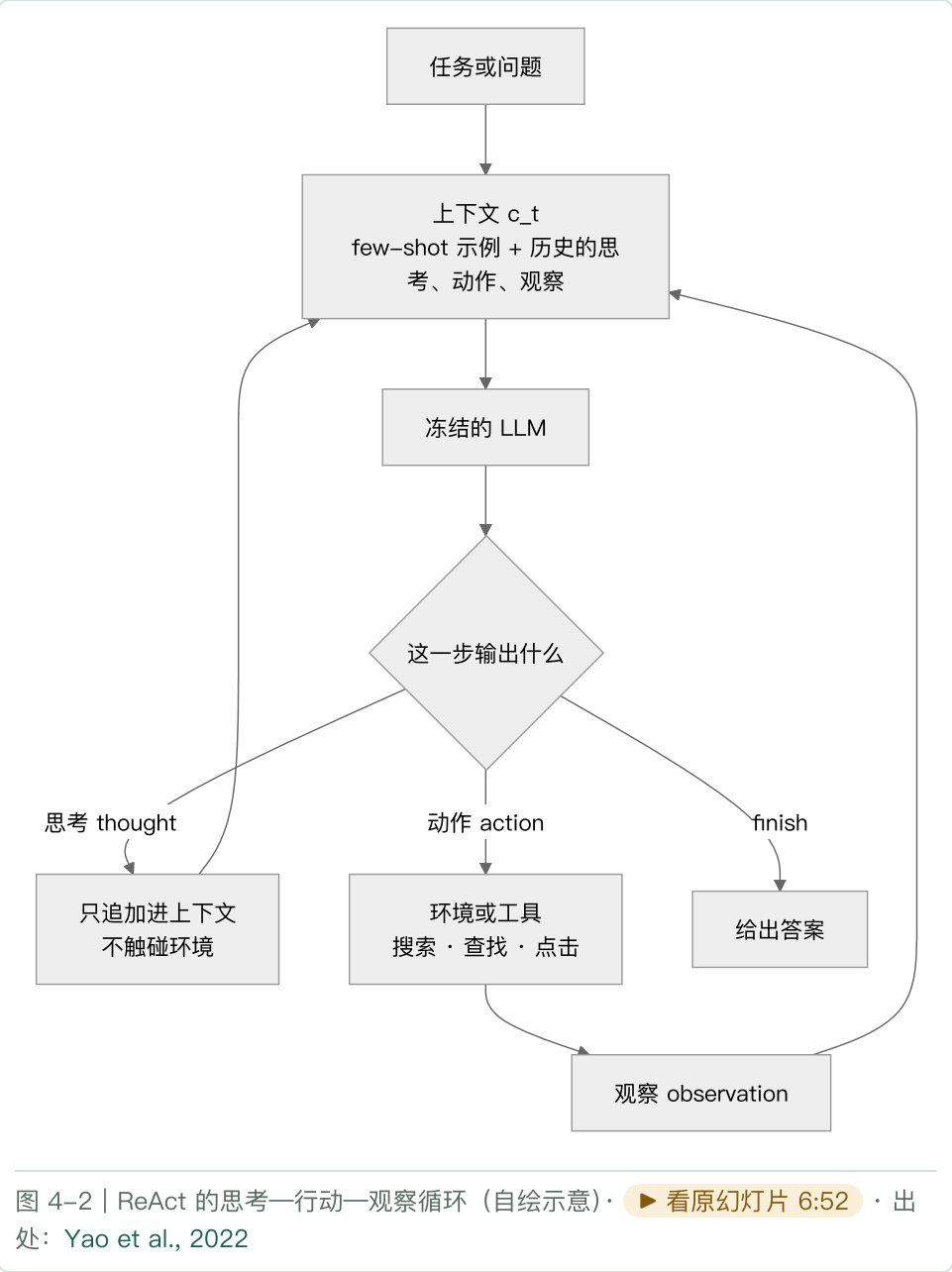
1. 主线：反馈从哪来，喂到哪去



- LLM 要在真实任务里有用，必须能和环境、工具、代码打交道，并从交互结果中学习。前两讲（重复采样、验证器）解决"多试几次，再挑出对的"；这一讲让外部世界直接告诉模型哪里不对。
- 讲者给三篇论文的分类标准只有一个——**反馈来源**：环境交互（ReAct）、可执行的客观检验（RLEF）、依据原则的批评（Constitutional AI）。

- 图 4-1 是我的整理：内环把反馈写回上下文、当场再试，对应第 1 讲的 test-time 轴；外环把反馈变成奖励或训练数据，对应 post-training 轴，也就是第 1 讲的自我改进回路。

2. 论文一：ReAct (Princeton + Google, 2022) ——在语言空间里交错推理与行动



- 动机：**人做事是"想 → 做 → 看结果 → 再想"的循环，当时的 LLM 却把两头割裂开。CoT 只推理：步骤看着合理，但全部出自内部状态，没有外部校验，容易幻觉（问今天的气温，不搜索就不可能知道）。WebGPT 一类只行动：在人操作浏览器的轨迹和偏好上微调，会用工具，却没有显式推理。

- **做法：**纯 prompting，不训练。冻结的 PaLM 加 few-shot 示例，示例里写出 thought / action / observation 三种条目，模型每一步自己决定输出思考还是动作。思考只存在于语言空间，不改变环境，只被追加进上下文；动作交给环境执行，观察同样追加进上下文。课上的形式化里，上下文 c_t 就是 RL 意义上的 state。这样不必训练 policy，只靠 prompt 就搭出了第 1 讲的 agent loop，轨迹还是人读得懂的。
- **让动作合法：**把"选动作"改写成在合法动作清单上的分类——把推理、当前状态和清单一起给模型让它挑，而不是自由生成。讲者说她在 Google 与机器人团队合作的工作也这么做，机器人没有动作清单根本无法操作。

小注：她没有点名；从描述看应指 PaLM-SayCan / PaLM-E 一类工作（推断）。

- **例题（HotpotQA）：**除了 Apple Remote，还有什么设备能控制它最初要操控的那个程序？

方式	推 理	行 动	在这道题上的表现
Stand ard	—	—	直接答错
CoT	有	—	步骤写得出来，但事实没有依据，仍然答错
Act- only	—	有	搜了 Apple Remote、又搜 Front Row，一路跟下去，最后给不出像样的答案
ReAct	有	有	先想清要查什么 → 搜到 Front Row → 直接搜它搜不到 → 想到改搜 Front Row software → 得出答案

- **和今天的模型的关系：**Qwen 等模型打开 thinking 模式后，"想一步、调一次工具"会自动出现——后训练（课上提到 thinking fusion 一类阶段）已经在这类轨迹上蒸馏过。ReAct 不再是要手写的技巧，而是理解这种行为的积木。
- **问答**
 - 模型怎么知道该不该搜：它"知不知道自己知道什么"目前有两派看法；让它自报置信度通常过度自信、校准差，仍是研究问题。做应用不必赌这一点，让它用对工具、拿到有出处信息更实际。
 - 思考是一次写完再逐个执行吗：不是，thought 与 action 严格交错，每个观察都会改变下一步思考；要并行采样得另外设计。

- 搜索结果互相矛盾：交给用户之前要有校验和护栏，例如第 2 讲的多数投票；带检索的幻觉仍比只靠内部状态可控得多。
- 为什么要显式写 thought：语言模型是在语言上训练的，推理 token 是引出正确动作的合适抽象；换成别的中间表示，未必需要。（另有学生问 thought 与 observation 在 KV-cache 里的表示差异，被建议当课程项目。）

3. ReAct 的结果、局限与课堂讨论

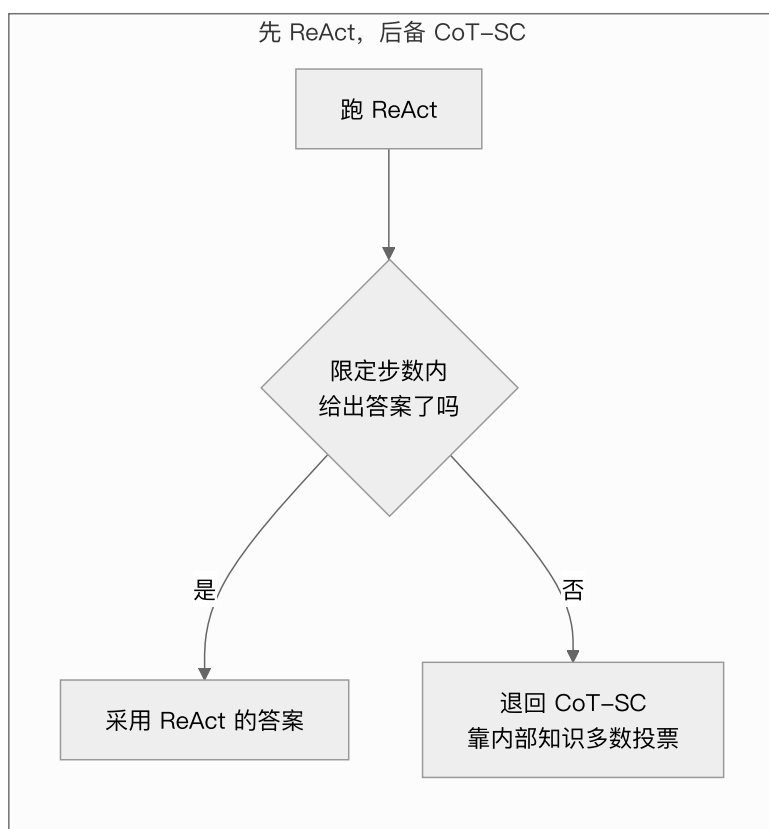
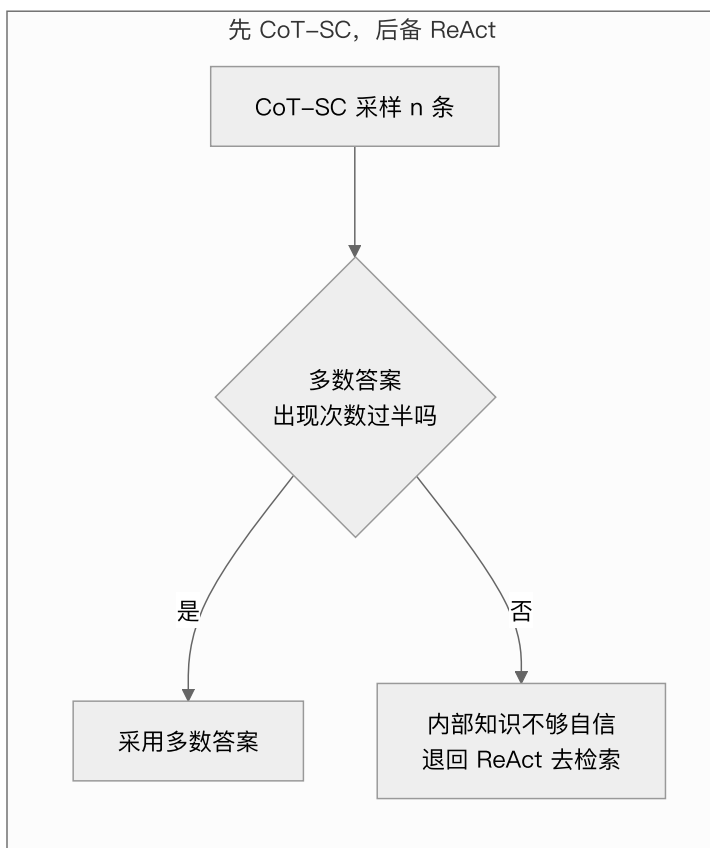


图 4-3 | ReAct 与 CoT-SC 互为后备的两种组合（自绘示意）·

- **知识密集任务** (HotpotQA 多跳问答、FEVER 事实核查)：动作只有三种——搜索 Wikipedia 词条、在页面里查找字符串、结束并作答。基线是 standard、CoT、CoT-SC (self-consistency, 即多数投票) 和不带思考的 Act-only。
 - ReAct 稳定好于 Act-only；对 CoT 互有胜负：FEVER 上赢，HotpotQA 上没赢。
 - 最好的是图 4-3 的两种组合：内部知识和外部检索要配合着用，推理是把两者接起来的中间层；多数票占比在这里充当"内部知识不够自信"的代理。
 - 失败模式分析：CoT 的主要问题是幻觉；ReAct 拿得到外部知识，错误更少、更可信。能微调时 ReAct 的优势更明显，再加外环训练还会更好（下周展开）。

小注：论文 Table 1 (PaLM-540B) 的 HotpotQA EM / FEVER Acc——CoT 29.4 / 56.3, CoT-SC 33.4 / 60.4, Act 25.7 / 58.9, ReAct 27.4 / 60.9, CoT-SC→ReAct 34.2 / 64.6, ReAct→CoT-SC 35.1 / 62.0；ReAct 的步数上限是 HotpotQA 7 步、FEVER 5 步。

- **决策任务** (WebShop)：模拟购物网站，按用户指令（如找带抽屉的床头柜）搜索、点选、下单。只靠 prompting 的 ReAct 在 score 和 success rate 上都高过 imitation learning (即 SFT) 和 IL + RL；但 score 66.6 对人类专家的 82.1，空间还很大。多步任务里一步出错就向后级联，所以 success rate 更低。

小注：WebShop 里 score 是所购商品与需求的匹配度 (平均 reward × 100)，success rate 是完全匹配的比例；课上把 score 说成中间步骤的得分，不太准确。论文的 success rate：ReAct 40.0、Act 30.1、IL 29.1、IL + RL 28.7、人类专家 59.6。

- **局限**：动作空间一大，需要的示范就塞不进上下文；多轮推理抬高推理成本。
- **课堂讨论**
 - 环境反馈有噪声甚至误导：加一层 reflection，先审视环境给的东西；允许回溯（噪声会把 agent 带进重复循环）；多次检索或重复执行，换一个更好的置信度估计。
 - 除了"想 + 做"还缺什么认知机制：任务分解；并行尝试几条思路再比较；记忆与过往经验；按任务学会"想多少再动手"（讲者顺带提

到现在的模型有 overthinking 的毛病，简单题也写很长的思考)；
按各模型所长分派子任务的 compound system。

4. 论文二：RLEF (Meta FAIR, 2024) ——把执行反馈接进 RL

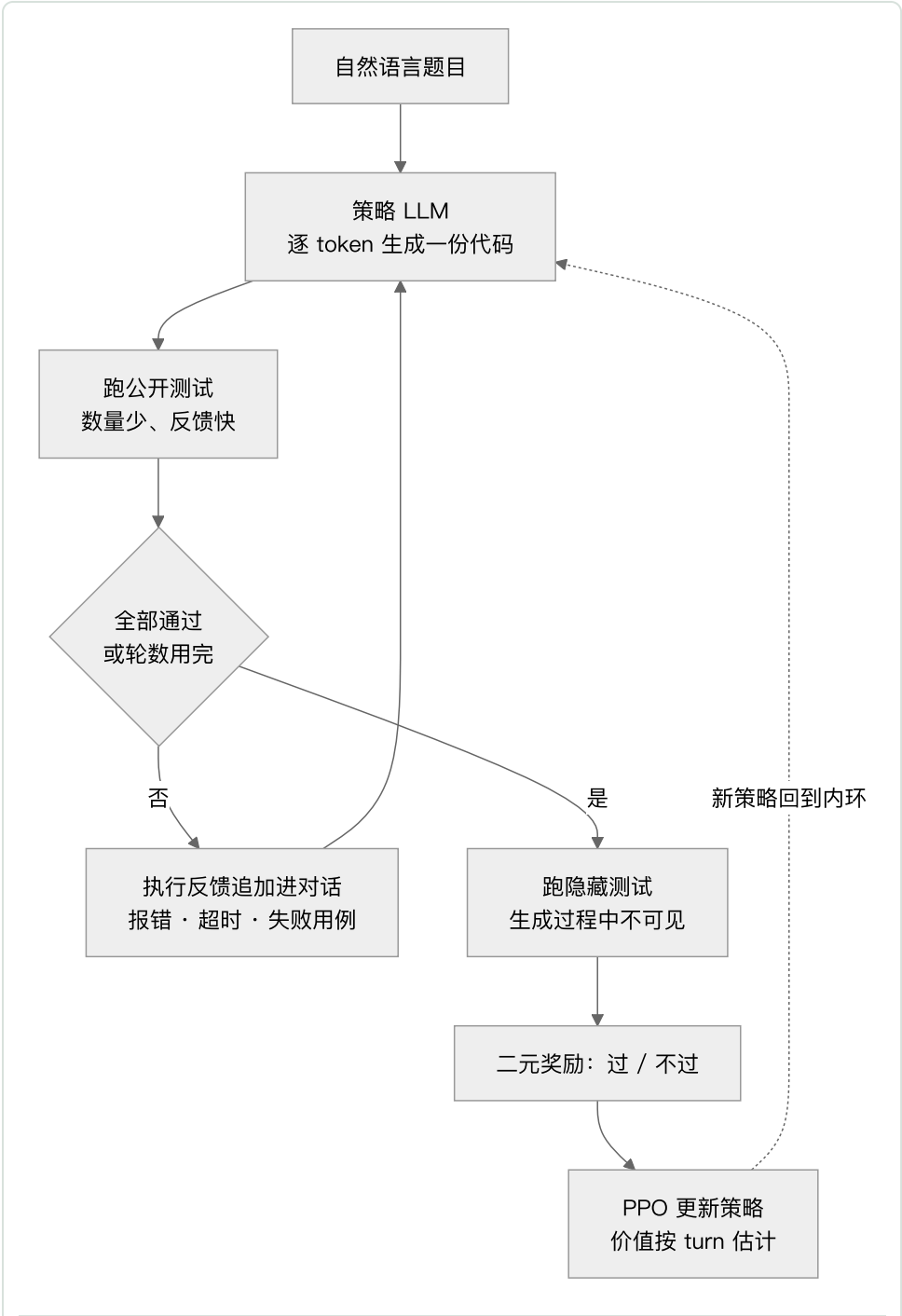


图 4-4 | RLEF：推理时的修复内环与训练时的 PPO 外环（自绘示意）。

► 看原幻灯片 29:16 · 出处：Gehring et al., 2024

- **动机：**课上举手用 Claude Code 的占了多数。要让 coding agent 足够强，一要读懂用户意图，二要能根据代码跑出来的结果迭代。

- **设定：**端到端 RL 微调。动作 = 一整份生成的代码；观察 = 执行反馈（跑测试的输出）；奖励 = 测试过没过的二元信号；优化用 PPO。内环（推理时）反复"生成 → 跑公开测试 → 读反馈"，直到通过或轮数用完；外环（训练时）用隐藏测试的结果给奖励、更新策略。讲者把内环称为对当前策略的 exploitation。
- **创新一：两级测试。**公开测试只是一小部分，跑得快，专给内环提供即时反馈；隐藏测试在生成过程中完全不可见，只用来给奖励。模型没法靠记住测试输出拿分，奖励也不会被"指导修复的那批测试"泄漏。问答里的解释：CodeContests 每道题都配有测试，留一部分给内环，另一部分留给奖励。
- **创新二：token 级策略 + turn 级价值。**策略逐 token 生成，保留细粒度控制；价值函数按回合估计——从 prompt 的最后一个 token 读出一个值，整段回答的所有 token 共用一个 advantage。讲者的类比：更接近 GSPO 那种"对整条序列给奖励"，而不是逐 token 给。
- **例题：**检测回文子串。第一轮朴素解在公开测试上超时；读到反馈后第二轮做了优化，通过公开测试，才提交给隐藏测试。
- **结果：**纵轴 solve rate，横轴采样预算（对数），验证集、测试集各一张，基座是 Llama 3.1。在 CodeContests 上做完 RLEF，同等预算下 solve rate 明显更高，能泛化到其他代码 benchmark，达到 SOTA 所需的采样预算也更少。（采样预算是第 2 讲的并行那一支，多轮修复是串行那一支。）

小注：n@k 是 AlphaCode 沿用下来的指标——每题采 k 个样本，从中挑 n 个提交，任何一个通过全部隐藏测试就算解出；课上对 10@k 的口头解释比较含糊，以此为准。论文的 1@3 (CodeContests test)：Llama 3.1 8B Instruct 10.5 → 16.0，70B Instruct 27.5 → 40.1；默认每题最多 3 轮；奖励是全过 +1、不过 -1、回答里没有合法代码 -0.2，另加 KL 惩罚。CodeContests 数据集本身就把每题的测试分成 public 与 private 等几组，不需要自己切。

- **为什么有效：**没经过这种训练的模型，把"错误解 + 执行反馈"摆在面前也基本用不上；RLEF 每一轮都让它看到反馈，它才学会利用。消融图按轮次统计错误数、相邻两轮的代码改动量和错误类型（小、大模型各一组）：训练之后，后面轮次的 wrong output 变少，改动是冲着出错位置去的真修复。收益来自两处——多轮带来的样本多样性，加上更有针对性的修改。作业会在数学题上试同样的思路：先定位解答里的错误，再修。

小注：论文还做了“喂随机的、与本题无关的反馈”的对照，修复能力明显下降，说明模型确实在读反馈，而不只是多采了几次样。

• 问答

- 二元奖励够吗：反馈里没有修改建议，只告诉模型哪里报了错（这些题不到 100 行）。学生猜够用是因为题目简单，难题还需要 error trace 等信息——讲者认为很可能，是好的项目题。她要大家带走的是：闭环本身成立，足够简单的问题上二元奖励就能转起来；要不要每一步都给反馈，就是第 3 讲 ORM 对 PRM 的争论，得按领域取舍。
 - 只有终局奖励，怎么鼓励一次做对：进 PPO 之前模型已在内环里有过几次改正机会；轮数上限等都可调。
 - 和 SFT 比：在推理轨迹上做 SFT 也能拿到一部分收益，域内尤其明显；RL 更能解新题，泛化略好——仍有争议。
 - 两级测试有没有消融（测试全部公开会怎样）：讲者没看到；她的理解是避免“用来反馈的”和“用来给奖励的”之间泄漏。这个问题课上没谈拢。
 - 为什么训练后 wrong output 变少、超时反而变多：讲者猜是逻辑错误修掉之后，剩下没解决的更多表现为超时；很依赖领域。
- **课堂讨论：代码库放不进上下文怎么办。**学生的方案：像 ReAct 那样带着搜索工具反复找，直到信息够用；给每个文件写摘要；把代码库建成图再按相似度检索。讲者：Claude Code 实际就是这么跑的——先找相关代码，再决定打什么 patch、跑测试；SWE-bench 考的就是这件事，Azalia 实验室的 CodeMonkeys 也针对同一问题。

5. 论文三：Constitutional AI (Anthropic, 2022) ——反馈来自模型自己

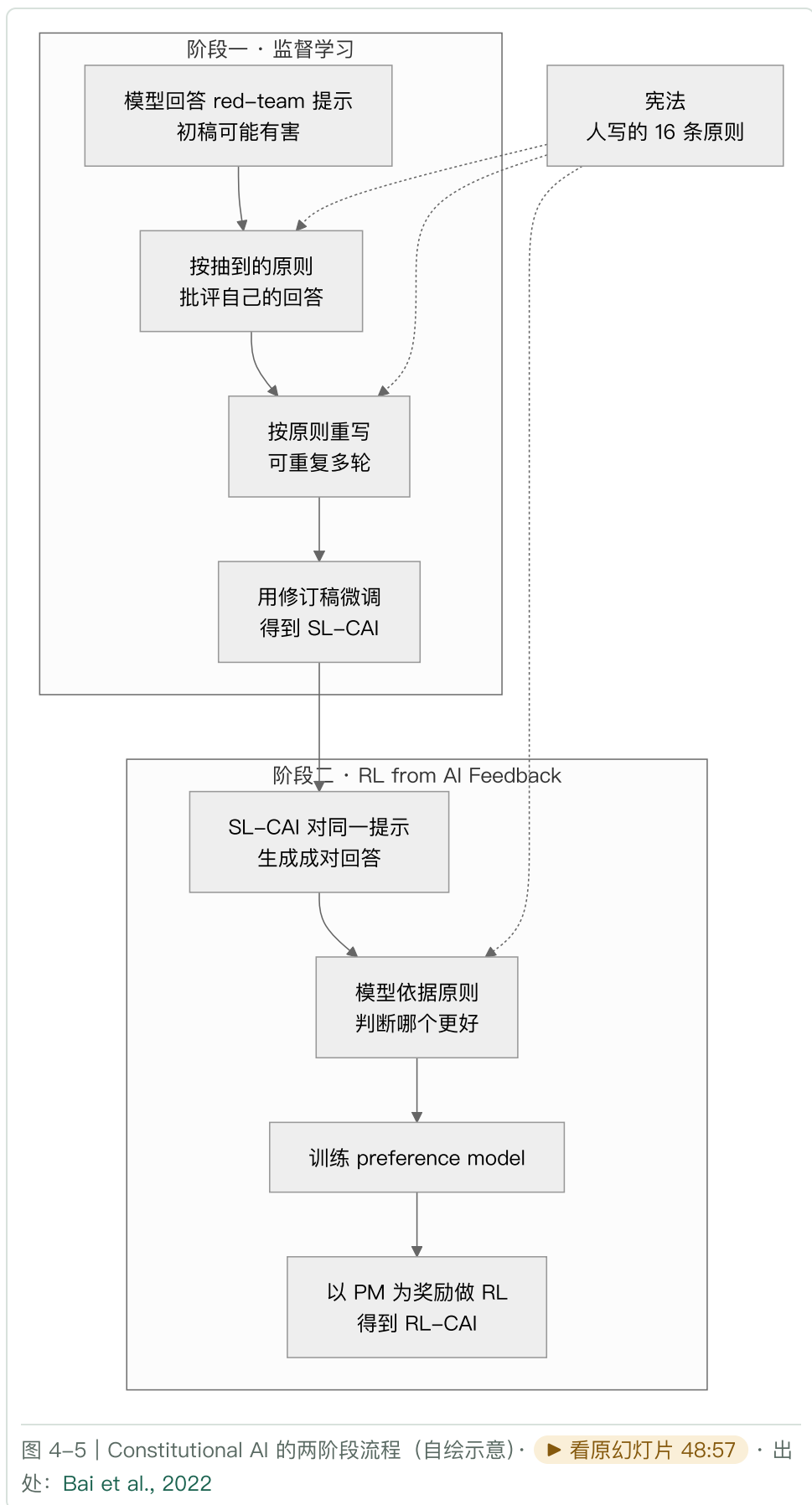


图 4-5 | Constitutional AI 的两阶段流程（自绘示意）· [▶ 看原幻灯片 48:57](#) · 出处: Bai et al., 2022

- **背景：**要把 base model 变成既 helpful 又 harmless 的助手，标准做法是 RLHF——人给两个回答排序（是否正确、有用、够具体）→ 训

练 reward model → 拿它当目标 hill-climb。课上的图：做过 Python 微调的模型加上 RLHF 后更好，模型越大差距越明显。瓶颈在标注：几万条人工偏好标签，又慢又累，扩不上去。

小注：这张图应出自 Anthropic 更早的 HH-RLHF 论文 (Bai et al., 2022a) 里对 Python 微调模型做 RLHF 的实验 (推断)。

- **核心想法**：人只写一份"宪法"——一组描述期望行为的自然语言原则 (论文用了 16 条)，其余反馈由模型自己产生。可行的前提是指令遵循已经够强：问它"这段回答有没有某类问题"能如实判断，让它"按要求重写"也能照做。
- **阶段一 (SL)**：用 red-teaming 提示诱出有问题的回答 → 按某条原则让模型批评自己 (课上的三个例子：有没有有害或不道德的内容；有没有性别偏见；对儿童是否不合适) → 按配套的 revision request 重写 → 用修订稿微调。修订次数越多，harmlessness 越高、helpfulness 越低，两者合起来的分数单调上升。
- **阶段二 (RL)**：模型依据宪法给成对回答打偏好标签 → 训练 preference model → 用它做 RL。流程和 RLHF 一样，只是 PM 的数据来自宪法而不是人。

小注：按论文，SL 阶段微调的目标是"原提示 → 修订后的回答"，critique 文本不进训练目标，同时混入 helpful RLHF 模型在普通提示上的回答来保住有用性；RL 阶段只有 harmlessness 的偏好标签由 AI 生成，helpfulness 的偏好标签仍是人标的，两者混合训练 PM。

• 结果

- Elo 曲线 (横轴是 RL 训练序列数，Elo 由人对回答的两两偏好算出)：helpful-only RLHF、HH RLHF、RL-CAI、RL-CAI + CoT 四条线。换成 AI 反馈后 helpfulness 大致持平 (略低)，harmlessness 明显更高。
 - Pareto 图 (横轴 helpfulness Elo，纵轴 harmlessness Elo)：pretrained base 两头都低；helpful-only RLHF 主要往右推；Constitutional SL 单独用还不如 RLHF；Constitutional RL + CoT 走到最外沿——这条更好的 Pareto 前沿就是论文的核心主张。
 - 讲者的评价：harmlessness 是 Claude 系列一直很强的地方，这套技术现在各家都在用。
- **推广与后续**：凡是要模型遵守一组 (可能互相冲突的) 规则的场景，都能套这个闭环。后续工作有 RLAIIF 与 RLHF 的系统比较、Self-

Refine（迭代式自反馈）、用 RL 教模型自我纠错（应为 SCoRe）。讲者提醒：模型过度自信，自己批评自己往往不灵，让一组别的模型来批评再取共识有时更好——和第 3 讲 Weaver 组合弱验证器是同一个直觉。

• 问答

- 宪法要修订怎么办：post-training 只占总算力的一小部分（她给的数量级约 5%），本来就更新得频繁，可以跟着重做。更深的问题是 continual learning——怎么让模型可靠地忘掉旧规则、改守新规则，仍是开放问题；借可解释性方法抵消特定知识的尝试，还不能证明模型真的"忘了"。
- CoT 版本的 helpfulness Elo 更低，是 CoT 拖了后腿吗：有可能，但讲者认为主因是 harmless 和 helpfulness 此消彼长，要找平衡点。

小注：论文自己也写了 RL-CAI + CoT 比不带 CoT 的版本略不 helpful、略更 harmless。

- SL 阶段是不是只在自己的 rollout 上提高似然、没有显式反馈：是，这一阶段没有奖励信号，和在推理轨迹上微调同理。微调规模远小于预训练，原有能力不会丢，只是输出分布偏向这类回答。
- 怎么知道 AI 的反馈准不准：给 PM 留一个带人工标签的验证集，检查它和人的一致性——仍需要人，但远不到上万条。
- 为什么不干脆从数据上限制（结尾处的提问）：预训练数据过滤确实在做，但模型是在互联网上训练的，滤不干净。

6. 回顾与最后的问答

三篇论文并排看（表是我的整理，"反馈可靠性"一行是我加的判断）：

	ReAct	RLEF	Constitutional AI
反馈来源	工具 / 环境返回的观察	代码执行与单元测试	模型依据人写的原则作出的批评与偏好
反馈可靠性	取决于工具，搜索结果可能有噪声	客观、可执行	受模型自身校准的限制
反馈怎么用	写回上下文，影响下一步思考	内环写回上下文，外环作 PPO 奖励	修订稿作 SFT 数据；AI 偏好训练 PM 再做 RL
改不改权重	不改	改	改
人的投入	few-shot 示例	题目与测试用例	一份宪法 + 少量一致性校验

- 这些设计都在类比人的认知，有没有更系统的方法：现阶段确实在借用"人怎么解题"（分解、分析、并行）。真正要回答的是推理空间里的搜索能不能自动化：搜索空间定义得清楚时（比如游戏）可以；现实任务的搜索空间定义不清，只能在语言空间里做，靠真实环境里的观察来积累。
- RL 后训练会不会让 ReAct 这类手工框架过时：是也不是。抽象层面都是"生成 token，再用奖励监督"，可以被训练吸收；但具体到会计、金融、法律，完成任务的 workflow 高度领域相关，很难一套通吃。ReAct 本质上规定了一种"想—做—想—做"的工作流，而第 1 讲里 agent 的定义正是对工作流的抽象；讲者也把本讲的内容看作 agent 的雏形，只是还没有 session 和 memory。
- 形式化里的 context 是不是 state：是，prompting 语境下的 state 就是上下文里的全部内容。动作空间包含"输出一段思考"和"输出一个环境动作"两类，模型每步二选一，思考可以单独成一步。

关键图表速查（点时间戳跳到原幻灯片）

图	看什么	跳转	出处
Apple Remote 例 题的四条轨迹	同一道 HotpotQA 题：standard 与 CoT 答错，Act-only 搜到了却收不了尾，ReAct 在搜索失败后改了检索词	9:29	ReAct
HotpotQA / FEVER 结果表	ReAct 对 Act 稳赢、对 CoT 互有胜负；最高的两行是 ReAct 与 CoT-SC 的组合	18:45	同上
成功 / 失败模式 分析	CoT 的失败大头是幻觉；ReAct 的推理有检索结果支撑	19:35	同上
WebShop 结果表	ReAct 对 IL、IL + RL 的 score 与 success rate；66.6 对人类专家 82.1	20:53	同上
RLEF 框架图	上方紫色块是带公开测试的多轮对话，黄色块是提交隐藏测试，右侧是 PPO 更新	29:16	RLEF
回文子串例题的对 话	第一轮超时 → 执行反馈 → 第二轮优化后通过公开测试	30:48	同上
solve rate 对采样 预算	横轴是对数；验证集、测试集两张；RLEF 之后的线整体在基座之上	35:26	同上
逐轮错误与代码改 动分析	第 1 / 2 / 3 轮的错误数与类型、相邻轮次的代码改动量；小、大模型各一组；留意超时一栏	36:50	同上
宪法示例	三组 critique / revision request：有害内容、性别偏见、儿童适宜性	49:28	Constitutional AI
修订次数对 PM 分数	harmlessness 升、helpfulness 降、两者合计单调升	50:59	同上

图	看什么	跳转	出处
Elo 对训练序列数	左 helpfulness、右 harmlessness；四条线是 helpful RLHF、HH RLHF、RL-CAI、RL-CAI + CoT	53:33	同上
Harmlessness—helpfulness Pareto 图	从 pretrained base 出发，各方法把点往哪个方向推；RL-CAI + CoT 在最外沿	58:12	同上

提到的工作

名称	在本讲里的作用
ReAct (Yao et al., 2022)	主讲论文一：用 prompting 把推理与工具调用交错起来
Chain-of-Thought、Self-Consistency (CoT-SC)	只推理的基线；SC 就是多数投票
WebGPT (应为 Nakano et al., 2021)	只学操作浏览器、没有显式推理的代表
PaLM	ReAct 实验用的冻结基座
HotpotQA / FEVER / WebShop	多跳问答 / 事实核查 / 模拟网购决策
与 Google 机器人团队合作的工作 (应指 PaLM-SayCan / PaLM-E 一类，推断)	在合法动作清单上做选择
Qwen 的 thinking 模式	交错推理与工具调用已被内化的例子
RLEF (Gehring et al., 2024)	主讲论文二：执行反馈 + PPO
CodeContests	RLEF 的训练与评测集：竞赛编程题，每题配有测试用例
Llama 3.1	RLEF 的基座
PPO、GSPO	RL 算法；后者用来类比"整条序列共用一个 advantage"
Claude Code、SWE-bench、CodeMonkeys	仓库级 coding agent、对应的 benchmark、Azalia 实验室的相关工作
Constitutional AI (Bai et al., 2022)	主讲论文三：用 AI 反馈替代人工的 harmlessness 标注
RLHF	Constitutional AI 要替代的人工偏好流程
RLAIF vs. RLHF (应为 Lee et al., 2023)	系统比较 AI 反馈与人类反馈
Self-Refine (应为 Madaan et al., 2023)	迭代式自反馈
Training Language Models to Self-Correct via RL (应为 SCoRe, Kumar et al., 2024)	用 RL 教模型自我纠错

术语对照

English	中文
tool calling	工具调用
grounding	落地（让输出有外部依据）
reasoning trace, thought	推理轨迹，思考
action / observation	动作 / 观察
action space	动作空间
interleaved	交错
hallucination	幻觉
calibration, overconfident	校准，过度自信
multi-hop question answering	多跳问答
fact verification	事实核查
imitation learning (IL)	模仿学习
backtracking	回溯
task decomposition	任务分解
overthinking	过度思考
compound system	复合系统
execution feedback	执行反馈
public / private tests	公开 / 隐藏测试
solve rate, n@k	解出率
sample budget	采样预算
turn-level value function	回合级价值函数
advantage	优势
exploitation	利用
leakage	泄漏
constitution, principle	宪法，原则
critique / revision	批评 / 修订
red-teaming prompt	红队提示
preference model (PM)	偏好模型
RLHF / RLAIF	基于人类反馈 / AI 反馈的强化学习
helpfulness / harmlessness	有用性 / 无害性
Pareto frontier	帕累托前沿
continual learning, unlearning	持续学习，遗忘
self-correction	自我纠错

字幕勘误

"Azalea" → Azalia; "Cloud Code" → Claude Code (27:45、45:49 两处); "Code Monks paper" → CodeMonkeys; "solve rate 10 at k" → 10@k (n@k 指标); "[isn't] context is more difficult" → CodeContests; "two-tiered task strategy" → two-tier test strategy; "where do you get the public license" → public tests; "help climate" → hill-climb; "compositional principles" → constitutional principles; "self-critiquing relations" → self-critique and revisions; "set right a set of principles" → write a set of principles; "immediate steps" → intermediate steps; 学生提问里的 "source" → thoughts; "much less harmless" 是讲者口误, 意思是 much more harmless; "thinking fusion" 应指 Qwen3 后训练里的 Thinking Mode Fusion 阶段 (推断); "Python fine-tuning with RLHF" 不是识别错误, 见第 5 节小注。

带走的问题

1. 模型什么时候该调工具、什么时候该信自己的内部知识? ReAct 的 fallback 用多数票占比当置信度的代理; 在模型普遍过度自信的前提下, 有没有更可靠的"知道自己不知道"的信号?
2. 二元的执行反馈在不到 100 行的竞赛题上够用; 到了仓库级任务, 需要多细的反馈 (error trace、过程奖励)? 公开 / 隐藏两级测试到底贡献了多少——课上没人见过这个消融。
3. RL 后训练把"推理 + 工具调用"内化之后, ReAct 这类手工编排的 workflow 还剩什么价值? 在搜索空间定义不清的领域 (会计、法律), 工作流由谁来定义, 能不能自动搜出来?
4. 用 AI 反馈替掉人类反馈之后, 谁来验证 AI 的反馈? 多模型共识是否比自我批评更可靠? 宪法要修订时, 怎样低成本地让模型真正"忘掉"旧规则?

CS329A 第 5 讲 | 规划与多步推理 (Planning and Multi-Step Reasoning)

Stanford CS329A: Self-Improving AI Agents (2025 秋) · 对应课表第 5 次课 (10 月 6 日, Multi-step Reasoning/Planning) ——课上讲的三篇论文都在这次课的阅读清单里 视频: https://www.youtube.com/watch?v=MI_fp9XkB8Y (1:14:56, 自带英文 CC, 专名仍有识别错误) 讲者: Azalia Mirhoseini (推断: 讲者没有自我介绍, 但讲 SPRINT 和 SWiRL 时都说"我们", 而这两篇论文唯一的共同作者是她) 配套阅读: LATS (Zhou et al., 2023) · SPRINT (NeurIPS 2025) · SWiRL (COLM 2025) ——课上讲的是这三篇; 清单里的 ADaPT (Prasad et al., 2024) 和 AB-MCTS (Wider or Deeper?) 课上没有讲

小注: 讲者说 SWiRL "下周"在 COLM 报告。COLM 2025 是 10 月 7–10 日 (蒙特利尔), 而课表上这讲是 10 月 6 日, 离开幕只差一天, 说"下周"对不太上——可能是口误, 也可能实际授课日期与课表略有出入。

一句话: 多步任务要模型交替地"想、做、搜", 三篇论文各管一头: LATS 在推理时把 MCTS 套到 agent 轨迹上, 用环境反馈和反思把算力换成成功率; SPRINT 用改写过的轨迹教推理模型把长思维链拆成"规划—并行执行"的回合, 把串行 token 变成并行 token; SWiRL 用自己合成的多步轨迹加 judge 的逐步奖励做 step-wise RL, 训练时一次工具都不调, 学到的多步能力却能跨任务、跨工具迁移。

时间轴

时间	内容
0:05	开场: 本讲三篇论文; 多步任务 = reasoning + acting + search (规划旅行的例子)
2:11	LATS 要解决的问题: 从"出一个计划就执行"到多样化探索, 把 MCTS 引入 LLM agent
3:48	夏威夷旅行的小例子; 与 Math-Shepherd、ReAct 的区别
7:58	六个阶段; 用迷宫例子走一遍 selection → simulation
12:38	backpropagation; value function、UCT、回传三个式子
16:49	reflection; HotpotQA 与 WebShop 结果
19:10	LATS 小结与局限: 成本、不可逆动作
20:43	课堂讨论: UCT 之外的 bandit 算法; 重复动作怎么计数
23:21	SPRINT: 推理模型越想越长, 但很多步骤互不依赖

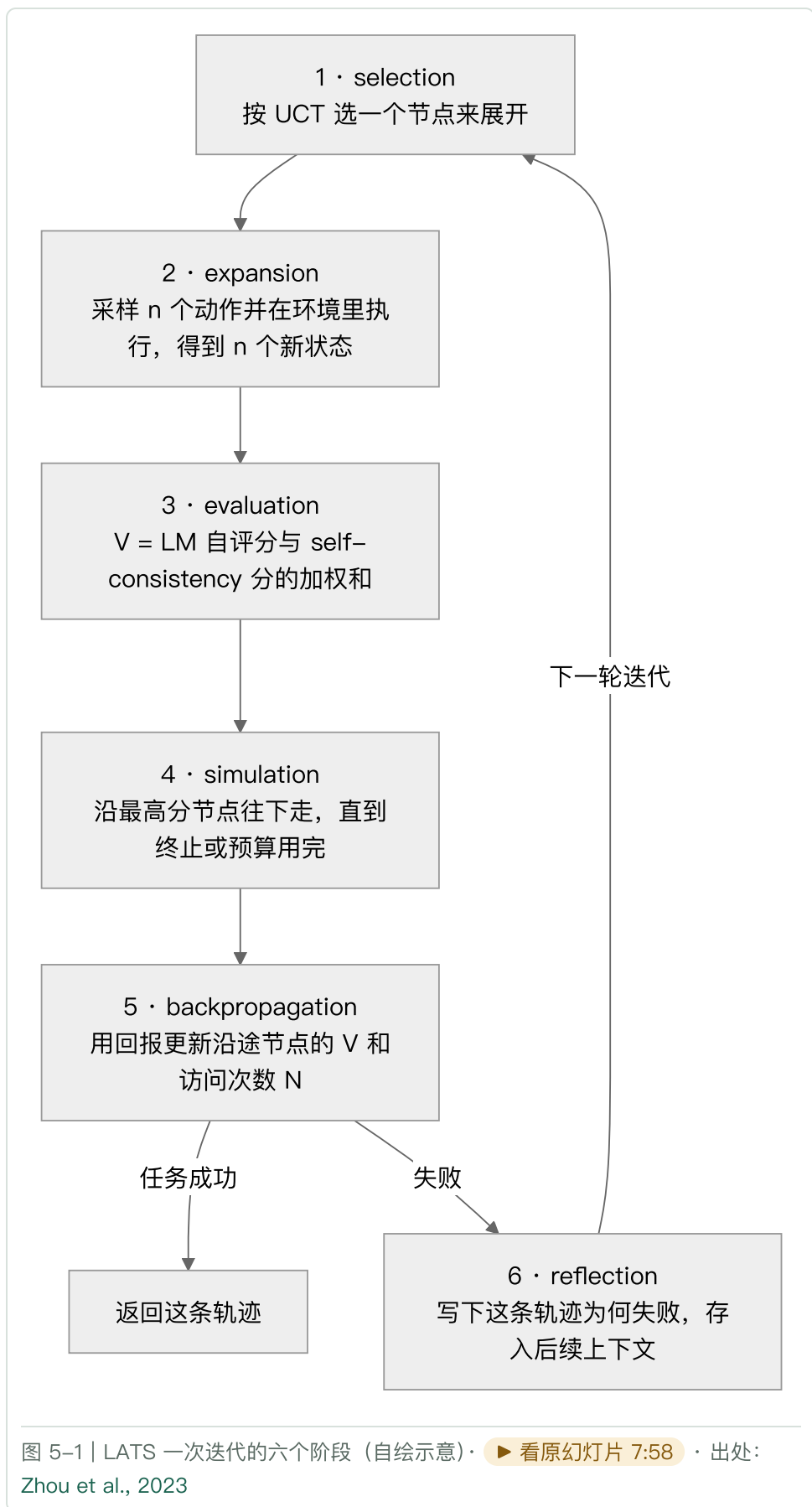
时间	内容
25:56	框架：planner + 并行 executors, 多轮交替
27:29	数据管线：R1 轨迹 → GPT-4o 标注 → DAG → 打包 → SFT
31:37	为什么重要：串行与并行的时间线；推理时怎么跑
33:40	问答：重新规划；有没有改变 next-token 预测
36:14	训练配方与结果：准确率也涨、域外泛化
38:55	问答：并行分支互相矛盾怎么办；两个观察（难题轮数多、前期并行多）
41:35	问答：负载均衡与 straggler；树有多宽取决于任务
45:13	省 token 的幅度随推理长度变化；Sonnet 4.5 "工具用 100 次"的旁证
46:47	问答：训练与推理时数据长什么样；SPRINT 的后续方向
50:04	SWiRL：动机、三个难点、设计目标
54:35	阶段一：迭代式提示合成多步轨迹、judge 逐步打分、四种过滤
58:30	阶段二：step-wise RL；训练时为什么不用调工具
1:01:09	问答：judge 要不要训练、看不到工具输出怎么打分；目标函数
1:04:24	推理时的迭代式提示；实验设置
1:06:56	结果：过程过滤最好；跨任务、跨工具泛化；数据规模曲线
1:11:06	每步过程奖励提升；RL vs SFT；总结

核心内容

1. 引子：多步任务要同时"想、做、搜"

- 这讲关心的任务不只要推理，还要按推理去行动，再根据反馈修订路线。讲者把"规划一次旅行"拆成三件事：**reason**（预算多少、去哪、怎么排）；**act**（把想法落成动作去拿信息：浏览网页、生成搜索 query、读 Reddit 或游记）；**search**（拿到反馈后改计划：换目的地、补查当地玩法）。三者要交替很多轮。
- 分工：LATS 是纯推理时的方法，算力花在轨迹空间的搜索上（第 2 讲的重复采样是整题独立采样，这里是边和环境交互边长出一棵树）。SPRINT 和 SWiRL 是训练方法，讲者特意点出两者共用的套路：**想让模型学会某种行为，先用 LLM 把数据改写或合成为体现这种行为的样子。**

2. 论文一：LATS——把 MCTS 搬到 agent 的轨迹上（ICML 2024）



- **要解决什么：**让 LLM "写一个计划、照着执行"只走一条路；讲者认为模型至今仍不擅长给出多样的方案并做多步优化。LATS 把规划领域成

熟的 Monte Carlo tree search 引入 LLM agent，并把环境反馈带回后续搜索。

- **和两篇前人工作的区别**：Math-Shepherd（第 3 讲）用验证器给推理步骤打分来引导搜索；LATS 打分的依据是动作在环境里的实际结果，外加对整条轨迹的反思。ReAct 是一条 reason-act 交替的单轨迹；LATS 在它之上加了树搜索、反思记忆和规划。直觉上 = CoT 的分步 + 树展开 + ReAct 式的环境反馈。
- **六个阶段**（课上用“走出迷宫”的例子过了一遍：昏暗的房间，左右各一扇门）
 1. *selection*: 按 UCT 选出要展开的节点。
 2. *expansion*: 从该节点采样若干动作（例中三个：开左门、开右门、检查房间找线索），逐个在环境里执行，把 observation 拼进上下文形成新状态。同一层的动作可以并行执行。
 3. *evaluation*: 给每个新状态算一个值，由两部分加权相加——LLM 自评（直接让模型给 0-1 的“有多大希望成功”）和 self-consistency 分（同一动作在多次采样中出现的频率，比如 50 次里占 75%）。前两讲里当基线的多数投票，在这里成了动作层面的价值信号。
 4. *simulation*: 从分最高的新节点一路往下展开，直到成功、失败或预算用完。
 5. *backpropagation*: 用轨迹的回报沿路径更新各节点的值和访问次数。
 6. *reflection*: 让模型写一段对这条轨迹成败原因的反思，附到后续迭代的上下文里。讲者说这一步对整体质量帮助很大。
- **三个式子**
 - 评估: $V(s) = \lambda \cdot LM(s) + (1-\lambda) \cdot SC(s)$
 - 选择: $UCT(s) = V(s) + w \cdot \sqrt{(\ln N(p) / N(s))}$, p 是 s 的父节点。前一项是利用，后一项是探索——一个节点相对父节点被访问得越少，探索项越大。只按当前 V 最大来选，会错过眼下分低、往后却可能更好的节点。
 - 回传: $V(s) \leftarrow (V_{old}(s) \cdot (N(s)-1) + r) / N(s)$, 就是按访问次数做的滑动平均。
- **结果**：HotpotQA（每题至少要检索两个 Wikipedia 页面）上，采样的轨迹越多成绩越高，加上反思再涨一截——更多的推理时算力被有效

换成了多步任务的成绩。WebShop（按一串约束购物：已组装的便携折叠桌、指定颜色和表面、限价）上不做微调，成绩接近人类专家。

- **优点与局限**：纯推理时、模块化、容易移植。代价是成本——每次展开和回传都在增加调用，而论文没做成本—收益分析。另一个没处理的前提是动作可撤销：付款这类不可逆操作真去执行后果很重，这套搜索没法直接套用。

小注：论文的 WebShop 表里 LATS 平均得分 75.9、成功率 38.0，人类专家 82.1 / 59.6——"接近专家"指平均得分，成功率差距仍不小。论文里反思只在轨迹失败时生成；limitations 一节也列了"开销更高"和"假设环境能回退到早先状态"；HumanEval (GPT-4, pass@1 92.7%) 和 Game of 24 课上没提。

- **问答**

- UCT 为什么是这个形式（提问听不清，从回答推断）：有理论直觉，但并没有证明它在这个场景下最优，要点是平衡探索和利用。
- 试过 UCT 以外的 bandit 算法吗：没有。multi-armed bandit 文献里其他的探索—利用策略都可以换进来，论文的贡献是搭了这个平台。
- 轨迹里重复出现的动作怎么算：同一父节点下再次采到同一动作，就累加它的计数，体现在 UCT 里；结构上假设的是一棵树，不是任意连通的图。

3. 论文二：SPRINT（上）——用改写过的轨迹教模型"并行地想" (NeurIPS 2025)

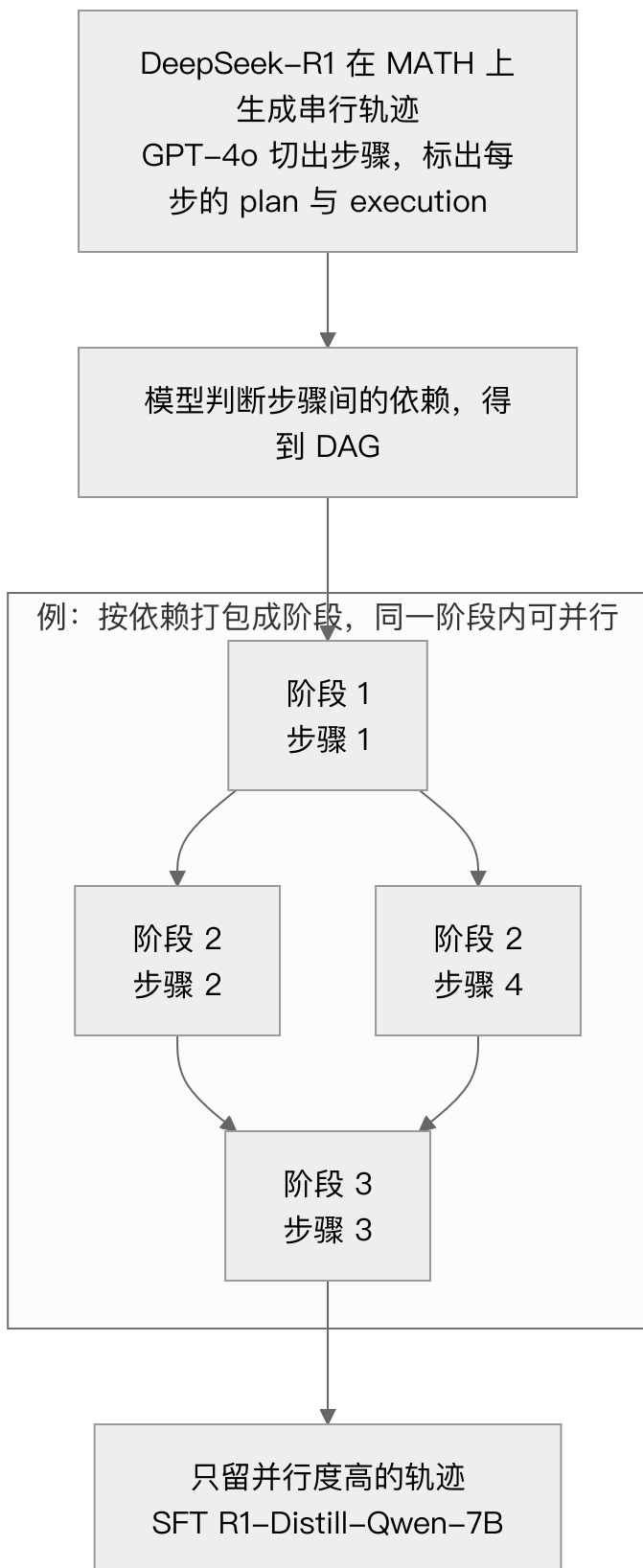


图 5-2 | SPRINT 的训练数据管线，中间是一个按依赖打包的小例子（自绘示意）·

► 看原幻灯片 28:01 · 出处: [Biju et al., 2025](#)

- **动机**：推理模型（o1、Gemini Thinking、Gemini 2.5 Pro 等）题越难想得越久，想得久又对应更高的准确率——DeepSeek-R1 的训练曲线上，AIME 准确率和平均回答长度是一起涨的。但长思维链里很多步骤其实互不依赖：换一种解法试试、把任务拆成几个子任务、回头验证前面的结果。这些没有必要一个 token 一个 token 地排队生成。
- **思路**：在后训练阶段教推理模型自己发现并行机会。推理时变成一个 planner 加一组可以并行的 executor，规划与执行多轮交替。和第 2 讲 Snell et al. 说的并行采样不同，这里是在一条推理轨迹内部把串行 token 换成并行 token。
- **数据管线**（图 5-2）：R1 照常生成串行轨迹 → GPT-4o 切成步骤，并标出每步里的 plan 与 execution（有的 plan 带多段 execution）→ 再让模型判断步骤间的依赖，得到 DAG（课上的例子：步骤 2 和 4 互不依赖，但都依赖步骤 1）→ 无依赖的步骤打包进同一阶段。思考内容没变，只多了并行结构的标签。
- **训练配方**：在 MATH 上生成约 6k 条轨迹，留下并行度高的，对 DeepSeek-R1-Distill-Qwen-7B 做 SFT。训练数据里，相互独立的 plan 1、plan 2 被并排写在一起，后面才是 execution 1、execution 2。

小注：论文里筛选后约剩 1,700 条；切步骤用 GPT-4o，建 DAG 用的是 GPT-4o-mini。

4. SPRINT（下）——推理时怎么跑，省了多少

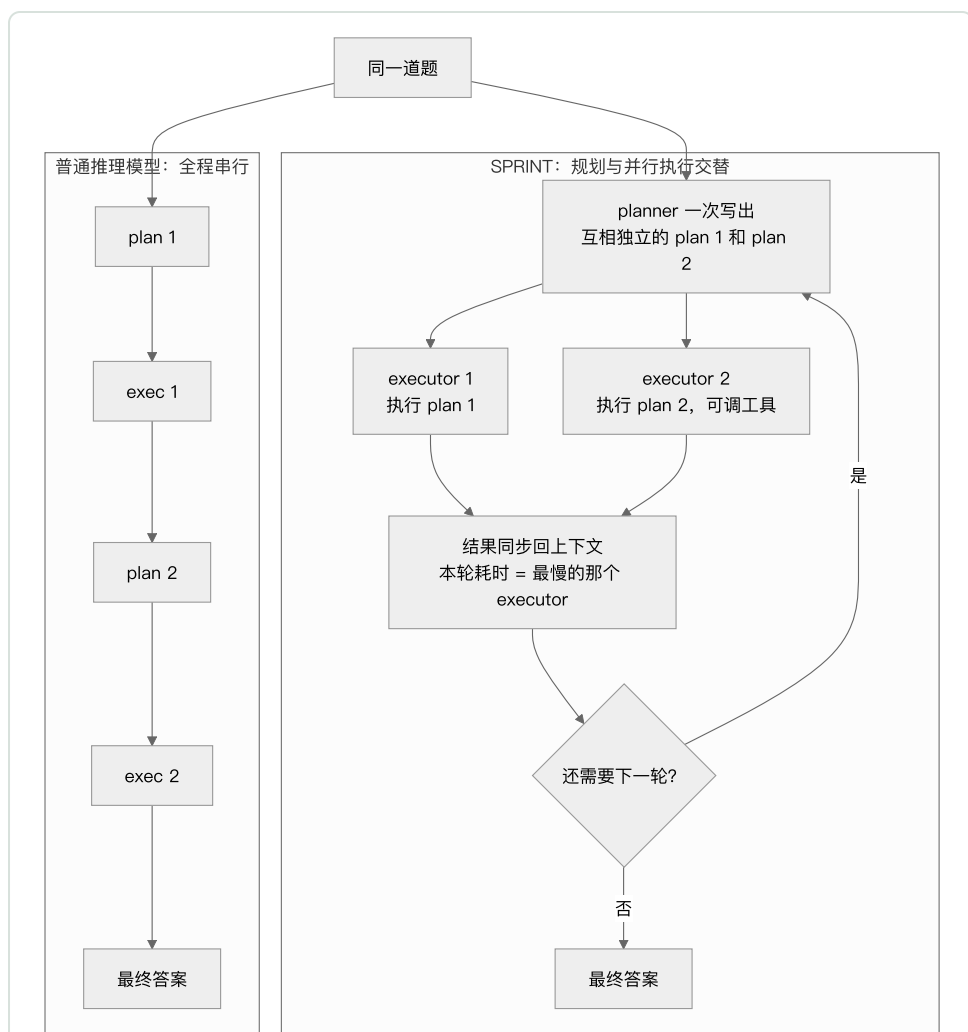


图 5-3 | 串行推理与 SPRINT 推理的对比（自绘示意）· [▶ 看原幻灯片 32:08](#) · 出处: Biju et al., 2025

- **推理流程：**模型先一次写出本轮相互独立的若干 plan（带标签）；运行时据标签分叉，多个 executor 同时执行（也可以是调 Python 计算器这类工具）；结果同步回上下文后，再写下一轮 plan 或给出答案。模型仍是 next-token prediction，结构没变；变的是它学会了“能一起出的计划就一起出”，标签则让外部系统得以并行。
- **时间账：**一轮的耗时取决于最慢的那个 executor，而不是各段之和。straggler 没法根除；论文的一个措施是把过于简单的 execution 并回 plan，凑出更大的并行块。
- **结果**
 - 初衷只是减少串行 token，结果准确率也涨了：比 R1-Distill-7B 高约 3.5 个点，串行 token 又远少于 32B 模型。讲者的解释是模型似乎受益于更结构化的思考方式；学生补充“同样预算下能探索更多思路”，讲者认为有可能。

- 域外泛化：只在 MATH 上训练，Countdown 和 GPQA Diamond 上的并行度和准确率也都更好。
- MATH 上和准确率相近的方法比，串行 token 少约 40%。
- 收益随推理长度增长：思考很短的题上，先规划再执行的额外开销可能让它比 RFT 基线还差；越是需要长思考的难题，省得越多。
- 两个观察：难题需要更多轮 plan—execute；前几轮并行度高（在探索），越往后 plan 越少（集中深挖一条路）。

小注：论文数字——MATH-500 上 SPRINT 92.5% / 平均 2,440 串行 token，RFT 91.0% / 2,880；在 RFT 要用 8,000 以上 token 的题上省 39%，GPQA 和 Countdown 的长轨迹上最多省 45% 和 65%。RFT 在论文里是 reasoning fine-tuned：用同一批 R1 原始轨迹（不改写成并行格式）做 SFT 的对照组；课上说成了 rejection fine-tuning。

- **为什么值得做**：多步推理的等待时间和成本已经是实际负担。讲者举了 Claude Sonnet 4.5 的例子：评测用的提示鼓励模型尽量多用工具、至少 100 次——任务的 token 规模正在远远超过 8–10k。

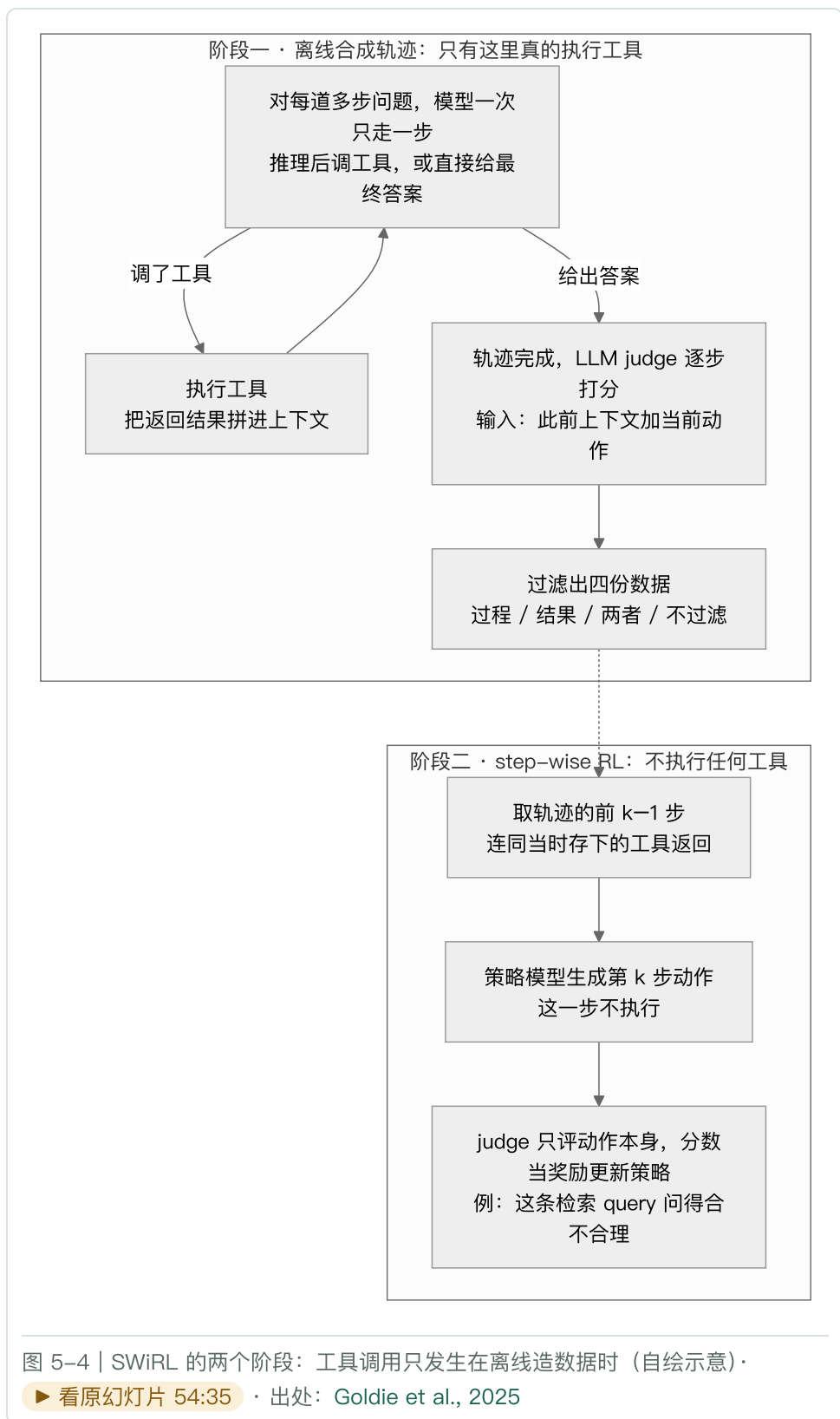
小注：这句提示出自 Anthropic 发布 Sonnet 4.5 时对 SWE-bench Verified 评测方法的说明（原意是尽量多用工具、最好超过 100 次），是评测时附加的一句提示；讲者说的是 system card。

• 问答

- 需要重新规划怎么办：全部上下文模型都看得到，想回头改计划照样可以。
- 各分支单看都对、合起来矛盾怎么办：所有分支最终汇入同一个上下文，模型要在作答前自己消解；串行推理同样会自相矛盾。而且实际看到的 plan 多是同一解法的不同步骤，不是同一问题的不同解法。
- 树有多宽：取决于任务，有的任务天生更可并行；讲者手头没有具体数字。

- **后续方向**：用 RL / GRPO 代替 SFT（泛化通常更好）；让不同工具的调用相互重叠；把 token 上的节省真正兑现为 wall-clock 加速。

5. 论文三：SWiRL（上）——合成多步轨迹，再逐步做 RL（COLM 2025）



- **动机与难点：**多跳问答、数学、软件工程、旅行规划、数据分析都需要一连串"推理 + 调工具"。难点有三：错误会沿步骤累积；训练时实时调工具又慢又脆（工具会失败、有 bug），而训练本来就耗时；RLHF、RLAIF、基于执行反馈的 RL（RLEF）基本都按单步任务设计——中间随模型怎么做，奖励只看最终答案。

- **设计目标：**模型要知道何时调工具、怎么写调用的 query、多步之间保持准确、出错后能恢复、知道何时该停下来给答案；训练过程中不调工具；学到的能力能迁移到新工具、新任务。
- **阶段一：离线合成数据。**用迭代式提示让模型一次只走一步：可以推理 (CoT)、调工具或给出最终答案；调了工具就执行，把环境返回拼进上下文，再问下一步。这样天然得到带步骤边界的轨迹，长度随题而变 (1 步、3 步、5 步……)。随后让 LLM judge 逐步打分，输入是此前的上下文加当前动作。全程离线、可大规模并行。最后有四种过滤：过程过滤 (每一步都被判为好)、结果过滤 (最终答案对)、两者都要、不过滤。
- **阶段二：step-wise RL。**训练样本是轨迹的每一个前缀：把题目、前 $k-1$ 步动作和当时存下的工具返回给模型，让它生成第 k 步——这一步**不执行**，直接由 judge 打分当奖励。目标是最大化"给定此前全部上下文时，单个动作的期望奖励"， k 从 1 取到 K 。工具调用全部发生在阶段一，RL 循环里没有任何工具。
- **judge 看不到工具输出，怎么打分 (学生追问)：**它评的是动作本身的质量。课上的例子是比较两个人谁更年长——"先去查第一个人的年龄"是不是一条合理的检索 query，不用知道查询结果就能判断；此前各步的工具返回已经在离线数据的上下文里。judge 没有经过训练，只靠提示。
- **推理时：**同样的迭代式提示。提示里写明可用的工具和标签格式 (算式写在指定标签里交给计算器；信息够了就用 answer 标签作答)，外部执行工具、回填结果、再次提示，直到模型给出答案。

6. SWiRL (下) ——实验结果

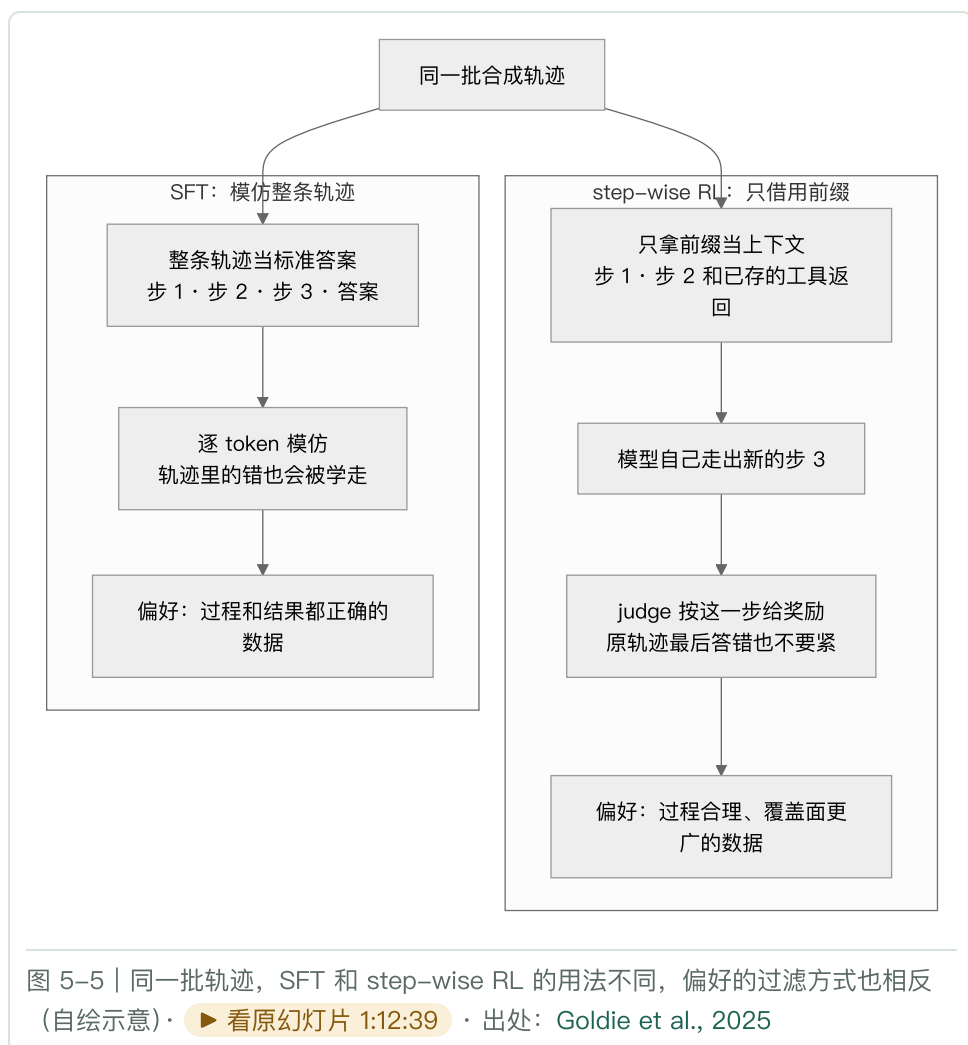


图 5-5 | 同一批轨迹，SFT 和 step-wise RL 的用法不同，偏好的过滤方式也相反（自绘示意）· [▶ 看原幻灯片 1:12:39](#) · 出处: Goldie et al., 2025

- **设置：**用 Gemma-2-27b 生成多步合成数据，题目来自 HotpotQA 和 GSM8K，约 50k 条，按四种过滤方式各出一份。

小注：论文里是 HotpotQA 1 万题 → 5 万条轨迹，GSM8K 7,500 题 → 37,500 条；judge 用的是 Gemini 1.5 Pro；问答数据集的“搜索”实际是向量库检索，GSM8K 用 SymPy 解释器当计算器。

- **只做过程过滤的数据最好**，好过结果过滤，也好过双重过滤——起初反直觉。讲者的解释：只留结果正确的轨迹，等于只留模型本来就会做的题；过程合理但结果错的轨迹，恰恰覆盖了它还不会的题。
- **跨任务、跨工具的泛化：**在 GSM8K + SymPy 计算器上训练，HotpotQA 从 65 升到 71；直接在 HotpotQA + 搜索上训练是 65 → 73。反方向同样成立。模型学到的不是某个工具的用法，而是“分步思考、在合适的时候调工具”这件事本身。

小注：论文 Table 2 的另一半——GSM8K 基线 0.65，在 GSM8K 上训练到 0.79，只在 HotpotQA 上训练也有 0.76（即摘要里的相对 +16.9%）。

- **数据规模**：在 HotpotQA + 搜索上训练，样本从 100 加到 10,000，GSM8K 这个完全不同的任务上成绩也持续上升。讲者称这是全文最重要的一张图：在容易造数据的环境里合成多步数据，能力可以迁移到全新的工具和领域，规模放大后可能是很强的方法。
- **为什么变好**：RL 之后模型每一步的平均过程奖励提高了，分布内（HotpotQA）和分布外（GSM8K）都是——每一步都想得更对了。
- **RL vs SFT**（图 5-5）：同一批数据也能做 SFT，但多步 RL 明显更好，而且两者偏好的数据相反——SFT 要过程和结果都正确的轨迹。SFT 是模仿学习，轨迹里的错会被一起学走；step-wise RL 只借用轨迹的前缀，模型自己走出新的一步并按这一步得奖励，所以能从不完美的轨迹里跳出来。
- **和课程主线的关系**：模型自己造轨迹 → 模型当 judge → RL 改进模型，又是一个自我改进回路（第 1 讲）；逐步打分的 judge 相当于一个不用训练的 PRM，"过程过滤最好"也和第 3 讲"过程监督优于结果监督"相呼应。

7. 三篇放在一起看

	LATS	SPRINT	SWiRL
发力阶段	推理时，不改权重	SFT，加推理时并行运行	RL 后训练
信号来源	环境 observation、LM 自评、自我反思	没有奖励，模仿改写后的 R1 轨迹	LLM judge 的逐步分数
换来什么	多步任务的成功率	串行 token 与延迟，顺带准确率	多步推理 + 工具的准确率、跨任务迁移
课上提到的代价	调用成本高；要求动作可撤销	并行度取决于任务；短推理反而吃亏；wall-clock 加速尚待兑现	课上没谈（见文末问题 3）

关键图表速查（点时间戳跳到原幻灯片）

图	看什么	跳转	出处
迷宫例子的六阶段走查	三个候选动作各自的 observation；A 分最高，被一路展开到标着出口的门	8:28	LATS
value / UCT / 回传三个式子	各管什么：V 负责评估，UCT 负责选择，回传是按访问次数的滑动平均	13:09	同上
HotpotQA 与 WebShop 结果	采样轨迹数增加时成绩上升，加上 reflection 再涨一截；WebShop（18:33 起）不微调、接近人类专家	17:23	同上

图	看什么	跳转	出处
R1 训练曲线两联图	一张是 AIME 准确率，一张是平均回答长度，两条一起往上走	24:22	应出自 DeepSeek-R1 (R1-Zero 的训练曲线)
SPRINT 数据管线	步骤标注 → DAG → 分阶段打包的三连图	28:32	SPRINT
串行 vs SPRINT 时间线	同样的 plan 1、2，执行段重叠之后总时长变短	32:08	同上
准确率 vs 串行 token	SPRINT 比 R1-Distill-7B 高约 3.5 个点，串行 token 远少于 32B	37:16	同上
省 token 幅度 vs 推理长度	最短一档可能为负（不如 RFT 基线），越长省得越多	45:13	同上
SWiRL 训练示意（谁更年长）	每步动作旁的 reward；环境返回来自离线数据，新动作不执行	58:30	SWiRL
四种过滤方式对比	只做过程过滤的那一组最高	1:06:56	同上
跨任务泛化表	HotpotQA: 65 → 71（只在 GSM8K 上训）对 65 → 73（在 HotpotQA 上训）；反方向同样成立	1:08:30	同上
数据规模曲线	训练样本 100 → 10,000，域外的 GSM8K 也一路上涨——讲者称全文最重要的图	1:10:04	同上

提到的工作

名称	在本讲里的作用
LATS (Zhou et al., 2023; ICML 2024)	论文一：MCTS + 环境反馈 + 反思的推理时搜索框架
MCTS / UCT	LATS 借用的搜索骨架与节点选择准则
Math-Shepherd (第 3 讲)	对照：验证器给推理步骤打分来引导搜索
ReAct (Yao et al., 2022)	LATS 的基础：reason-act 交替并读环境反馈；在第 4 次课的阅读清单里
Chain-of-thought、self-consistency、LLM-as-a-judge	LATS 的部件：分步推理、动作频率分、状态自评
multi-armed bandit 文献	问答里提到的其他探索—利用算法的来源
HotpotQA / WebShop	LATS 的评测；HotpotQA 也是 SWiRL 的训练与评测集
SPRINT (Biju et al., NeurIPS 2025)	论文二：交替的规划与并行执行

名称	在本讲里的作用
o1、Gemini Thinking、Gemini 2.5 Pro、DeepSeek-R1	"题越难想得越久"的推理模型；R1 也是 SPRINT 轨迹的来源
DeepSeek-R1-Distill-Qwen-7B / 32B	SPRINT 微调的基座 / 串行 token 的对照
GPT-4o	SPRINT 数据管线里的标注模型
RFT 基线	SPRINT 的主要对照（见第 4 节小注）
MATH / Countdown / GPQA Diamond / AIME	SPRINT 的训练集、两个域外评测；R1 训练曲线用的评测
Claude Sonnet 4.5 的评测提示	"工具用 100 次以上"——长程任务 token 规模的旁证
GRPO	SPRINT 后续可以换用的 RL 方法
SWiRL（Goldie et al., COLM 2025）	论文三：合成多步数据 + step-wise RL
RLHF / RLAIF / RLEF	被指为"按单步任务设计"的现有 RL 微调
Gemma-2-27b	SWiRL 合成数据所用的模型
GSM8K + SymPy	SWiRL 的数学训练、评测与计算器工具
ADaPT、AB-MCTS	在阅读清单里，课上没有讲

术语对照

English	中文
multi-step reasoning	多步推理
planning	规划
trajectory	轨迹
Monte Carlo tree search (MCTS)	蒙特卡洛树搜索
selection / expansion / evaluation / simulation	选择 / 展开 / 评估 / 模拟
backpropagation（MCTS 语境）	回传（不是神经网络的反向传播）
reflection	反思
UCT (upper confidence bounds applied to trees)	树上的置信上界
exploration vs exploitation	探索与利用
value function	价值函数
visit count	访问次数
observation	观察（环境返回）
multi-armed bandit	多臂老虎机
irreversible action	不可逆动作
large reasoning model (LRM)	大型推理模型

English	中文
sequential tokens	串行 token
planner / executor	规划器 / 执行器
interleaved planning and execution	规划与执行交替
DAG (directed acyclic graph)	有向无环图
packing	打包（无依赖的步骤并入同一阶段）
straggler	拖后腿的慢任务
wall-clock time	实际耗时
out-of-domain generalization	域外泛化
step-wise RL	逐步强化学习
synthetic data	合成数据
iterative prompting	迭代式提示
process / outcome filtering	过程过滤 / 结果过滤
tool call	工具调用
multi-hop question	多跳问题
imitation learning	模仿学习
mean process reward	平均过程奖励

字幕勘误

"LATs" → LATS；"UTC" → UCT；"reasoning theorists" → 应为 reasoning steps / trajectories（推断）；"Amy" → AIME；"banded / embedded learning""multi-armed bounded" → bandit learning、multi-armed bandit；"Gemini Think" → Gemini Thinking；"spring" → SPRINT；"GPQ domain" → GPQA Diamond；"infant stage" → inference stage；"exit 1" → exec 1；"outfitted together" → outputted together；"proping" → prompting；"soil of generalizes" → SWiRL generalizes；"COLM in Conference in Language Modeling" → COLM（Conference on Language Modeling）；"HotPotQA" 即 HotpotQA；"rejection fine tuning" → 论文里 RFT 指 reasoning fine-tuned 基线（见第 4 节小注）。

带走的问题

1. LATS 默认环境可以回滚。动作不可逆（付款、发消息）时，树搜索还剩多少能用——靠世界模型在"想象"里搜，还是只在可逆的子空间里

搜？多出来的调用成本又怎么和成功率的提升折算？

2. SPRINT 的并行结构是从已有串行轨迹里"挖"出来再模仿的。改用 RL、把延迟直接写进奖励，模型会不会找到 R1 轨迹里根本没有的分解方式？准确率的提升来自更结构化的思考，还是同样预算下更多的探索？
3. SWiRL 的 judge 不看工具返回，只评动作"看上去是否合理"。什么情况下这会失效（query 合理，但在这个环境里查不到东西）？judge 既做过滤又当 RL 奖励，会不会出现第 3 讲讨论过的 reward hacking？
4. 数据规模曲线只画到 10,000 条、两个任务、两种工具。继续放大、换成更杂的环境，跨任务迁移还会一直涨吗？"在容易造数据的环境里学多步能力"的边界在哪？

CS329A 第 6 讲 | 训练时扩展与 RL 的规模化 (Train Time Scaling / Scaling RL)

Stanford CS329A: Self-Improving AI Agents (2025 秋) · 对应课表第 6 次课 (10 月 10 日, 周五) ——讲者开场就说这是第六讲, 所讲三篇论文与该次课的指定阅读完全一致 视频: <https://www.youtube.com/watch?v=yVnmHSAy3ck> (1:12:39, 自带人工校对英文 CC, 学生提问处多为 [INAUDIBLE]) 讲者: Aakanksha Chowdhery (推断: 没有自我介绍; 她说上周五讲过用执行反馈做 RL 的论文, 即第 4 讲的 RLEF, 并预告下周五由自己讲 AlphaCode 等) 配套阅读: STaR: Bootstrapping Reasoning With Reasoning (Zelikman et al., 2022) · DeepSeekMath (Shao et al., 2024, 提出 GRPO) · DAPO: An Open-Source LLM Reinforcement Learning System at Scale (Yu et al., 2025)

一句话: 前几讲的 test-time scaling 是"多采样、挑一个", 模型本身没变; 这一讲把挑出来的好输出训回模型, 让自我改进的环真正合上。三篇论文层层递进: STaR 用"答对才保留 + 给答案倒推过程"做最朴素的自举 → DeepSeekMath 用 GRPO 去掉 critic, 让在线 RL 在有限显存里跑得起来 → DAPO 补齐长推理链 RL 里没人明说的四个工程细节, 在 Qwen-32B 上把 AIME 从 30 分推到 50 分。结论很克制: 这些方法抬高的是 maj@K (更稳定), 不是 pass@K (更聪明)。

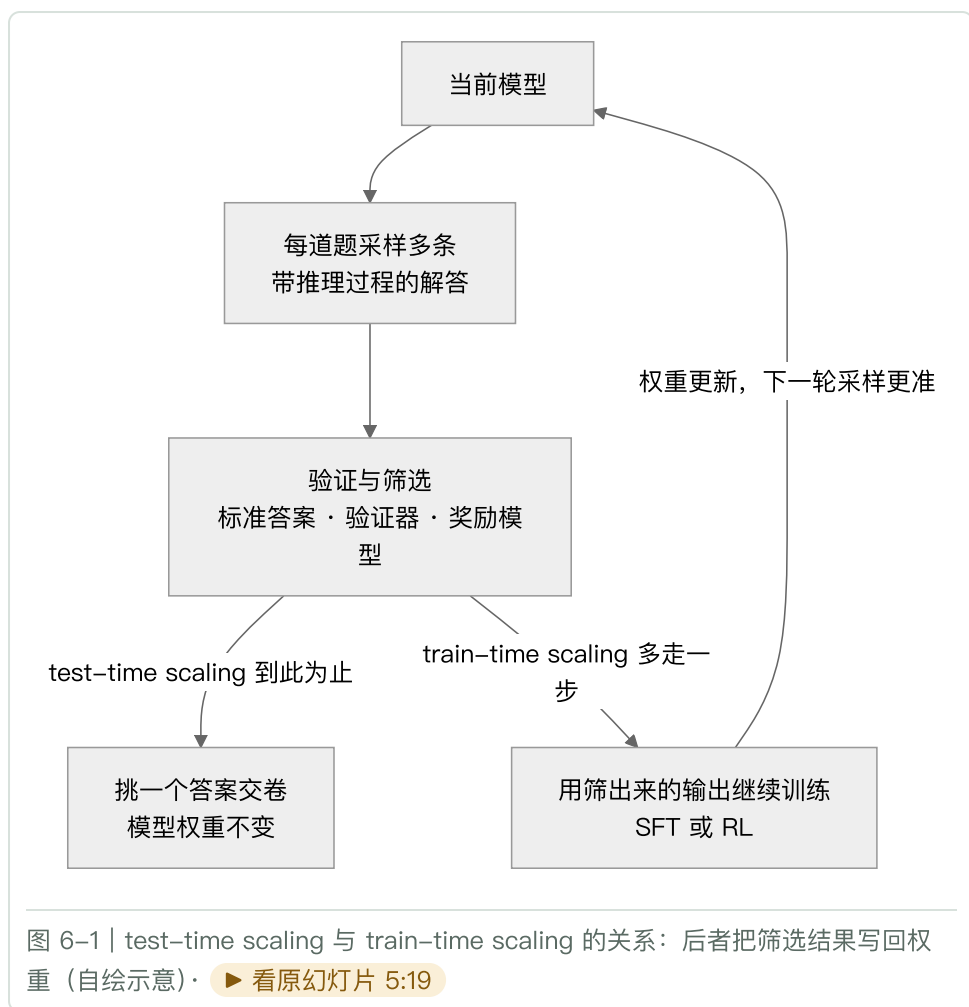
时间轴

时间	内容
0:05	开场: 第六讲, 三篇论文
1:07	动机: AIME 上 7B / 32B 的模型为什么能这么强
3:14	三个要点: 四种范式与 train-time scaling 的闭环
6:10	o1 的两张 scaling 图; 数学可验证, 所以环能合上
7:40	推理模型的行为模式与 o1 示例; 分领域胜率
11:32	问答: test-time 和 train-time 谁更划算; 训难题会不会伤到简单题
16:09	论文一 STaR: 动机与现有做法的不足
17:42	核心循环、rationalization、三条假设
21:17	问答: 答案从哪来、rationale 要不要过滤、负样本
25:10	算法细节与实验设置 (GPT-J、三个数据集)
28:30	结果: 数据利用率、见顶、人工评 rationale、GSM8K

时间	内容
32:00	小结：优点与局限；问答：小模型为什么不直接蒸馏
36:54	后续工作 V-STaR、Quiet-STaR；课堂讨论 STaR 的上限
40:50	论文二 DeepSeekMath：7B 在 MATH 上的跳变
41:50	数据与起点：Common Crawl 数学网页、从 code 模型出发
43:50	PPO 的四个模型 vs GRPO 的组内基线
46:25	统一视角：数据来源 × 梯度系数
47:45	问答：奖励要有分布、KL 项、要不要更新全部权重、连续分数
52:19	结论：online 胜 offline；maj@K 涨、pass@K 不涨
53:10	论文三 DAPO：朴素 GRPO 在 Qwen-32B 上的症状
54:00	补丁一 Clip-Higher
55:22	补丁二 Dynamic sampling
57:32	补丁三 Token-level loss
58:45	补丁四 超长截断的处理
59:45	消融 30 → 50；训练时该盯哪些指标
1:01:15	SFT 还是 RL；三种方法怎么选；什么会涨、什么不会
1:04:25	pass@K 为什么难；奖励的两头风险；可验证信号从哪来
1:06:38	开放问题与研究方向
1:08:05	问答：RL 占多少算力；没见过的知识学不会；题少怎么办

核心内容

1. 动机：小模型靠"把自己的输出训回去"变强



- **引子：**第 1 讲说参数越多能力越强，但在 AIME 上，GPT-3.5（据信 175B）只有约 5%，7B 的 DeepSeekMath 却有 51.7%（加技巧约 60%），Qwen-32B 加 DAPO 是 50%。之所以用 AIME 2024 / 2025（也是作业 1 的基准），是因为 MATH 已经饱和、且被多数模型的训练数据污染。

小注：DeepSeekMath 论文里的 51.7% 和 60.9%（64 个样本做 self-consistency）是 **MATH** 上的数字，不是 AIME——后面讲到该论文时讲者说的也是 MATH。这一页混放了不同基准的数字，只能当“小模型也能很强”的示意。

- **三个要点：**① 让模型从自己的输出里学（先聪明地过滤），还能继续变强——也就是第 1 讲的自我改进回路；② 花在“用自身输出训练”上的算力可以顶替参数量；③ RL 不同于监督学习，规模一上去，实现里的小修小补会决定成败。
- **四种范式：**pre-training；fine-tuning（RLHF / RLAIIF，得到聊天模型）；test-time scaling（多数投票等推理时技巧）；train-time

scaling——把经过 test-time scaling 筛选的输出拿回去继续训练。
讲者说，别的都忘了也行，记住这个环就够了。

- **o1 的两张图**：AIME 的 pass@1 对训练算力、对推理算力都近似 log-linear 上升，两条轴可以同时加。它在数学上行得通，是因为数学可验证（第 3 讲）——知道哪些输出是对的，才有东西可以喂回去。

2. 推理模型在想什么，哪些领域受益

- 思维链里常见的模式：问题分析、任务分解、自我评估、发现错误后回溯、同时试几条思路。o1 的例子：写 bash 脚本转置矩阵，先弄清输入输出格式，再拆成"解析 → 建矩阵 → 转置 → 按原格式输出"；算溶液 pH 时中途自己叫停、换了公式。
- 分领域胜率（o1 对 GPT-4o，人类偏好）：编程、数据分析、数学计算明显高于 50%，个人写作和改稿几乎不涨——能验证、能闭环的领域受益最大。

3. 问答：test-time 和 train-time 各管什么

- 有学生注意到 o1 图里 test-time 那张看起来涨得更多。讲者：两者没有"谁该更好"的直觉，这只是一个实例；还顺口调侃了一句图未必画得对。

小注：两张图的横轴分别是训练算力和推理算力，都是对数刻度，彼此没有可比的单位，本来就不能逐点比斜率。

- 讲者的定位：**test-time compute 便宜**——模型训好后可以反复推理，第 2 讲的 coverage 结论保证多采总能采到对的，只要验证器够好，搜索理论上可以无限加（下周五讲 AlphaCode 等）；但它只在验证足够稳健的基准上好用，好比往墙上扔意面，得事先知道哪根该粘住。**train-time scaling 抬的是 pass@1**，让模型一次就更可能答对；代价是要"扩得对"，闭环里得有足够多的成功样本，也同样离不开验证。
- 训难题会不会让简单题退步：一般不会，除非推理链本身坏掉了，比如不断重复的 overthinking。

4. 论文一：STaR (Zelikman et al., 2022) ——答对才留，答错就给答案倒推

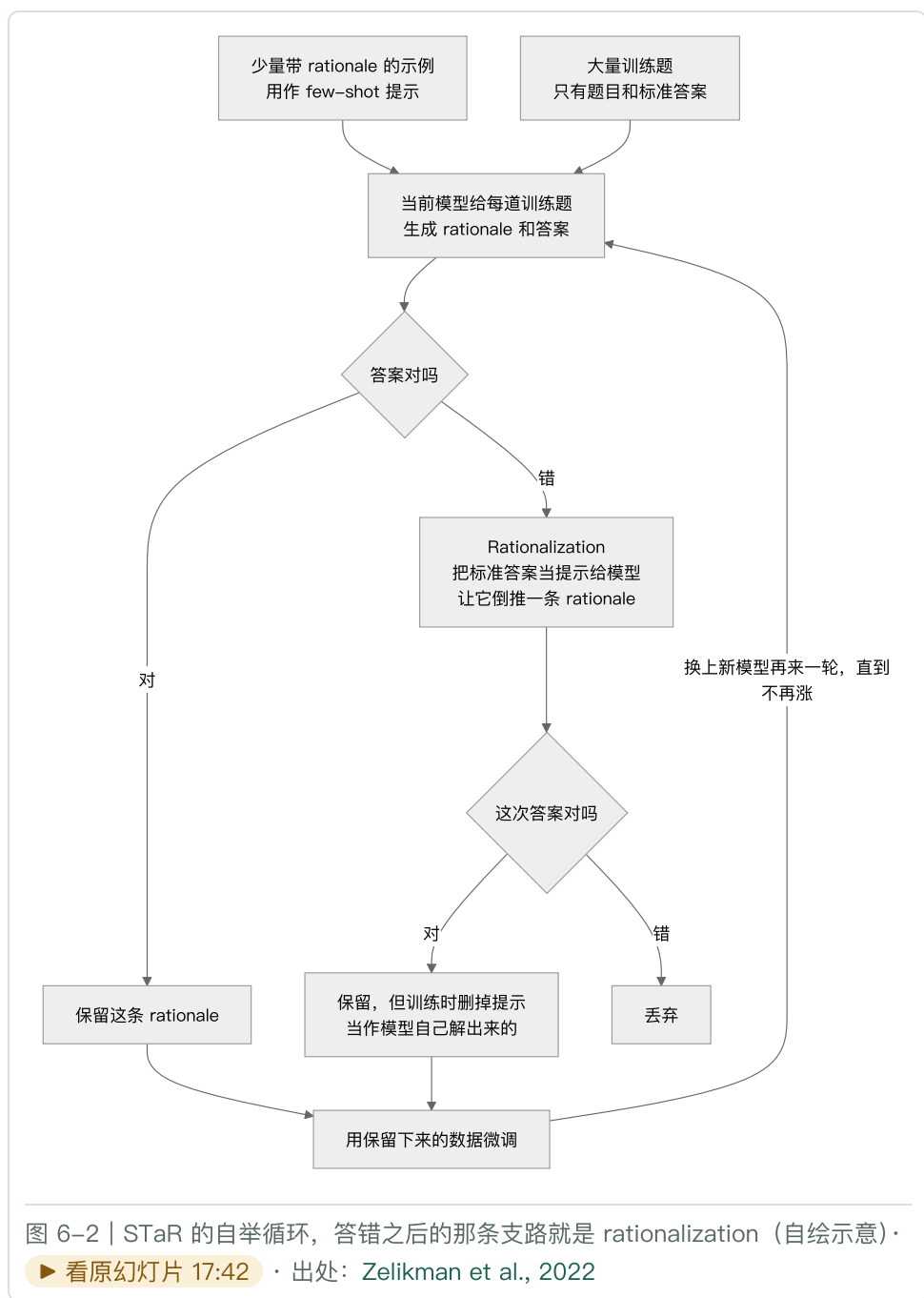


图 6-2 | STaR 的自举循环，答错之后的那条支流就是 rationalization（自绘示意）·

► 看原幻灯片 17:42 · 出处：Zelikman et al., 2022

- **动机**：CoT 能提升推理，但带推理步骤的数据极少——互联网上几乎没有，人工写太贵，按模板自动生成只适用于很窄的领域；few-shot 提示又打不过更大（不带推理过程）的数据集上直接微调。
- **主循环**：训练题自带答案、但没有推理过程。few-shot 提示模型写出"rationale（推理过程）+ 答案"，只留答对的拿去微调，换上新模型再来一轮。
- **Rationalization**：只学答对的题，模型永远碰不到它不会的题，那部分没有任何信号。做法是把正确答案当提示交给模型，让它倒着写出推理；训练时把提示删掉，当作模型自己解出来的。训练集由此扩展到更难的问题。

- **定性与假设：**讲者说这几乎可以算一种极简的 off-policy RL。它依赖三条假设：① 终答案正确 $\approx$ 推理质量好（数学上大体成立）；② 看到答案后，模型写得出站得住的推理；③ 初始模型足够强，能从 few-shot 起步——题目远超模型能力就寸步难行，这也是要迭代的原因。
- **实验：**GPT-J (6B)；外层循环若干轮，内层训练步数逐轮增加（起步要慢）。数据集：GSM8K（约 9k 题）、CommonsenseQA（多选常识题）、多位数加法（合成）。
- **结果**
 - CommonsenseQA：带 rationalization 的 STaR 达到 72.5%，只用到约 86% 的训练题；总体只有 70%–87% 的数据最终被用上，其余始终没得到正确解。
 - 迭代几轮后见顶——讲者说它毕竟不是真正的 RL，轮数要调。
 - 请人工评判 rationale：STaR 生成的质量相当合理。
 - GSM8K 上 rationalization 几乎不加分；讲者的解释是题目对模型偏简单时，逼它倒推过程没有额外收益（模型自己用的计算步数和标准解法差不多）。

小注：论文的表——CommonsenseQA：直接微调 60.0%，STaR 68.8%（用 69.7% 的数据），加 rationalization 72.5%（86.7%），接近 30 倍大的 GPT-3 微调的 73.0%。GSM8K：直接微调 5.8% $\rightarrow$ STaR 10.1% $\rightarrow$ 加 rationalization 10.7%，只用到 25%–29% 的训练题，其中靠 rationalization 新增的仅 0.5%。讲者此处口头说的“51.7%”应是口误（那是 DeepSeekMath 的数字）。另外，论文每一轮都从原始预训练模型重新微调，而不是接着上一轮的权重训，以防过拟合。

- **局限：**rationale 没有好的自动评估，要么人看、要么上 PRM；只按终答案过滤，会混入“过程错、答案对”的样本（第 3 讲的 false positive），也等于只留下模型本来就会的题（第 5 讲 SWIRL 有同样的观察）；few-shot 示例的写法会把风格带进训练数据。
- **问答**
 - 倒推出来的 rationale 要不要再过滤：本文没有，后续工作有；可以用 PRM 逐步检查——被点名为项目选题。模型若学到“错过程 + 对答案”，这个框架里没有机制能纠正。
 - 答错的样本能不能用：从正样本学已经成熟，从负样本学还没做好，本讲不展开。
 - 小模型为什么不直接蒸馏前沿模型：讲者承认实践中蒸馏更划算；但这条线要回答的是模型怎样靠自己的输出变强——提问的学生自

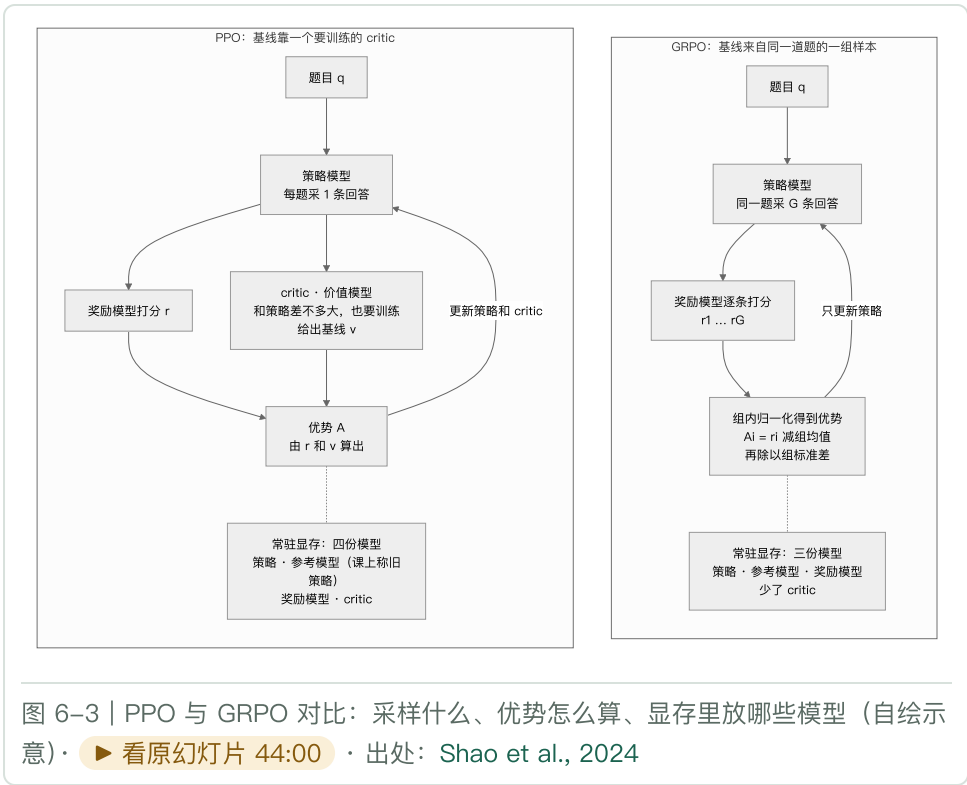
己也点到，训练最前沿的模型时没有更强的老师可蒸馏。

- 后续工作：V-STaR 在环里加了一个一起训练的验证器；Quiet-STaR 让模型"在内部思考"。

小注：Quiet-STaR 的内部思考仍是采样出来的 token（每个位置后并行生成一小段不外显的 thought），MLP 只是混合"有 / 无 thought"两种预测的 mixing head，并非在连续隐空间里推理。V-STaR 则把 STaR 丢掉的错误解也拿来训练验证器。

- 课堂讨论"STaR 的上限由什么决定"：学生的回答——不会出现训练数据之外的逻辑跳跃；受限于基座的推理能力，给了提示也推不出的题永远进不了训练集；有些答案天然好倒推（225 容易想到 15×15 ），有些不行。讲者：都对；进入 RL 之后也要记着，**基座模型能做什么是一切的地基**。

5. 论文二：DeepSeekMath 与 GRPO (Shao et al., 2024)



- 看点：**MATH 的 top-1 准确率过去大体随模型变大而缓慢上升，DeepSeekMath-7B 出现时突然跳了一截。讲者认为关键在于他们把闭环里 RL 这一步做对了。
- 第一步，先把基座在数学上喂强。**此前的代表做法是在 PaLM 上继续训练 arXiv 等科学数据（应为 Minerva，字幕漏了论文名）。DeepSeek 发现 arXiv 论文帮助不大；他们以 OpenWebMath 为种

子，从 Common Crawl 里挖数学网页，覆盖更广、token 多得多；并且从 DeepSeek-Coder 出发——先学代码再学数学明显更好，推理和用工具都受益，讲者说这是第一次有人证明这一点。这也回应了上一节的讨论：基座在某个领域不够强，就得先补。

小注：论文里这份语料叫 DeepSeekMath Corpus，约 120B token，用 fastText 分类器从 Common Crawl 迭代召回；基座是 DeepSeek-Coder-Base-v1.5 7B；继续预训练后的 Base 7B 在 MATH 上已经超过 Minerva 540B。

- **第二步，SFT 之后上 RL。**RLHF 常用的 PPO 要同时放四个模型：旧策略、正在学的新策略、给出基线的 critic、打分的奖励模型——7B 还行，再大显存就吃紧。GRPO 去掉 critic：同一道题采样一组回答，奖励模型逐个打分，**优势 = (该回答的奖励 - 组内均值) ÷ 组内标准差**。道理在于奖励模型本来就是用"问题回答之间的比较"训练出来的，用组内相对值正好对口；省下一个模型，RL 才能往大了做。

小注：按论文的图，PPO 的四个模型是 policy、value（这两个要训练）和 reference、reward（冻结）；GAE 是 PPO 借助 value model 算优势的方法，GRPO 恰恰是用组内归一化把它换掉——讲者口头说 GRPO 用了某种 GAE，应为口误。论文每题采 64 条回答；KL 项直接加在损失里，而不是混进奖励。

- **结果：**MATH 46.8% → 51.7%，讲者称它是第一个在 MATH 上过 50% 的 7B 级开源模型。
- **统一视角：**各种方法的差别只在"数据从哪来"和"梯度系数怎么算"。

方法	数据来源	奖励	梯度系数
STaR 一类	离线：生成一次	对 1、错 0	只强化答对的，答错的直接丢
Online rejection fine-tuning	在线：从当前模型采样	对 1、错 0	同上
GRPO	在线，每题一组	奖励模型打分	组内归一化的优势，有正有负、有大有小

结论：从当前模型在线采样的 RL 环，胜过 STaR 式的离线做法。

- **涨的到底是什么：**多次采样时 maj@K 提高了，pass@K 没有——是大多数样本变对了，而不是原来做不出的题现在做得出了。讲者的总结：模型变得更稳定，不是从根本上更聪明。换成第 2 讲的话：RL 没有抬高 coverage，只是把已有的 coverage 兑现成了多数票。

6. 承上启下：奖励必须"有对有错"

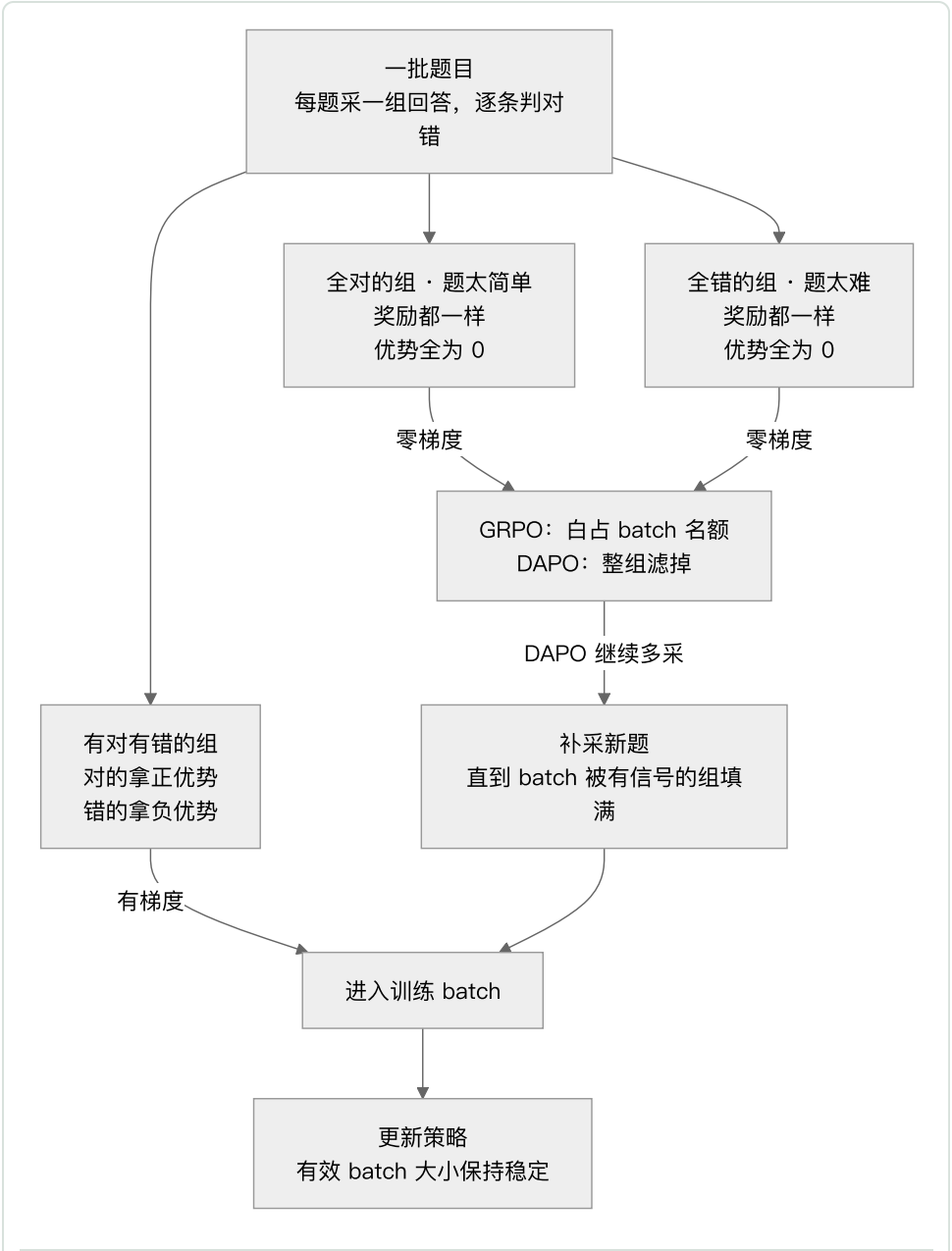


图 6-4 | 组内归一化下，全对和全错的组没有学习信号；DAPO 的 dynamic sampling 把它们滤掉再补采（自绘示意）· [▶ 看原幻灯片 48:30](#) · 出处: Shao et al., 2024; Yu et al., 2025

- 起因还是"会不会在简单题上退步"那个问题。讲者换了个角度回答：要让闭环转起来，一批题上的奖励必须有分布。简单题永远全对、太难的题永远全错，组内归一化之后优势全是 0，模型什么也学不到——它需要能爬坡的信号。这个缺口留给下一篇来补。
- 一位学生补充：防止模型偏离原有能力，靠的是目标里的 KL 散度项。讲者认可。

小注：GRPO 保留了 KL 项；DAPO 则认为长 CoT 训练中模型本来就该大幅偏离初始分布，直接把 KL 项删了。

- 加一个新能力要不要更新全部权重：不一定，近期有博客表明用 LoRA 这类低秩更新也行（字幕作 lower updates，应为 LoRA；所指应是 Thinking Machines 的 LoRA Without Regret，推断）。
- 0 / 1 之外的连续分数从哪来：训练一个奖励模型；只有 0 / 1 也能用，对一组样本取平均即可。

7. 论文三：DAPO (Yu et al., 2025) ——长推理链 RL 的四个补丁

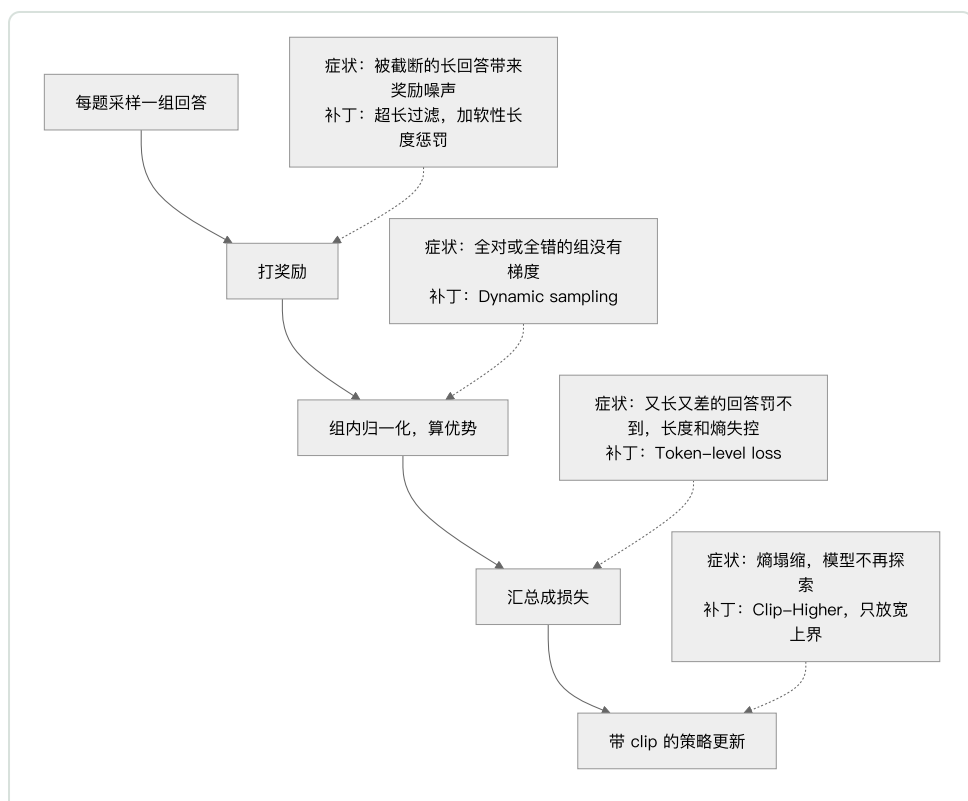


图 6-5 | DAPO 的四个补丁分别打在 GRPO 训练步骤的哪一环（自绘示意）·

► 看原幻灯片 59:45 · 出处：Yu et al., 2025

- **问题**：把 GRPO 原样搬到 Qwen-32B 上，AIME 只有 30 分，症状是熵塌缩（模型过度自信、不再探索）、训练不稳、回答长度失控。DeepSeek 自己做到过 47 分，但细节没有写出来；DAPO 的贡献就是把这些技巧明说并开源。

小注：47 分指 DeepSeek-R1 论文里直接在 Qwen2.5-32B 基座上做 RL 的 R1-Zero-Qwen-32B，不是从 R1 蒸馏出的 32B（讲者后面口头说成了 distilled）；DAPO 只用了它一半的训练步数。DAPO 的奖励是纯规则的（对 +1、错 -1），不训练奖励模型；训练集是自建的 DAPO-Math-17K，

AIME 2024 只用于评测 (avg@32); 每题采 16 条回答, 课上举例的 64 是 DeepSeekMath 的设置。

• 四个补丁

1. **Clip-Higher (非对称裁剪)**: PPO 的 clip 对上调和下调一视同仁, 低概率 token 每一步能涨的幅度很小, 探索很快就没了。办法是把上界放宽。效果: 准确率更高, 熵保持平稳而不是塌到 0——熵是“还剩多少探索”的代理指标。

小注: 论文取 $\epsilon_{\text{low}} = 0.2$ 、 $\epsilon_{\text{high}} = 0.28$; 下界不动, 是怕把低概率 token 直接压到 0。

2. **Dynamic sampling** (图 6-4): 多采一些题, 把全对和全错的组滤掉, 只留有信号的组, 直到填满 batch——不浪费梯度, 有效 batch 大小保持稳定。有学生问每题 64 个样本是不是太少: 讲者说数字只是举例, 取决于基准难度和基座能力。

3. **Token-level loss**: 样本级损失里每条回答权重相同, 一条很长的垃圾回答和一条很短的好回答等权。改成按 token 计损失之后, 熵和平均长度不再失控上冲。

小注: 论文的解释是, 样本级平均让长回答里每个 token 分到的权重变小, 于是长而优质的推理学得不够, 长而重复、胡言乱语的输出又罚得不够。

4. **超长样本**: 难题上推理链越写越长、被截断, 截断样本带来大量噪声。DAPO 用超长过滤, 再加一个随长度逐渐加重的软惩罚; 别的论文则选择在 RL 过程中逐步放宽上下文长度。

- **消融 (AIME 2024)**: 朴素 GRPO 30 → 超长过滤 36 → Clip-Higher 38 → 软性超长惩罚 41 → token-level loss 42 → dynamic sampling 50。
- **训练时该盯什么**: loss 本身不够。要同时看回答长度、熵 (不能太低也不能太高)、准确率为 1 的样本占比 (它决定要多采多少)。长度爆炸就回头管损失, 熵出问题就去管熵; 若干步后不再进步, 可能是奖励信号已经饱和。

8. 三种方法怎么选; 什么会涨、什么不会

- **SFT 还是 RL** (回应前面学生的提问): 奖励信号强的领域, RL 用更少的样本就能持续爬坡, 但要花很多功夫才能调对; 手里有大量高质量数据时, SFT 往往更快见效, 只是它不太能带来或放大推理能力。

方法	适用前提	适合的任务
STaR	只有约一百条带推理的样例，没有 RL 基础设施	GSM8K 这类简单推理；有提升，但不出基座的能力圈
GRPO (DeepSeekM ath)	基座够好，并先用指令数据预热；GPU / 显存有限	标准数学推理，已被充分验证
DAPO	推理链很长、要 SOTA；算法和基础设施里的每个变量都得管住	AIME、IMO 级的竞赛题

- 三者都会提升：maj@K、答案格式、多步推理的连贯性。三者都还不会提升：根本能力、解决新问题、域外泛化。

9. 开放问题与最后的问答

- 要让 pass@K 也涨，等于提升模型解决新问题的根本能力；讲者的看法是，这类台阶式的跃升历来来自某种突破，或者某个维度上的 scaling。
- 整个环两头都受制于奖励：策略太强，奖励会被 hack；奖励信号不足，就爬不动坡。数学和代码有现成的可验证奖励（终答案、第 4 讲 RLEF 用的执行反馈、单元测试），可这样的领域有多少？什么样的验证信号才算够好？能不能用一组验证器互补（第 3 讲的 Weaver）？
- 讲者列的三个开放问题：为什么只有 maj@K 涨；回溯、自检这些行为是真的涌现，还是基座里本来就有、只是统计上变得更常见；怎样从失败里学（目前基本是直接滤掉）。可做的方向：更好的训练数据、对奖励噪声稳健的算法、奖励的设计、把 STaR 的 rationalization 和 DAPO 结合。

小注：DAPO 论文的 case study 称，反思和回溯在训练早期几乎不出现、随 RL 推进才出现——算“涌现”一方的证据，但不是对照实验。

- RL 占总训练算力多少**：讲者的粗略印象是去年约 1%，现在大概 5%；Grok 4 宣称做到了五成左右，却没有换来相称的跃升——瓶颈正是奖励不够强、有噪声。Anthropic 没有公开过这个比例。
- 模型从没见过的知识（比如微积分），RL 能教会吗**：不能。RL 是让模型把已经会的做得更好——在已知的空间里更好地探索和搜索。
- 像 AIME 这样题目很少，信号够吗、会不会泄题**：RL 本身更省数据；该问的不是“数据够不够多”，而是“有没有足够多能爬坡的题目”（dynamic sampling 还会再滤掉一批）。验证信号取决于验证器的质量，可以上多个验证器。

关键图表速查（点时间戳跳到原幻灯片）

图	看什么	跳转	出处
AIME 对比页	GPT-3.5 约 5%，7B 的 DeepSeekMath 和 32B 的 DAPO 在 50% 上下；注意各数字并非同一基准（见第 1 节小注）	1:07	讲者汇总
o1 的两张 scaling 图	左：训练算力，右：推理算力；纵轴都是 AIME pass@1，横轴对数，两边都近似直线上升	6:10	OpenAI, 2024
分领域胜率	o1 对 GPT-4o：编程、数据分析、数学计算高于 50%，个人写作和改稿基本持平	10:45	同上
STaR 流程图	主循环之外那条"给答案当提示 → 倒推 rationale"的支路	25:10	Zelikman et al.
CommonsenseQA 结果表	STaR 加 rationalization 72.5%，只用约 86% 的训练题，对比用全量数据的直接微调	30:10	同上
MATH top-1 随时间的走势	前面随模型变大缓慢爬升，DeepSeekMath-7B 处陡升	41:10	Shao et al.
PPO vs GRPO 结构图	PPO 多出来的那个 critic；GRPO 用一组样本的均值和标准差代替它	44:00	同上
统一视角表	三行方法只差两栏：数据是离线还是在线、梯度系数是 0 / 1 还是归一化优势	46:25	同上
Maj@K vs Pass@K	RL 之后 Maj@K 曲线上移，Pass@K 两条线基本重合	52:35	同上
Clip-Higher 前后	紫线 = 用了 Clip-Higher：左图 AIME 准确率更高，右图熵保持平稳；不用则熵塌到接近 0	54:45	Yu et al.
Token-level loss 前后	不用 token-level loss 的那条线：熵和平均回答长度一起失控上冲	58:15	同上
DAPO 消融表	30 → 36 → 38 → 41 → 42 → 50，最后一步 dynamic sampling 贡献最大；对照线是 DeepSeek 的 47	59:45	同上

提到的工作

名称	在本讲里的作用
STaR (Zelikman et al., 2022)	论文一：答对才保留 + rationalization 的自举微调
V-STaR (Hosseini et al., 2024)	在 STaR 的环里加一个一起训练的验证器
Quiet-STaR (Zelikman et al., 2024)	让模型"在内部思考"的后续工作
DeepSeekMath / GRPO (Shao et al., 2024)	论文二：数学语料 + 去掉 critic 的在线 RL

名称	在本讲里的作用
DeepSeek-Coder	DeepSeekMath 的起点模型；"先代码、后数学"
OpenWebMath、Common Crawl	数学语料的种子与来源
Minerva (Lewkowycz et al., 2022, 应为)、PaLM	此前"在 arXiv 等科学数据上继续训练"的代表
PPO (Schulman et al., 2017)、RLHF / RLAIF、DPO	GRPO 的对照；四个模型的显存负担
Online rejection fine-tuning (论文中记作 Online RFT)	统一视角里介于 STaR 和 GRPO 之间的形态
DAPO (Yu et al., 2025)	论文三：四个补丁与开源的训练系统
DeepSeek-R1 (R1-Zero-Qwen-32B, 应为)	DAPO 的对照：Qwen-32B 上 47 分
OpenAI o1 (Learning to Reason with LLMs)	两张 scaling 图、思维链示例、分领域胜率
GPT-3.5、GPT-4o、GPT-J、Qwen-32B (应为 Qwen2.5-32B)、Gemini Flash Thinking、Grok 4	被提到的模型
AIME 2024 / 2025、MATH、GSM8K、CommonsenseQA、多位数加法、IMO	基准与任务
RLEF (第 4 讲)	代码领域用执行反馈当可验证奖励的例子
Weaver (第 3 讲)	用一组验证器弥补单个验证器的盲区
AlphaCode	预告：下周五讲"搜索可以无限加"的例子

术语对照

English	中文
train-time / test-time scaling	训练时 / 测试时扩展
rationale	推理过程 (解题思路)
rationalization	事后补推理 (给定答案、倒推过程)
bootstrapping	自举
few-shot prompting	少样本提示
rejection (sampling) fine-tuning, RFT	拒绝采样微调
online / offline sampling	在线 / 离线采样
off-policy	异策略

English	中文
policy, critic (value model), reward model, reference model	策略、评论家（价值模型）、奖励模型、参考模型
advantage, baseline	优势、基线
group–relative baseline	组内相对基线
gradient coefficient	梯度系数
KL divergence penalty	KL 散度惩罚
clipping, Clip–Higher (asymmetric clipping)	裁剪、抬高上界的非对称裁剪
entropy collapse	熵塌缩
dynamic sampling	动态采样
effective batch size	有效 batch 大小
sample–level / token–level loss	样本级 / token 级损失
overlong filtering, soft overlong punishment	超长过滤、软性超长惩罚
truncation	截断
overthinking	过度思考
reward hacking	奖励投机
verifiable reward	可验证奖励
hill climbing	爬坡（沿信号持续改进）
pass@1, pass@K, maj@K	单次准确率、K 个样本里至少一个对、K 个样本多数票的准确率
win rate	胜率
distillation	蒸馏
backtracking, self–correction	回溯、自我纠错

字幕勘误

"CS320A" → CS329A; "in all series" → o–series; "Gemini flash thinking" → Gemini Flash Thinking; "batch script" → bash script; "path at 1" → pass@1, "past K / Pass K" → pass@K, "majority at K / majority had K" → maj@K; "archive data / archive papers" → arXiv; 讲到改进 PaLM 在 STEM 上表现的那篇论文时，字幕漏掉了论文名，应为 Minerva; "DeepSeek carbon paper" → 应为 DeepSeek–R1 论文; "DeepSeeL–R1" → DeepSeek–R1; "the quantity to be model" → Qwen–32B model; "you need double kind of techniques" → DAPO 这类技巧; "few short examples" → few–

shot examples; "common call like mixed token prediction" → Common Crawl、next-token prediction; "lower updates" → 应为 LoRA updates; 学生提问里的 "key-node score / clean score" 推断为 continuous score (连续分数); "[? AlphaCode ?]" 是字幕自己标的不确定, 按上下文(无限搜索加验证)应为 AlphaCode。另有几处是讲者口误而非字幕错误(STaR 结果里的 51.7%、GRPO 与 GAE、R1-Zero 与 distilled), 见正文小注。

带走的问题

1. 训练时算力和测试时算力该怎么分配? 有没有一条"再往训练里投就不划算"的 ROI 曲线? 学生开场就问, 讲者说留到后面, 但全讲没有给出定量的回答。
2. RL 为什么只抬 maj@K、不抬 pass@K? 要抬高能力上限, 缺的是新知识(回到预训练)、更好的探索(更长的上下文、搜索式采样), 还是更难、分布外的训练题?
3. 回溯、自检这些行为是 RL 涌现出来的, 还是基座里本来就有、只是被放大了? 什么样的对照实验能把两者分开?
4. 怎样从失败里学? STaR 把答错的直接丢掉, GRPO 给负优势, DAPO 却把全错的组整组滤掉——而全错的难题恰恰是模型最该学的。给答案倒推的 rationalization 能不能给这些题造出第一条成功轨迹, 又不把"错过程 + 对答案"带进训练?

CS329A 第 7 讲 | 自我改进与 Deep Research 智能体 (Self-Improvement and Deep Research Agents)

Stanford CS329A: Self-Improving AI Agents (2025 秋) · YouTube 编号是 Part 7, 内容对应课表第 8 次课 (10 月 17 日, Self improvement with Search & Deep Research Agents)。课表第 7 次课 (10 月 13 日, Open-Ended Evolution: ADAS / Darwin Gödel Machine、The AI Scientist、AlphaEvolve) 不在本视频里, 讲者只在讨论环节用一句"上一讲讲过的 scientist 式工作"带过 视频: <https://www.youtube.com/watch?v=Uni9dqyuuDM> (1:12:27, 自带英文 CC) 讲者: Aakanksha Chowdhery (推断: 视频里没有自我介绍; 她以第三人称提到 Azalia 讲过 Large Language Monkeys) 配套阅读: Competition-Level Code Generation with AlphaCode (Li et al., 2022) · AlphaCode 2 Technical Report (Google DeepMind, 2023) · Search-o1: Agentic Search-Enhanced Large Reasoning Models (Li et al., 2025)

一句话: 两种"搜索", 同一个问题——答案多半已经在模型够得着的地方, 难的是把它找出来。AlphaCode / AlphaCode 2 在模型自己的输出分布里搜: 海量采样, 再用过滤、聚类、打分模型把一百万份程序压成 10 份提交; Search-o1 向模型之外搜: 推理到知识缺口就发起检索, 并且先把文档提炼干净, 再放回推理链。

时间轴

时间	内容
0:06	开场: 两类搜索——代码里的"采样再筛选"与 deep research 式检索; 和作业 2、3 的关系
1:12	AlphaCode: 竞赛编程的问题设定, 与行级补全、HumanEval 的差别
3:45	系统总览: 预训练 → 微调 → 大规模采样 → 过滤聚类 → 提交
5:17	训练细节: 约 700GB GitHub 代码、GOLD、value conditioning
6:53	每题 100 万份样本; 过滤与聚类
8:26	为什么只能提交一小部分; Codeforces 与 CodeContests 两套评测; 前 54.3%
10:35	问答: 各场比赛成绩为何起伏; 为什么要采这么多
13:41	pass@k 与 10@k; 9B / 41B / 41B + clustering 结果表
17:27	扩展曲线: 对数线性、斜率、选择瓶颈
19:02	问答: 采样到一万亿会怎样; 多样性前提; 对数线性的来历

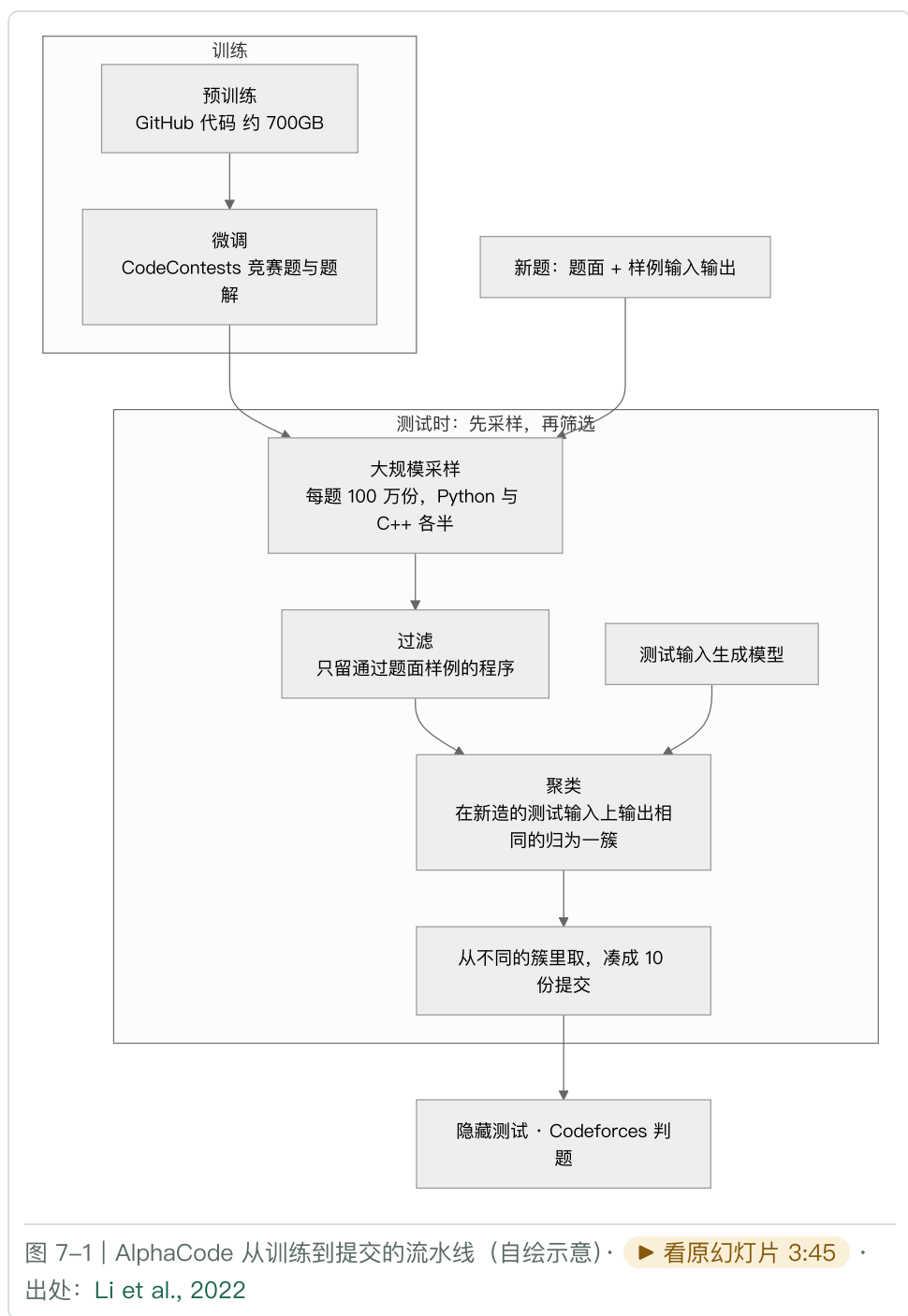
时间	内容
22:40	AlphaCode 小结与局限
24:41	AlphaCode 2: Gemini Pro、模型家族、打分模型、新数据
28:55	采样与评测流水线：只用 C++、淘汰 95%、10 个最大簇、打分重排
30:05	结果：100 份样本追平 100 万份；43% 对 25%
31:32	问答：95% 的浪费怎么省；与推理模型的关系；打分模型的数据
37:46	第 85 百分位；AlphaCode 2 的经验与代价
39:24	课堂讨论：按难度调整方法；把推理嵌入模型；树搜索
46:45	Search-o1：知识截止、不确定词、为什么一次性 RAG 不够
49:50	两点改动；化学题的三种做法对比
55:03	Agentic RAG 与 Reason-in-Documents；问答
1:00:05	结果：文档数扩展、对比人类专家、多跳问答
1:06:05	要点回顾；Search-R1（提及未讲）
1:08:59	最后的问答：置信度与校准

核心内容

1. 主线：解在搜索空间里，难的是把它挑出来

- 本讲把"用搜索改进模型"分成两类。一类在模型自己的输出里搜：对一道编程题采样极多份程序，再从中筛选（AlphaCode、AlphaCode 2，套路可用于作业 2 的 HumanEval）。另一类向外部世界搜：推理途中检索资料，补上知识缺口（Search-o1，作业 3 要搭的就是这类 agent）。
- 共同的前提来自第 2 讲：coverage 随采样数上升，说明正确答案大概率就在模型的输出空间里，剩下的是选择问题。

2. AlphaCode (DeepMind, 2022)：海量采样，先筛后交



- **任务：**竞赛编程。输入是一大段自然语言题面加几组样例输入输出，要先读懂题、想出算法，再写出完整程序，比行级补全（Copilot 式）和题意已写明的 HumanEval 都难得多。
- **模型：**当时没有现成的强 LLM，团队自己训练 encoder-decoder：先在约 700GB 的 GitHub 代码上预训练，再在 CodeContests（竞赛题与题解，另留出一份测试集）上微调。微调用三个技巧：一种正则化、value conditioning & prediction，以及 GOLD——按 token 似然加权的损失，让模型把概率压在自己有把握的模式上，追求 precision。

小注：论文里预训练两种损失并用——decoder 做 next-token prediction, encoder 做 masked language modeling; GitHub 语料 715.1GB。课上没点名的正则化技巧叫 tempering; value conditioning 是训练时把"这份解对 / 错"写进 prompt, 采样时一律声明"对"。

- **采样**：每题 100 万份，Python 和 C++ 各一半；prompt 里的题目 tag 和难度 rating 随机化，再配较高温度，一切为了多样性。这是第 2 讲重复采样的极端版本。
- **过滤**：只保留能通过题面样例测试的程序。
- **聚类**：另训一个"测试输入生成模型"给新题造一批输入；在这些输入上输出完全一致的程序视为语义等价（写法不同、行为相同），归为一簇。提交时从不同的簇里取，避免把 10 次机会浪费在同一个解法上。

小注：论文：样例过滤约淘汰 99% 的样本；提交顺序是从最大的簇到最小的簇各取一份——错法千奇百怪，对的程序行为一致，正确解更容易聚成大簇。这相当于在"程序行为"上做多数投票（对照第 2、3 讲的 majority voting）。

- **为什么必须先选**：此前课上的闭环都是"模型出多少、就测多少"。这里不行——Codeforces 不可能接收 100 万份提交，每题最多交 10 份，选择本身成了系统的一部分。
- **评测**：一是 Codeforces 实战（模拟参赛 10 场，每场 5,000 人以上），每题最多 10 次提交时平均排名前 54.3%，大致是参赛者的中位水平，按讲者的说法约相当于近半年活跃选手里的前 28%；二是 CodeContests 留出集，用来做可重复的研究迭代。讲者的评价：这是第一次展示 AI 能走出窄任务、端到端地解题。

小注：论文原话是：每题 10 次提交的成绩对应估计 Codeforces rating 1238，位于"近 6 个月参加过比赛的用户"的前 28%。

3. pass@k、10@k 与扩展规律

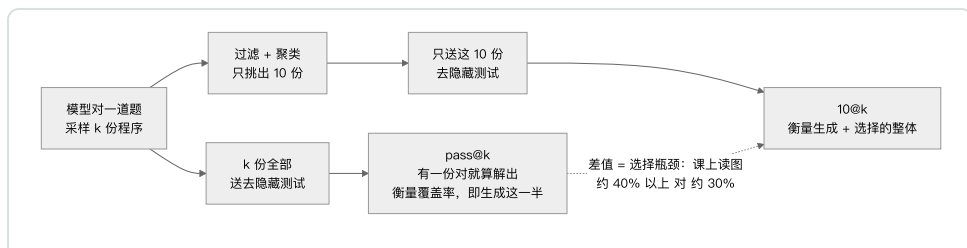


图 7-2 | pass@k 与 10@k 各自衡量什么（自绘示意）· [▶ 看原幻灯片 14:05](#) · 出处：Li et al., 2022

- **pass@k**: k 份样本全部送隐藏测试，有一份对就算解出——就是第 2 讲的 coverage，只衡量"生成 / 搜索"这一半。
- **10@k**: 采样 k 份、只许提交 10 份，把选择环节也算进去。
- **结果表**: 9B → 41B → 41B + clustering, 逐行变好; 10@1k → 10@1M, 逐列变好。模型更大、样本更多、加聚类, 三个方向都单调。

小注: 论文 Table 5: 41B + clustering 在验证集上从 10@1k 的 21.0% 升到 10@1M 的 34.2%; 测试集只跑到 10@100k, 为 29.6%。

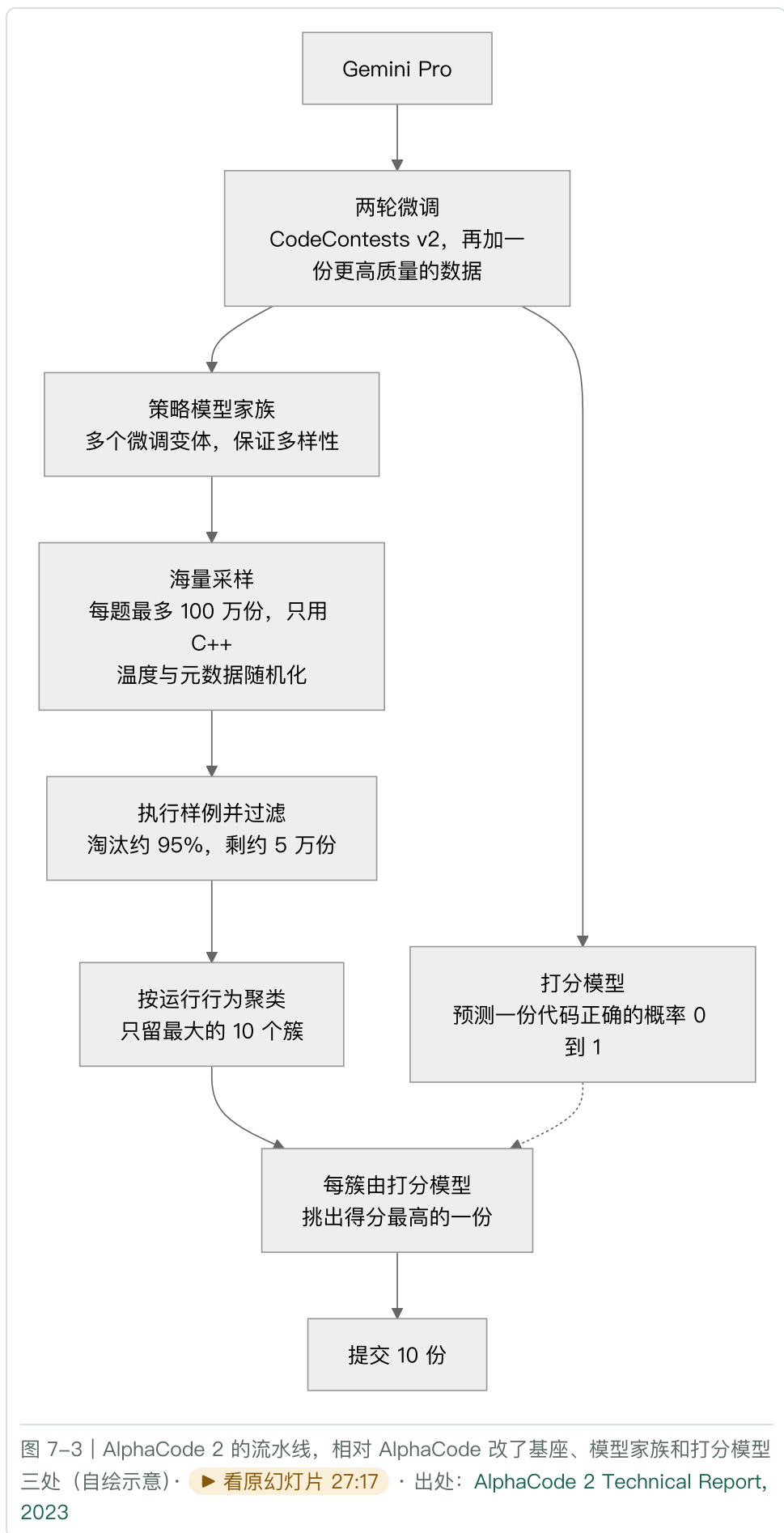
- **扩展曲线**: 解题率对采样预算呈对数线性, 而且经过选择之后仍然成立; 模型越大斜率越大 (41B 对 300M)。但 10@k 最高约 30%, pass@k 在"无限次提交"这个假设定下超过 40%。差出来的十个点就是选择环节的瓶颈, 也就是第 2、3 讲说的 generation-verification gap 在代码上的样子。
- **问答**
 - 各场比赛成绩为什么起伏很大: 一是题目离训练分布的远近不同; 二是选择环节可能成为瓶颈, 候选里会有"差一点就对"的解。
 - 为什么要采到 100 万这么极端: 模型小。第 2 讲的结论——模型越弱, 达到同样的 pass@k 需要的样本越多。
 - 趋势不变的话, 采样到一万亿会不会翻倍: 对数线性若能延续, 理论上会, 成本允许可以当项目做; 对一道特别重要的题, 把算力全押上去也说得通。但有两点保留: 提升模型是比堆样本更容易的一条轴 (见 AlphaCode 2), 基座不够强迟早撞上能力上限; 而且这条曲线默认"采得越多、解法越多样", 如果 10 倍样本换不来新解法, 就不会再涨。
 - 为什么是对数线性: 可以从理论上推出来, 属于 Azalia 讲过的 Large Language Monkeys 一线工作。

小注: 推断: 讲者说的"紧接着的那篇"应指 Schaeffer et al., 2025 《How Do Large Language Monkeys Get Their Power (Laws)?》 (Azalia 是作者之一): 单题的失败率随尝试次数指数下降, 少数极难的题把总体曲线拉成幂律。

- **小结与局限**: 大规模采样 + 过滤 + 聚类能换来高 coverage; 团队在训练集里检索过, 生成的代码不是照抄, 有分布外泛化。局限有四: 训练 loss 不是解题率的好代理 (一题多解); 动态规划、构造类题目

明显偏弱；百万级采样不实用，有时间限制时更应该边生成边排序；
难题靠 one-shot 不行，需要多步解法。

4. AlphaCode 2 (Google DeepMind, 2023): 更强的基座、模型家族、学出来的选择器



- **三处改动**

1. 不再自己预训练，直接微调 Gemini Pro（是微调，不只是 prompt）。
2. 微调出一个**模型家族**（不同超参，讲者还提到按难度、tag 切分数据），采样预算分摊给各个模型。多样性从"一个模型调高温度"升级为"多个模型各出一份"。
3. 加一个**打分模型**：同样由 Gemini Pro 微调，给每份代码估一个 0 到 1 的正确概率。过滤和聚类是启发式，打分模型是学出来的选择函数——本质上是 reward model，对应第 3 讲的 ORM。

- **数据**：CodeContests v2（题更多、质量更高），外加一份人工精选的高质量数据集，用于第二轮微调，也用于训练打分模型。
- **流水线**：每题最多 100 万份，只用 C++，温度和 prompt 元数据随机化 → 在样例上执行，去掉答案错或编译不过的，约 95% 被淘汰、剩约 5 万份 → 按运行行为聚类，只留最大的 10 个簇 → 打分模型在每簇里挑一份 → 提交 10 份。
- **结果**：约 100 份样本就追平 AlphaCode 的 100 万份；同样采 100 万份，解题率 43% 对 25%，接近两倍，而且曲线还在涨。换算成排名约在第 85 百分位，介于 Codeforces 的 Expert 和 Candidate Master 之间；AlphaCode 大约只超过 46% 的选手。

小注：技术报告的口径：评测用 12 场较新的 Codeforces 比赛（每场 8,000 人以上，共 77 题）；课上提到的 99.5%，指它表现最好的两场比赛里超过了 99.5% 的参赛者，不是"取前两个解"；更新版 CodeContests 约 1.5 万题、3,000 万份人类代码，报告没写它是否公开（公开的是原版）；模型家族在报告里靠"改超参"得到，没有提切分数据。

- **讲者的总结**：一年内的进步不靠堆样本，靠的是更强的基座、更有保障的多样性、学出来的选择器。代价仍然高，方法只适用于代码，大部分算力花在了最终被过滤掉的样本上。
- **问答**
 - 95% 的采样被浪费，能不能改进：学生提议用作业 1 的 self-refinement。讲者的归纳：AlphaCode 本质是**并行搜索**；加入带反馈的迭代改进可以砍掉样本数，代价是时间，而且要先回答"反馈从哪来"。另一条路是 RL——模型本身变强，测试时要花的算力就少（留到 train-time scaling 再讲）。
 - 这和今天只采几十个样本的 thinking model 是什么关系：把 AlphaCode 2 看成"从零搭一个能推理、能闭环的系统"的蓝图——

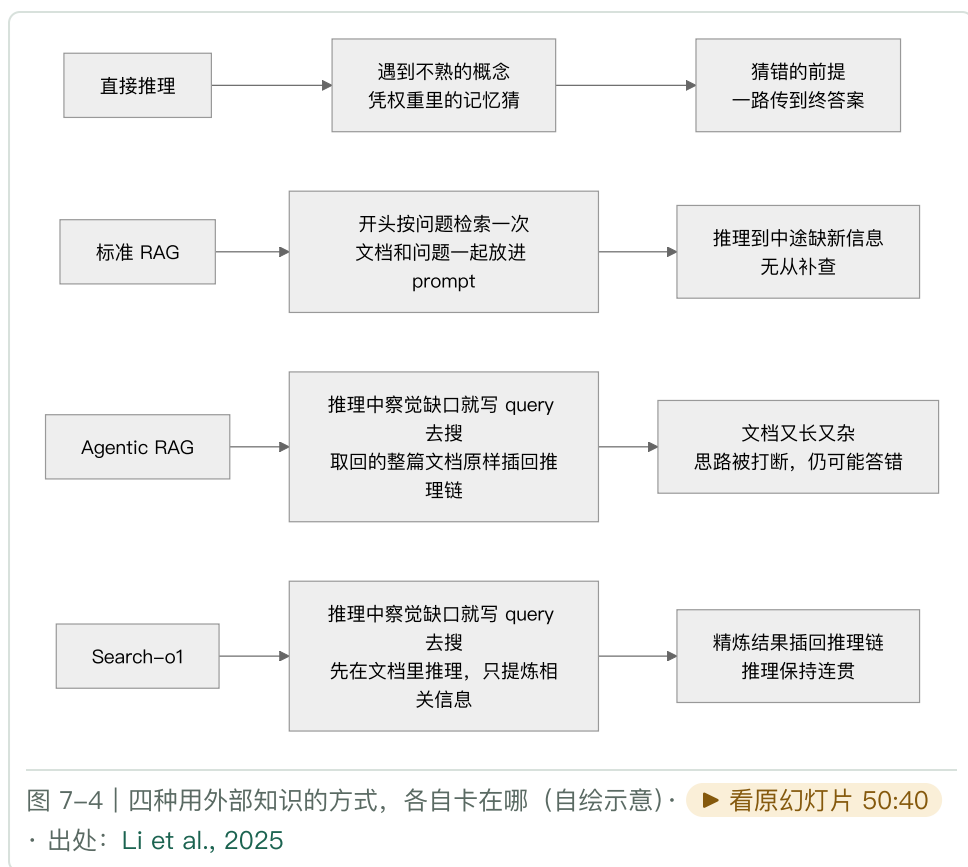
一族模型生成，另一个模型打分，近似多智能体系统；把整套系统的能力蒸馏进单个模型，得到的就是 large reasoning model。

- 打分模型为什么另用一份数据：避免污染。它不该见过同一批题，但需要同分布的数据来学“两份解里该偏好哪一份”。分阶段微调时若不混入前一阶段的数据，模型会遗忘。

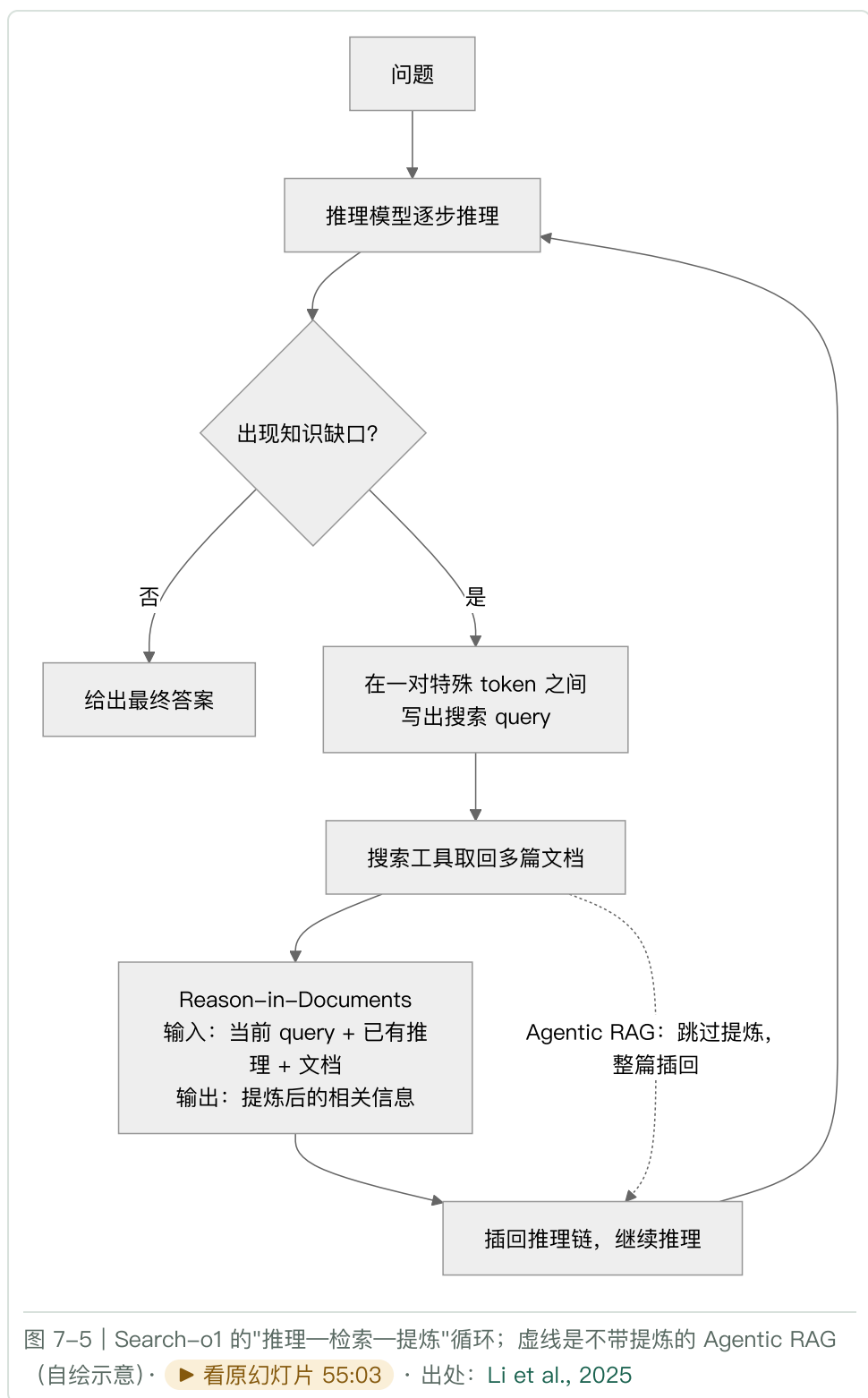
5. 课堂讨论：按难度调整方法，把推理嵌进模型

- **按任务难度**：简单题用很少的样本就能达到 coverage，或者出几份初稿再迭代修正；难题才需要大规模并行采样——并行采样买到的是不同的解题思路，不是对已有答案的修补。也有学生提议先判难度，再交给家族里对应难度的模型去采样。
- **把推理嵌进模型**：在训练数据里加“用什么算法、分几段”这类提示；更一般地，把写代码之前的思考过程（chain of thought）放进训练集，或者用 STaR 的办法——把答案当提示，让模型补出推理过程，再拿去训练。难题可以分解成子问题、逐段给提示。
- 学生追问：第一步对了、第二步错了怎么办？讲者的回答指向多步闭环加树搜索——逐步采样、逐步检查、可以回溯，并说这可以是一个项目方向。如果解法模式本身就在分布之外，可能需要 human in the loop，像上一讲的 scientist 式工作那样。

6. Search-o1 (2025)：让推理模型边想边查



- 课上的 deep research agent 是泛称，实际拆解的只有 Search-o1 这一篇，没有点名任何具体产品。
- **动机**：large reasoning model 推理强，但知识有截止日期。知识缺口在长推理链里表现为大量表示不确定的词（GPQA 上的 perhaps、alternatively、wait）；模型一旦靠猜往下走，错误会一路传到终答案。
- **标准 RAG 为什么不够**：只在开头检索一次。查天气这类问题够用，但复杂问题的每一步需要的信息不同，推理到一半才知道缺什么。所以 RAG 相对直接推理有提升，在多步推理上仍然吃亏。
- **Agentic RAG**：模型在推理中察觉不确定时，自己在一对特殊 token 之间写出搜索 query，工具取回文档后插回推理链，一次推理可以检索很多轮。问题是文档又长又有噪声，塞进 10 到 20 篇之后，模型的长上下文推理跟不上，原来的思路反而被打断。
- **Search-o1 = Agentic RAG + Reason-in-Documents**：检索之后多一步"在文档里推理"，见下图。



- **Reason-in-Documents**：输入是当前 query、此前的推理和取回的文档，输出只有提炼过的相关信息，再注入推理链。从外面看，它只是把"检索工具"这个函数做成了两步：取文档，再提炼。讲者的类比：人做研究不会把参考文献原样堆在一起，而是边读边做笔记。
- **例子**（一道化学题，问多步反应后产物 3 的碳原子数）：直接推理靠猜结构，答 10，错；Agentic RAG 取回的文档太长，答 14，仍然错；

Search-o1 先搜到相关反应，提炼出结构，再干净地接回推理，得到正确答案。有了检索到的信息之后，推理里的不确定词也少了。

小注：论文：基座是 QwQ-32B-Preview，检索用 Bing Web Search API、默认取前 10 篇；query 和结果分别包在

`<|begin_search_query|>...<|end_search_query|>`、
`<|begin_search_result|>...<|end_search_result|>` 里；GPQA diamond 上每条推理链里 perhaps 平均出现 30 次以上。例题按反应式推算，正确答案是 11 个碳（肉桂醛 9 个，格氏试剂 +1，氧化不变，硫叶立德 +1）。

• 问答

- 模型怎么知道该搜什么：找出问题里的关键实体。做真实应用时，应当事先划定哪些信息不该信任权重、必须走工具调用。
- 之前的推理存在哪、需要修正怎么办：框架里隐含一个状态 / 记忆缓冲，保存当前 query 和已有推理。
- 直接在 Agentic RAG 里加一句"先总结再继续"行不行：鼓励在作业 3 里试。这样做默认模型的长上下文推理足够好；瓶颈其实在"上下文里放什么，模型才处理得好"，属于最近 Agentic Context Engineering 一类的问题。
- 检索本身的 precision / recall 呢：论文的主张是，即便取回的都是大致相关的文档，全部塞进上下文对模型的要求也太高——差别来自提炼这一步。

7. Search-o1 的结果

- **文档数扩展**（GPQA，分物理 / 化学 / 生物 / 总体，纵轴 pass@1）：检索文档越多，直接推理和标准 RAG 基本不涨，Search-o1 持续上升——增加的并不是上下文长度，而是提炼后的有效信息。讲者把它看作又一条扩展轴：除了迭代式改进，还可以"并行地多取、再压缩"。
- **对比人类专家**（GPQA）：要看对角线，即物理学家做物理题、化学家做化学题。Search-o1 在物理和生物上不输对应专家，化学还差得远。讲者措辞克制：这只说明在这类题上"和专家有竞争力"，不等于超过专家。学生猜测化学差是因为化学式一字之差含义就变，提炼时容易失真；讲者认为有可能，也可能是该领域训练数据偏弱，留给大家自己验证。

小注：论文 Table 2（GPQA extended set）对角线上的人类专家成绩：物理 57.9 / 化学 72.6 / 生物 68.9；Search-o1 对应 68.7 / 40.7 / 69.5。讲者一开始说物理、化学有竞争力，看完对角线后改口为物理、生

物好而化学差，后一种说法与论文一致。同一篇论文还报告：Search-o1 只取 1 篇文档，也超过了取 10 篇的标准 RAG。

- **多跳问答** (HotpotQA、2WikiMultihopQA、MuSiQue、Bamboogle)：标准 RAG 和 Agentic RAG 都会饱和，表里加粗的最好成绩大多来自 Search-o1。
- 推理链里的不确定词数量显著下降；模型越大效果越好。
- **Search-R1** (提及未讲)：Search-o1 靠 prompt 把"搜索—提炼—继续推理"的闭环搭起来；Search-R1 用强化学习教会模型自己这样做。放回第 1 讲的三条扩展轴，就是把同一种能力从 test-time 挪到 post-training。

8. 最后的问答：模型的置信度可信吗

- 学生问：采样 1,000 个答案，各自的生成概率和对错相关吗？讲者：把输出 token 的 log-prob 合理聚合（取平均或几何平均，而不是简单求和）之后，会发现模型普遍**过度自信**——实际只对一半的时候，它可能有八成把握；表现出来就是被纠正时不肯改口。
- 对的答案模型通常很有把握，但错的答案它同样可能很有把握。已有工作让模型在第二遍里估计自己的置信度，再用 RL / RLHF 做校准，让它少输出没把握的答案；各家实验室怎么做并不透明。
- "模型是否知道自己知道什么"是活跃的研究方向，适合做课程项目；RAG 会不会改变答案的概率，讲者留作实验。

关键图表速查（点时间戳跳到原幻灯片）

图	看什么	跳转	出处
AlphaCode 系统总览	四个阶段：训练、海量采样、过滤聚类、挑 10 份提交；注意"先选择、再评测"这一步	3:45	Li et al.
Codeforces 十场比赛排名表	每场的估计 / 最好 / 最差百分位；场次之间起伏很大，平均前 54.3%	9:58	同上
模型规模 × 采样数结果表	三行 9B / 41B / 41B + clustering，列从 10@1k 到 10@1M；每个方向都单调变好	15:25	同上
10@k 与 pass@k 扩展曲线	横轴采样预算（对数），纵轴解题率；41B 斜率最大、300M 最小；10@k 约到 30%，pass@k 超过 40%	17:50	同上
AlphaCode 2 系统框图	对着 AlphaCode 的框图找三处改动：Gemini Pro 微调出的模型家族、两份数据、打分模型	27:17	AlphaCode 2 报告

图	看什么	跳转	出处
AlphaCode 2 对 AlphaCode 的采样效率	横轴每题采样预算，纵轴解题率；约 100 万份样本处追平 AlphaCode 的 100 万份，到 100 万份时 43% 对 25%	30:05	同上
与人类选手的排名对比	人类选手得分分布曲线上的两个点：AlphaCode 约 46%，AlphaCode 2 约 85%	37:46	同上
化学题三栏对比	左：直接推理靠猜；中：Agentic RAG 整篇文档塞入；右：Search-o1 先提炼再接回	50:40	Li et al.
Agentic RAG 与 Reason-in-Documents 示意	特殊 token 包住的 query、检索结果插回的位置、提炼模块吃哪三样输入	55:03	同上
文档数扩展曲线	四个小图（物理 / 化学 / 生物 / 总体），纵轴 pass@1；直接推理和 RAG 是平的，Search-o1 随文档数上升	1:00:05	同上
GPQA 对比人类专家	只看对角线；物理、生物不输对应专家，化学差距大	1:01:00	同上
多跳问答结果表	HotpotQA / 2WikiMultihopQA / MuSiQue / Bamboogle 四列，加粗的最好成绩大多落在 Search-o1 一行	1:04:30	同上

提到的工作

名称	在本讲里的作用
AlphaCode (Li et al., 2022)	百万级采样 + 过滤 + 聚类；提出 10@k；首次达到竞赛选手中位水平
AlphaCode 2 Technical Report (Google DeepMind, 2023)	换 Gemini Pro 基座、模型家族、学出来的打分模型；第 85 百分位
Search-o1 (Li et al., 2025)	推理模型 + Agentic RAG + Reason-in-Documents；作业 3 的原型
Search-R1 (Jin et al., 2025)	用 RL 教模型搜索；提及未讲
RAG / Agentic RAG	Search-o1 的两级基线
Agentic Context Engineering (Zhang et al., 2025)	回答"只加一句总结 prompt 行不行"时提到：瓶颈在上下文里放什么
Large Language Monkeys (Brown et al., 2024)	pass@k 随采样数的扩展规律（第 2 讲）
STaR (Zelikman et al., 2022)	把推理嵌进模型的一种办法：给答案、让模型补推理、再训练
GOLD (Pang & He, 2020)	AlphaCode 微调时的加权损失
Gemini Pro	AlphaCode 2 的策略模型和打分模型的基座
Copilot	行级补全的例子，与端到端解题对照

名称	在本讲里的作用
HumanEval / Codeforces / CodeContests (v1、v2)	作业 2 的评测 / 实战平台 / 训练与留出评测数据
GPQA / HotpotQA / 2WikiMultihopQA / MuSiQue / Bamboogle	Search-o1 的评测集
上一讲的 scientist 式工作（应指课表第 7 次课的 The AI Scientist 等）	解法模式在分布外时需要 human in the loop 的例子

术语对照

English	中文
competitive programming	竞赛编程
large-scale / massive sampling	大规模采样
sampling budget	采样预算
filtering	过滤
clustering	聚类
semantically equivalent	语义等价
test input generation model	测试输入生成模型
example tests / hidden tests	题面样例测试 / 隐藏测试
pass@k	k 份全交的解题率（覆盖率）
10@k	k 份里只交 10 份的解题率
solve rate	解题率
log-linear scaling	对数线性扩展
policy model family	策略模型家族
scoring model, reranking	打分模型，重排序
tempering	tempering（训练时调 logits 温度的正则化）
value conditioning	价值条件化
contamination	数据污染
parallel search vs. iterative refinement	并行搜索 vs. 迭代改进
tree search, backtracking	树搜索，回溯
task decomposition	任务分解
human in the loop	人在回路
large reasoning model (LRM)	大推理模型
knowledge cutoff / knowledge gap	知识截止 / 知识缺口
retrieval-augmented generation (RAG)	检索增强生成

English	中文
agentic RAG	智能体式 RAG
Reason-in-Documents	文档内推理（先提炼、再注入）
multi-hop QA	多跳问答
deep research agent	深度研究智能体
calibration, overconfidence	校准，过度自信
hallucination	幻觉

字幕勘误

"AlphaGo 2" (21:07、25:43 两处) → AlphaCode 2; "passer k" → pass@k; "Azalea ... large language monkey's paper" → Azalia、Large Language Monkeys; 10:35–13:41 问答里的 "context / context IDs" → contest / contest ID (不同场比赛); "50 case samples" → 50k samples; "code contest"、"code count as V2"、"code counts as V2" → CodeContests、CodeContests v2; "human eval" → HumanEval; "repetitive sampling" → repeated sampling; "STAR approach" → STaR; "clustering the prompt with 10 documents" → cluttering (把 prompt 塞满)，不是聚类; 学生提问里的 "summation" → summarization; "2Wiki, MusiQue" → 2WikiMultihopQA、MuSiQue; 学生提问里的 "authenticate a month's" 推断是 "20K-a-month" (指传闻中 OpenAI 的高价研究型 agent)。

带走的问题

- AlphaCode 的对数线性扩展暗含"采得越多、解法越多样"这个前提。怎样度量并主动提高样本的多样性，而不是靠温度和随机 tag 碰运气？多样性饱和之后，预算该转向顺序式 refinement、树搜索，还是换更强的基座？
- 过滤靠样例测试，聚类靠程序的运行行为，这两样都是代码独有的。换到没有可执行信号的任务，"过滤—聚类—打分"这条选择流水线拿什么替代？
- Search-o1 靠模型在"感到不确定"时发起搜索，但最后的问答指出模型普遍过度自信——最需要核查的，恰恰可能是它最笃定的错误前提。"何时搜索"能不能不依赖模型的自我感知？Search-R1 式的 RL 学得到这一点吗？

4. Reason-in-Documents 的提炼本身也会丢信息、引入错误（化学式这类一字之差就变义的内容尤其脆弱）。提炼出的错误怎么被发现和回滚？长上下文推理足够强之后，这个模块会不会变得多余？

CS329A 第 8 讲 | 智能体评测与长程任务

(Agentic Evaluations and Long Horizon Tasks)

Stanford CS329A: Self-Improving AI Agents (2025 秋) · 对应课表第 17 次课 (11 月 17 日): 课上逐篇讲的三篇论文与该次课的阅读材料完全一致 视频: <https://www.youtube.com/watch?v=8JAqLnTaZu4> (1:15:18, 自带英文 CC; 学生提问收音差, 字幕里 [INAUDIBLE] 很多) 讲者: 推断为 Aakanksha Chowdhery (本讲没有自我介绍; 她以亲历者的口吻回忆四五年前"模型能写出一段解释就是大事"的阶段, 又把"AI 能自己造出下一代模型"称作"我们对 AGI 的定义", 与 PaLM / Reflection AI 的背景吻合; 仅凭字幕不能完全确认) 配套阅读: [Measuring AI Ability to Complete Long Tasks](#) (Kwa et al., METR, 2025) · [GDPval: Evaluating AI Model Performance on Real-World Economically Valuable Tasks](#) (Patwardhan et al., OpenAI, 2025) · [DeepScholar-Bench](#) (Patel et al., Stanford / UC Berkeley, 2025)

一句话: 聊天式 benchmark 饱和之后, agent 评测先要回答"量什么"。这一讲用三篇论文给出三把互补的尺子——METR 用人类专业者的耗时给任务定标, 量模型在 50% / 80% 成功率下能做多长的任务 (约 7 个月翻一倍, 但 80% 下只剩 15 分钟量级); GDPval 让行业专家在模型和专家的交付物之间做两两比较, 量胜率 (两年近似线性, 最高 47.6%); DeepScholar-Bench 量"检索—综合—引用"的质量 (没有系统过 19%)。三者共同的盲区是上下文: 题目都把该知道的写全了、一次交付, 而真实工作难在自己弄清该做什么。

时间轴

时间	内容
0:05	开场调查: 大家的项目里用了什么 agentic eval
1:38	本讲三篇论文; 为什么要换尺子: 传统 benchmark 饱和, 能力与经济影响两条线
4:32	METR: 任务时长 × 可靠度, 以人类专业者的耗时为锚
5:42	三个任务集 SWAA / HCAST / RE-Bench; 四步方法
7:55	课堂互动: 给自己想外包的任务估时
9:57	问答: 人类基线要几个人; 示例任务; 几何平均与"熟手低估难度"
12:38	散点图: 成功率 vs 人类耗时
13:48	趋势图: 50% time horizon 约每 7 个月翻倍

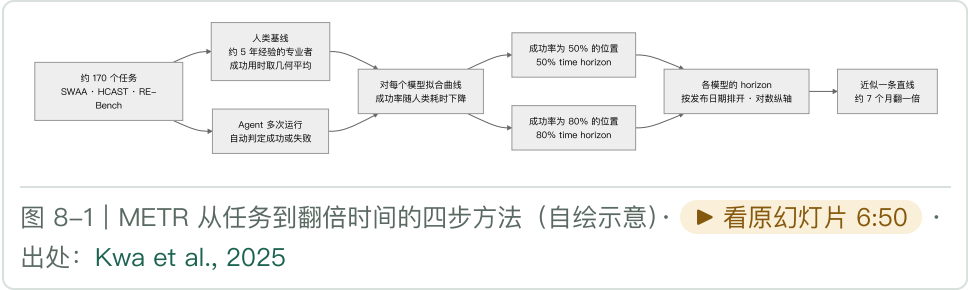
时间	内容
15:12	驱动因素；学生补充 context engineering、规划、用户反馈、记忆
20:18	80% 可靠度：59 分钟变成 15 分钟
22:37	失败模式：GPT-4 vs o1；失败分析怎么做
25:22	局限：messy 任务、SWE-bench Verified、内部 PR 与"低上下文的外包者"
27:51	GDPval：问题设定、覆盖的行业、示例任务
30:31	课堂讨论：主观性、伦理合规、作曲
33:40	数据集构造：9 个行业、44 个职业、1,320 个任务、gold 220
35:38	主结果：12.4% → 47.6%；"线性 vs 指数"之问
38:24	模型差异、失败原因、GPT-5 失败的严重程度
40:35	试 1 次 vs 试 n 次的速度 / 成本账
41:50	分行业 / 时长 / 模态；逐职业密集图
45:40	欠指定 prompt 实验；真实工作重上下文；近期含义
47:55	问答：美元标价可靠吗；模型赢了人意味着什么
51:34	DeepScholar-Bench：任务、数据、三个维度七个指标
55:18	结果：没有系统过 19%；失败模式；oracle 检索仍只有约 50%
58:53	问答：为什么讲这个 benchmark；"重要文献"怎么定
1:01:03	三篇合看；benchmark 与真实任务差在哪
1:04:38	有把握的 / 把握不大的；总结
1:08:20	最后问答：趋势能否持续、两年后的研究工作、50% 怎么算、CS 还值得学吗、长尾卡在哪

核心内容

1. 为什么要换尺子

- 几年前模型能写出一段像样的解释就值得报道；今天要回答的是它对经济和安全的影响有多大。聊天、问答、"答案就在上下文里"的单轮题饱和太快，既分不开前沿模型，也没法外推。
- 讲者把"量什么"拆成两条主线加一块短板：**能做多长、多复杂的任务**（METR）；**做出来的东西有没有经济价值**（GDPval）；以及两者都绕不开的**查资料、用资料**（DeepScholar-Bench）。前两个指标都站得住，但画出来的趋势一个是指数、一个近似线性——这是全讲的伏笔。

2. 论文一：METR 的 time horizon——用"人要做多久"给任务定标



三个任务集合计约 170 个任务：

任务集	数量	人类耗时	内容
SWAA	66	1—30 秒	软件工程里的原子动作
HCAST	97	1 分钟—30 小时	多样的软件与研究工程任务
RE-Bench	7	约 8 小时	完整的 ML 研究工程任务，与课程前面讲过的 AI Scientist 同类

- **指标有两个维度：**任务时长 × 可靠度。时长不看模型自己跑了多久，而以"熟练的人类专业者做完要多久"为统一的锚；可靠度取 50% 和 80% 两档。课堂上学生给想外包的任务估时：核对 BibTeX 几分钟，写文献综述几小时，重构代码库几小时到几天、还得有人验收。
- **示例任务的跨度：**认出哪个文件是 shell 脚本（约 3 秒）→ 修仿真输入文件里的 bug（约 10 分钟）→ 给回测工具写定制 CUDA kernel 并达到指定加速（约 8 小时）；中间还有查 Wikipedia、转换 JSON 数据格式。
- **人类基线：**请约 5 年经验的专业者真的去做，只记成功完成的用时，取几何平均当作任务难度。人很贵，每题只有少数几位；讲者给的通行做法是看 inter-rater agreement，分歧大就补样本。讲者还点出一个系统性偏差：熟手对"做成是什么样"有预设，容易低估难度；人觉得难和模型觉得难也未必是一回事。
- **拟合：**每个模型在每个任务上跑多次得到成功率，对人类耗时拟合成一条曲线，曲线上成功率为 50%（或 80%）处的耗时就是该模型的 time horizon；再把各模型按发布日期排开看趋势。

小注：论文的做法是对每个模型做 logistic 回归 $p(\text{成功}) = \sigma(\beta \cdot (\log h - \log t))$ ：t 是该任务人类成功基线用时的几何平均，学出来的 h 即 50% time horizon，80% horizon 取同一条曲线上 0.8 的位置。每个"模型—任务"对跑 8 次；任务按所在任务族大小的平方根倒数加权；连续得分用任务级阈值二值化；169 个任务中 148 个有真人基线，其余用研究者估时。趋

势线是 $\log(\text{horizon})$ 对发布日期的线性回归：翻倍时间 212 天（95% 区间 171–249 天），80% horizon 为 213 天。

• 结果

- 散点图：SWAA 成功率普遍高；HCAST 从 0.8 到 0 铺满；RE-Bench 有成功但偏低。任务越长，模型越难不跑偏。
- 趋势：GPT-2（2019）约 2 秒 → GPT-4（2023）约 8 分钟 → Claude 3.7 Sonnet（2025）59 分钟，约每 7 个月翻一倍。
- 但这只是 50%：好比把活交给实习生，回来的结果一半能用。换成 80%，翻倍速度几乎不变，整条线却低一大截——Claude 3.7 Sonnet 只有约 15 分钟。讲者的解读：真要部署，中等复杂度的任务就会撞上可靠性问题，模型之上的系统层还有很大空间。这和第 2 讲的 coverage 是同一个故事：50% horizon 说的是“做得出来”，到 80%、95% 才算“交得出去”。

- **是什么把 horizon 拉长**：推理、代码生成、工具使用（ReAct 一脉）变强；更少原地打转；出错后能恢复，并始终记得最终目标和当前进度。学生补充了工程侧因素：context engineering（Claude Code 的 compaction）、来自用户日志的反馈（Cursor 的 Tab 补全用 online RL）、先显式规划再按需重新规划、对代码库和环境的记忆。
- **失败模式**（人工归类 GPT-4 和 o1 的失败轨迹）：规划或工具选得不好；心算 / 推理出错；过早放弃——判断不了“做成”是什么样；重复已经失败的动作——失败后它仍是概率最高的动作。最后一类在 GPT-4 上很多，到 o1 明显减少。讲者的方法论建议：提出一个难 benchmark，失败模式分析就该是标配，靠人读轨迹，没有捷径。

小注：论文的归类表（GPT-4 1106 共 31 条失败轨迹 / o1 共 32 条）：规划或工具选择 4 / 6，心算或推理 6 / 7，过早放弃 8 / 16，重复失败动作 12 / 2，其他 1 / 1——“重复失败动作”大减的同时，o1 的“过早放弃”翻了一倍。

• 局限

- messy 的任务（没有唯一答案、环境更复杂）成绩更低，但随时间上升的趋势相似。
- 在 SWE-bench Verified 上重做分析，趋势相似、翻倍时间更短。讲者给的理由：标注者低估了任务耗时，且模型多半见过这些 GitHub 仓库。

小注：论文在 SWE-bench Verified 上得到的翻倍时间约 70 天，解释是那里用的标注者估时对最简单的任务低估得最厉害，弱模型的 horizon 被压低、趋势线被拉陡；"模型见过这些仓库"是讲者补充的。

- 内部代码库的真实 PR：没有上下文的外包者比仓库维护者慢 5—18 倍，而模型的表现更接近外包者。今天的 agent 应被看成"聪明但低上下文的人"——这一点贯穿后两篇。

3. 论文二：GDPval——和行业专家的交付物对打

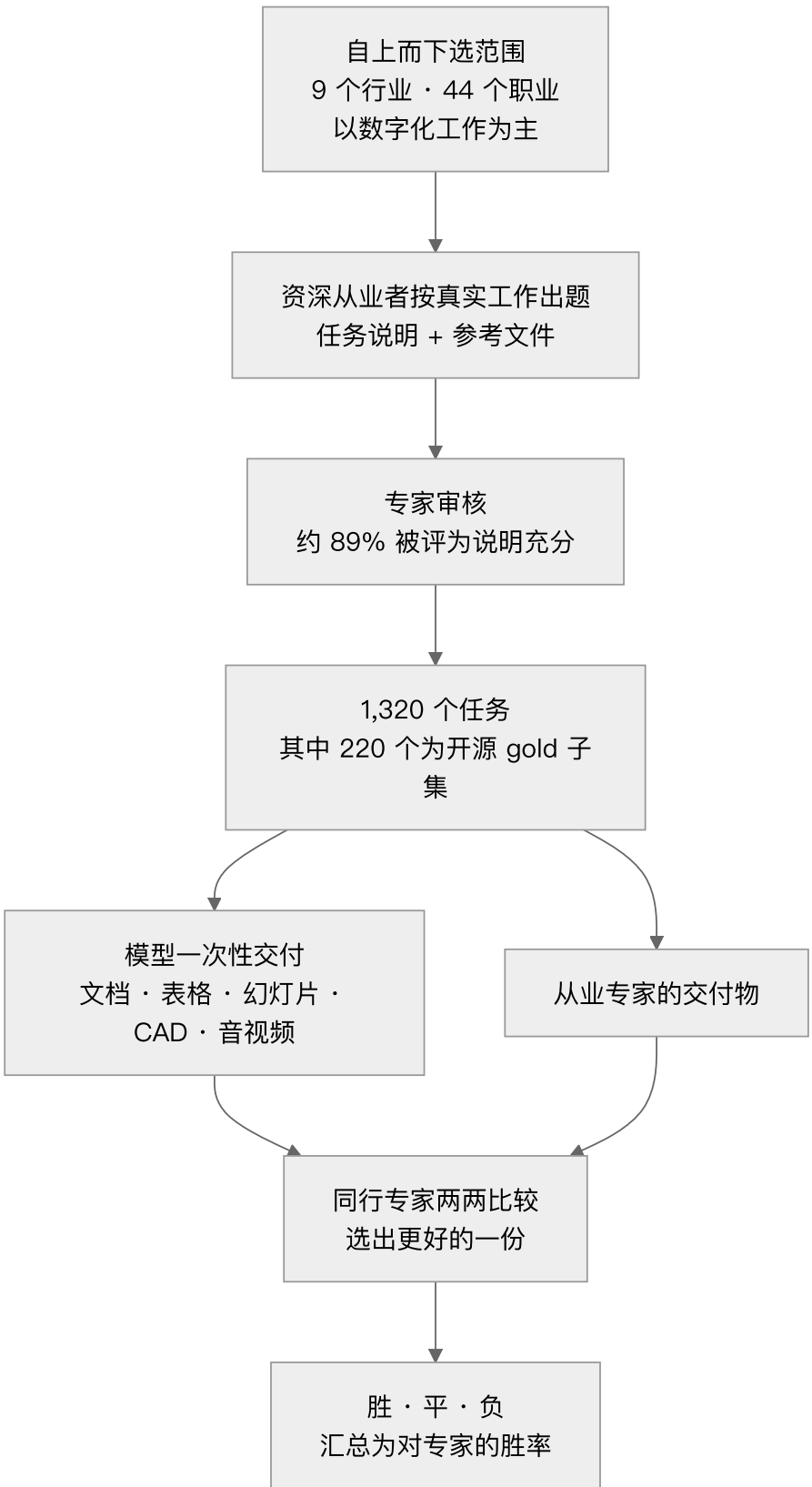


图 8-2 | GDPval 的任务来源与评分方式（自绘示意）· [▶ 看原幻灯片 33:40](#) · 出处: Patwardhan et al., 2025

- **问题换了：**不问"AI 能不能做"，而问"交给模型而不是人，交回来的东西够不够好"。对手是从业十年以上的行业专家。

- **任务怎么来**

- 自上而下：对 GDP 贡献最大的 9 个行业 → 44 个职业 → 1,320 个任务，其中 220 个作为 gold 子集开源在 Hugging Face。只收以数字化工作为主的职业，用 O*NET（职业—任务分类体系）判定。

小注：论文的口径是先取对美国 GDP 贡献超过 5% 的 9 个行业，每个行业内按总工资取排在最前面的几个（至多 5 个）"数字化为主"的职业，判据是该职业在 O*NET 里 60% 以上的任务属于数字化任务。课上的"GDP 前 5%"和"60% 的 O*NET 任务进了数据集"是口头简化。出题专家平均从业 14 年。

- 任务按从业者的真实工作来出：平均要专家做约 7 小时，长的要几周；除文本外还有 CAD、视频、音频、表格、幻灯片；gold 子集里平均每个任务约值 400 美元（讲者拿 SWE-Lancer 里更高价的任务作对比）；约 70% 要读随附的参考文件；约 89% 被专家评为说明充分——模型输了不能怪题目没写清楚。
- 示例：设计流水线用的线缆盘支架 3D 模型（制造工程师）、看病灶图片写会诊报告（注册护士）、按脚本剪片头（视频剪辑）、核对多张采购单的价格出入（审计）、优化市集的摊位布局（文体活动工作者）。

- **怎么判：**学生的第一反应是太主观。讲者：正因为主观，指标才是胜率——专家在模型和专家的交付物之间做两两偏好选择；主观性本身就是真实工作的一部分。课堂上还分开了"能不能做"和"出于伦理、合规要不要让它做"，benchmark 只回答前者；作曲这类好坏更难说清的工作没有收进来。

小注：论文里是盲评，47.6% 是"胜 + 平"的口径。专家之间的一致率 71%，OpenAI 提供的自动评审与专家的一致率 66%。放回第 3 讲的语境：自动评审是一个不完美的验证器，人与人七成的一致率是它的天花板。

- **主结果：**GPT-4o 12.4%（2024）→ 中间几代 25%、30% 多 → Claude Opus 4.1 47.6%，大致线性。学生问这和 METR 的指数是否矛盾。讲者：量的不是一个东西，谈不上谁错；但指数曲线容易让人顺着推"现在一小时，接着几小时、几天"，而 GDPval 把几小时到几周的真实工作按职业拆开，看到的是参差不齐、可靠度有限。

- **模型差异与失败**

- Claude 强在美观度、文档排版和 PDF / 表格 / 幻灯片；GPT-5 强在指令遵循、计算正确性和纯文本任务。
- 输的原因以指令遵循类最多，典型症状是嘴上说会看参考数据、实际没看，用幻觉内容盖过去；其次是格式错误。
- GPT-5 失败案例的严重程度：约一半"可接受但不如人"，约 29% "差或灾难性"，另有约两成复评后认为模型其实更好；评分者之间的分歧会影响这组数字。

小注：按论文，这组比例的分母是 GPT-5 输给专家的交付物，不是全部任务，其中灾难性的约 3%。论文原话是 Claude、Grok、Gemini 主要输在指令遵循，GPT-5 high 主要输在格式；"答应了却没交、无视参考数据"点名的是 Gemini 和 Grok。

- **分组看**：政府、零售、批发接近与专家持平；几小时以内的任务胜率高，越长越低。逐职业图里贴近"持平"红线的有柜台与租赁职员、收发货与库存职员、采购、软件开发者、合规专员、个人理财顾问、客服代表、编辑、新闻分析等。讲者提醒，这里比的是平均水平的从业者，不是顶尖专家。

4. GDPval 的两条引申：人在环里的经济账，和"上下文"这条限制

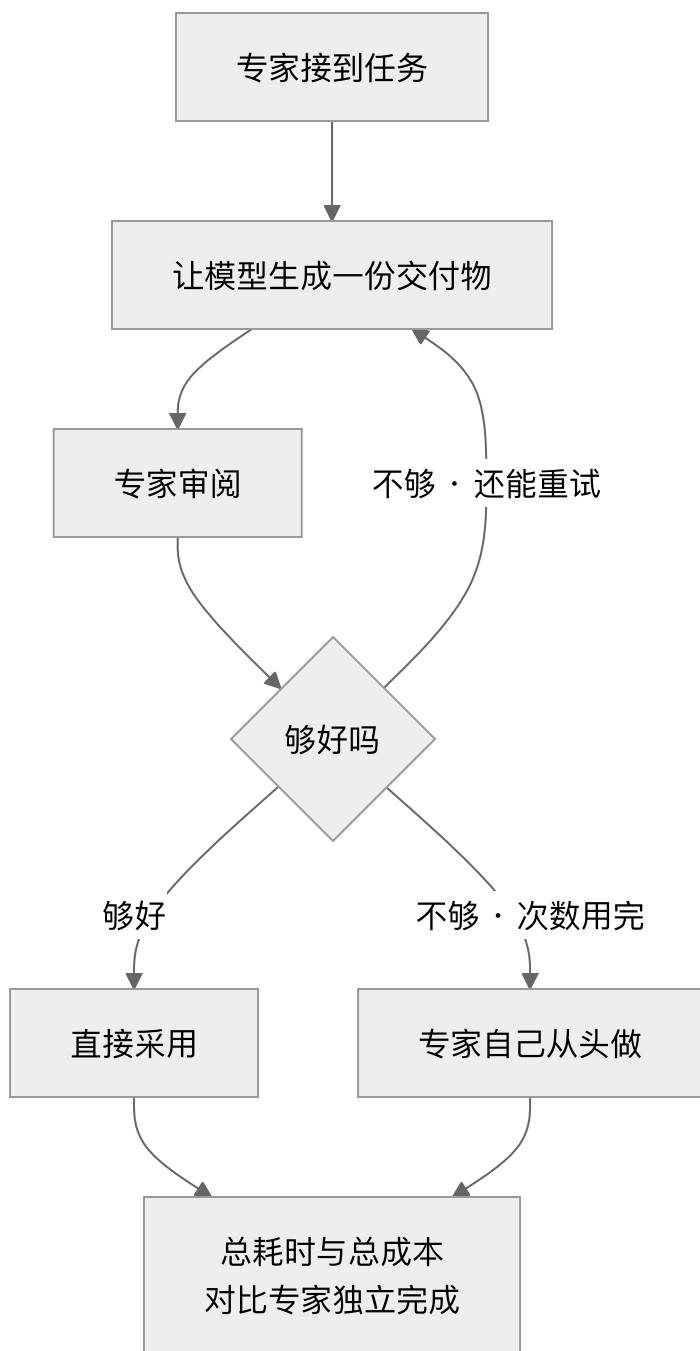


图 8-3 | "试 n 次、不行再自己做"的人机协作模型（自绘示意，按论文定义）·

► 看原幻灯片 40:35 · 出处: Patwardhan et al., 2025

- **经济账：**讲者点题，这是 self-improving agents 课——并行多采几次，或串行地让模型看着上一版自己改，结果通常明显变好（对应第 2 讲的并行 / 串行 test-time compute）。GDPval 比较了"试 1 次"和"试 n 次"：GPT-5 在后一种用法下，相对不用 AI 的专家，速度约 1.4 倍、成本约 1.6 倍；模型做得成的任务上，花费不到专家薪酬的 10%。

小注：论文比较三种算法——naive ratio（只比双方各自的耗时和成本，不管质量）；try once then fix（采样一次 → 专家审 → 不满意就自己做）；try n times then fix（不满意就重新采样，最多 n 次，再不行自己做），审阅时间和胜率都计入。GPT-5：试 1 次 1.12×（速度） / 1.18×（成本），试 n 次 1.39× / 1.63×。论文里的重试是独立重采样，不是课上口头说的"在上一版基础上改"。

• 最重要的一条限制：上下文

- 把 prompt 里的上下文删掉一部分，胜率掉几个点，模型不知道该怎么办。
- 评测量的是"把脑子里的上下文全写给一个聪明人，他能不能做"，没有量"身在这份工作里的人自己弄清该做什么"。定优先级、选问题、分配资源才是职场里的难点，目前是人当架构师、模型来执行——和 METR 的维护者 vs 外包者是同一个现象。
- 近期含义：AI 辅助已进入专业工作流，配上人的监督是划算的，但要按任务类型挑模型。

• 问答

- 模型赢了人，人就不需要了吗？讲者：先去读那条 prompt——那是一位专家把多年积累的上下文全写了进去。
- 美元标价可靠吗？讲者认为重点不在金额，而在第一次有了一张按职业列任务、可量化的清单，能看出模型擅长的类别（深度调研、代码生成、消化多来源信息）落在哪些职业；上一届课程只能拿 MLE-bench、RE-Bench 这类离 ML 研究者最近的评测来讲。

5. 论文三：DeepScholar-Bench——检索、综合、可验证引用

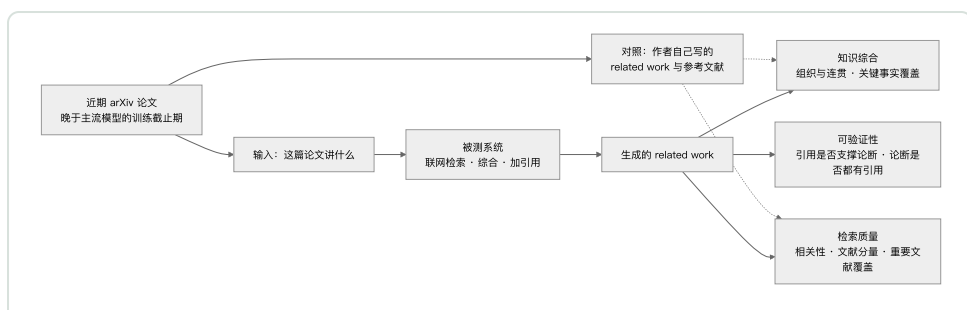


图 8-4 | DeepScholar-Bench 的任务与三个评测维度（自绘示意）·

► 看原幻灯片 53:35 · 出处：Patel et al., 2025

- 动机：GDPval 里多数任务要读参考文件，"去查、去读、去引用"值得单独量。同类系统已经很多：OpenAI / Gemini / Perplexity 的 Deep

Research、Stanford 的 STORM、OpenScholar，还有作业 3 里学生自己搭的 deep research agent。

- **任务与数据**：给一篇论文写 related work——人人要写，查询永远是新的。取近期 arXiv 论文（讲者称 PhD 级难度、22 个领域），只要主流模型训练截止之后发表的，避开数据污染；每月用新论文重跑，所以是"活的" benchmark。

小注：论文 v1（2025 年 8 月，课上用的版本）是 63 篇论文、18 个 arXiv 领域（课上说 22 个），发表于 2025 年 4—6 月，即 Llama-4 发布之后；给系统的输入是论文摘要，对照物是作者自己写的 related work。

- **三个维度、七个指标**

- 知识综合：组织与连贯性；nugget coverage（关键事实写到了多少）。
- 检索质量：relevance rate（找来的文献相关吗）；document importance（按引用数看有没有分量）；reference coverage（重要文献找回了多少）。
- 可验证性：citation precision（引用真的支撑那句话吗）；claim coverage（每个论断都有引用撑着吗）——一个管准，一个管全。
- 自动指标与人工标注的一致率在 70—80%。

小注：各指标大致都以人写的 related work 为参照：组织性由 LLM 评审与人写版本两两比较；nugget 由 LLM 从人写版本里抽取；document importance 比较双方所引文献的引用数；reference coverage 算人写版本里被标为"重要"（不可省略或替换）的文献找回了多少。

- **结果**：没有系统超过 19%，离饱和很远（讲者：下一届可以直接拿来当期末项目）。
- 综合：OpenAI Deep Research 写得最好，但几乎所有系统都漏关键事实——英文漂亮不等于事实全。
- 检索：都找不全重要文献，document importance 全部低于 12.5%。
- 可验证性：DeepScholar-base 的引用精确度能到约 90%；OpenAI Deep Research 文笔最连贯，可验证性反而低。

小注：v1 的原话应理解为"没有系统能在全部指标上都超过 0.19"，短板在检索——reference coverage 最高 .187、document importance 最高 .124（均为 OpenAI DeepResearch），并没有一个 19% 的总分；citation precision 上 DeepScholar-base 最高 .936，OpenAI DeepResearch 为

.399。2026 年 2 月的 v2 把参考系统改名为 DeepScholar-ref，结论改为"各指标的几何平均不超过 31%"。

• 失败模式

- 找得到相关的，找不到奠基性的。专家靠多年积累知道哪些必引；系统只会判断"相关"，不会判断"重要"。
- 即使把标准文献直接给它（oracle 检索），关键事实覆盖率也只有约 50%——瓶颈不只在检索，也在提炼。
- 综合质量和可验证性此消彼长，没有系统两头都好。

• 问答

- 为什么讲这么窄的一个 benchmark：它代表"必须用好外部知识库"这一类真实任务，补上前两篇的缺口。
- "重要文献"由谁定：以人写的 related work 的参考文献为准；但评测不止数文献——related work 的本质是提出"本文与前人有何不同"的论断并逐条用引用撑住，论断与引用的对应同样要查。

6. 三把尺子合看：benchmark 和真实工作差在哪

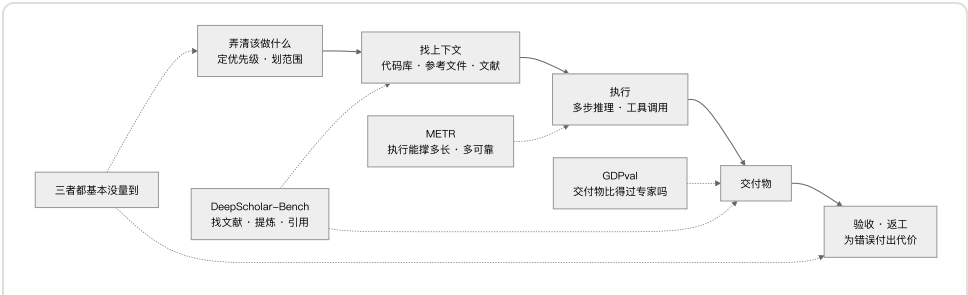


图 8-5 | 一件真实工作的链条，以及三个 benchmark 各量到哪一段（自绘示意，按讲者的总结归纳）。 ▶ 看原幻灯片 1:02:50

	METR time horizon	GDPval	DeepScholar-Bench
量什么	给定成功率下能完成的任务时长	交付物对专家的胜率	综合、检索、可验证性
以谁为锚	约 5 年经验专业者的耗时	十年以上从业者的交付物	人写的 related work
怎么判	自动判分	专家两两偏好	自动指标，经人工校验
现状	7 个月翻倍；50% 下约 1 小时，80% 下约 15 分钟	两年近似线性，最高 47.6%	没有系统过 19%
离真实工作差在哪	单 agent、自动判分、犯错不受罚、不限资源	说明极其充分、一次交付没有往返、依赖模型已有的隐性知识	找不全重要文献、提炼不出关键事实、引用不稳

- **把研究综述放进前两把尺子：**这类任务人要做 30 分钟到 8 小时，正落在当前约 50 分钟的能力带里；但在 time-horizon 图上"看起来能做"，不等于质量过关。缺口在多步推理、全面的信息收集，以及在多次工具调用之间拼接上下文时守住可验证性（作业 3 的体会）。
- **比较有把握的：**孤立、说明清楚的任务在稳步变好；软件工程和 ML 研究进步最大（讲者：我们最懂这两个领域，于是先自动化了自己的工作），已能做小时级任务；强模型的输出组织得很好，哪怕内容不全对。
- **把握不大的：**重上下文、prompt 含糊的任务；对抗性环境；95% 级别的可靠度；软件 and 知识工作之外的泛化；从知识库里找全高质量来源并核实。
- **结论：**METR 的外推是 2028—2031 年出现能做"一个月长"任务的 agent；GDPval 的线性趋势给这种外推降了温——约 48% 的总胜率背后是大量胜率很低的职业；DeepScholar-Bench 显示一旦要自己找上下文，质量和可验证性都跟不上人。时长、经济价值、综合质量三类指标要一起看，而且都得用真人校准。

7. 最后的问答

- **这条指数会像摩尔定律一样持续吗：**编码方向（如 SWE-bench Verified）还会涨；但大量工作不是"一个 PR"这种形态，长尾最难——类比 Waymo，前 80% 容易，最后 20% 最磨人；分布式系统是模型很难做对的方向之一。
- **两年后讲者自己的工作：**AI 的时间线一年都难预测。能提出假设、跑实验、闭合整个环的 AI scientist 还没到；眼下更有价值的是 AI co-scientist——知识面广、很强的头脑风暴伙伴。各领域的"最后一公里"都是长尾可靠性。
- **"50% 成功率"怎么算**（学生问：是"全体任务 × 多次尝试"的平均，还是要求每个任务都过 50%）：讲者答每个任务多次运行得到该任务的成功率，再按人类耗时排开；50% 就是掷硬币。精确定义见第 2 节的小注——两种猜法都不对，它是拟合曲线与 50% 的交点。
- **CS 还值得学吗：**职业不会消失；从基本原理出发推理的能力更重要，因为 AI 要靠人给上下文。
- **长尾卡在数据还是能力：**都有。很多创业公司在造环境和任务，难点是造出有代表性的任务、再补上对应的能力缺口。法律、金融调研这种宽类别落地快；很窄的场景数据不够；机器人领域有一批公司把"采数据"当成主要假设。

关键图表速查（点时间戳跳到原幻灯片）

图	看什么	跳转	出处
METR 方法总览	任务 → 人类与 agent 两路数据 → 拟合 → horizon 对发布日期，四步流程	6:50	Kwa et al.
成功率 vs 人类耗时散点	横轴人类耗时、纵轴成功率 0—1；绿点 SWAA 在高处，HCAST 从 0.8 到 0 铺开，RE-Bench 偏低	12:38	同上
50% time horizon 趋势	纵轴是 50% 成功率下的任务时长；GPT-2 约 2 秒 → GPT-4 几分钟 → Claude 3.7 Sonnet 59 分钟	13:48	同上
50% 对 80% 两条线	灰线 50%、蓝线 80%；斜率几乎一样，蓝线整体低一截（15 分钟 vs 59 分钟）	20:31	同上
失败模式对比	GPT-4 与 o1 各类失败的条数；"重复失败动作"一行差别最大	22:37	同上
GDPval 职业与示例任务	九个行业下的职业列表和九个示例任务；边看边想哪些模型可能做得好	28:24	Patwardhan et al.
GDPval 主结果	横轴按模型排（GPT-4o → GPT-5 high → Claude Opus 4.1）；浅蓝 = 胜 + 平，深蓝 = 只算胜；12.4% → 47.6%	35:38	同上
失败原因分布	蓝色柱 = 指令遵循类错误，是最大头；GPT-5 最少	38:50	同上
速度 / 成本表	试 1 次 vs 试 n 次；GPT-5 在试 n 次下约 1.4×（速度）和 1.6×（成本）	40:35	同上
逐职业胜率（三页密集图）	红线 = 与人类专家持平，各色 = 不同模型；找贴近红线的职业	42:20	同上
DeepScholar-Bench 指标体系	三个维度各自下面的指标；citation precision 与 claim coverage 一个管准、一个管全	53:35	Patel et al.
DeepScholar-Bench 结果	没有系统过 19%；OpenAI Deep Research 综合强、可验证性弱，DeepScholar-base 相反	55:41	同上

提到的工作

名称	在本讲里的作用
Measuring AI Ability to Complete Long Tasks (Kwa et al., METR, 2025)	论文一：50% / 80% time horizon，7 个月翻倍
SWAA / HCAST / RE-Bench	METR 的三个任务集：原子动作 / 多样的软件与研究工程任务 / 8 小时级 ML 研究工程
SWE-bench Verified	METR 的外部复核；也是最后问答里"编码方向还会涨"的例子
GDPval (Patwardhan et al., OpenAI, 2025)	论文二：44 个职业的真实交付物，对专家的胜率

名称	在本讲里的作用
O*NET	美国的职业—任务分类体系，GDPval 用它筛"数字化为主"的职业
SWE–Lancer（字幕作 "cvLancer"，推断）	带真实报价的自由职业软件任务，讲者用来对比任务价值
DeepScholar–Bench（Patel et al., 2025）与 DeepScholar–base	论文三：写 related work 的活 benchmark 及其参考系统
OpenAI / Gemini / Perplexity 的 Deep Research、STORM、OpenScholar	已有的研究综述类系统
ReAct、AI Scientist、记忆专题（课表第 4 / 7 / 14 次课）	讲驱动因素和 RE–Bench 时回指的前序内容
MLE–bench（字幕作 "ML–Bench"，推断）与 RE–Bench	上一届课程讲评测时用的例子
Claude Code、Codex、Cursor	课堂讨论里的 coding agent：compaction、显式规划、Tab 补全的 online RL
AI co–scientist	讲者认为现阶段比全自动 AI scientist 更有价值的形态
GPT–2、GPT–4、o1、Claude 3.7 Sonnet；GPT–4o、GPT–5（high）、Claude Opus 4.1	两条趋势线上的模型
Waymo	长尾类比：前 80% 容易，最后 20% 最难

术语对照

English	中文
agentic evaluation	智能体评测
long–horizon task	长程任务
50% / 80% time horizon	50% / 80% 成功率下的任务时长（时间跨度）
human baseline, baseliner	人类基线，基线测试者
geometric mean	几何平均
doubling time	翻倍时间
reliability	可靠度
inter–rater agreement	评分者间一致率
failure mode analysis	失败模式分析
premature task abandonment	过早放弃任务
repeating failed actions	重复已失败的动作

English	中文
messy task	杂乱任务（无唯一答案、环境复杂）
low-context contractor / repo maintainer	低上下文的外包者 / 仓库维护者
economically valuable task	有经济价值的任务
deliverable	交付物
win rate, pairwise preference	胜率，两两偏好
reference file	参考文件
gold subset	开源的金标子集
under-specified prompt	欠指定的 prompt
tacit knowledge	隐性知识
one-shot (non-interactive)	一次交付（没有多轮往返）
human oversight	人类监督
generative research synthesis	生成式研究综述
live benchmark	活的（持续更新的）benchmark
data contamination	数据污染
nugget coverage	关键事实覆盖率
document importance	文献重要性
reference coverage	重要文献覆盖率
citation precision / claim coverage	引用精确度 / 论断覆盖率
verifiability	可验证性
oracle retrieval	直接给定标准文献的检索设定
context engineering, compaction	上下文工程，上下文压缩
long tail	长尾
saturate	饱和

字幕勘误

"genetic evaluations" → agentic evaluations；"GDPVal" → GDPval；"meter" / "the beta paper" → METR / the METR paper；"five weeks shorter" → 推断为 5x shorter（80% horizon 约为 50% horizon 的四到五分之一，15 分钟对 59 分钟；讲者此前从图上估读的"8—10 分钟"以幻灯片的 15 分钟为准）；"Too use" → tool use；"React" → ReAct；"online URLs"（Cursor 的 Tab 补全）→ online RL；"Cuda" → CUDA；"refractor" → refactor；"open goal set" / "hugging phase" → open gold set / Hugging

Face; "cvLancer" → 应为 SWE-Lancer; 讲 GDPval 模型差异时句首的 "it always" → 应为 Claude (与论文对 Claude Opus 4.1 的描述一致); "ML-Bench" → 应为 MLE-bench; "Storm" → STORM; "DeepScholar base" → DeepScholar-base; Waymo 那句里听不清的部分按上下文应为"前 80%". 开场学生提到的 "[INAUDIBLE] bench" 无法还原。

带走的问题

1. 用"人类耗时"当难度的代理变量, 该用谁的时间? 熟手会低估, 低上下文的人比维护者慢 5—18 倍, 而模型更像后者。一个 agent benchmark 是否应该同时报告"高上下文人类"和"低上下文人类"两条基线, 并说明模型被给了哪一种上下文?
2. 从 50% 到 80% horizon 缩到约四分之一, 到 95% 会缩到多少? 高可靠度下翻倍时间还一样吗? "长尾可靠性"究竟是更多环境和数据能解决的, 还是需要新的能力 (知道何时算做完、何时该停)?
3. METR 的指数和 GDPval 的线性是矛盾, 还是同一进步在不同坐标下的样子——胜率有上界, time horizon 是对数轴上的无界量? 有没有办法在同一批任务上同时测两者来检验?
4. 三个 benchmark 都把上下文写进了题目。怎么评"弄清该做什么、主动找上下文"的能力: 任务说明故意不全、允许 agent 澄清和探索、返工有代价、评分仍可复现——这样的评测该怎么设计? 主观交付物上人与人只有七成一致, 自动评审的噪声地板又在哪?

CS329A 第 9 讲 | 未来研究方向 (Future Research Areas)

Stanford CS329A: Self-Improving AI Agents (2025 秋) · 对应课表第 20 次课 (12 月 5 日, 学期最后一讲; 结尾是结课致辞, 问答里说的"周一那节课"即 12 月 1 日 Danny Driess 的机器人客座) 视频:

<https://www.youtube.com/watch?v=AyO6wyu4DEg> (1:07:42, 自带英文 CC, 专有名词错得不少, 见文末勘误) 讲者: Aakanksha Chowdhery (学期回顾 + 前三篇论文)、Azalia Mirhoseini (Intelligence per Watt + 开放方向); 两人远程接入, 幻灯片由 Aakanksha 代翻 (推断) 配套阅读: 课表这一讲没有列阅读。课上展开的四篇: [Multiagent Finetuning](#) (Subramaniam et al., 2025) · [DeepSeekMath-V2](#) (Shao et al., 2025) · [Absolute Zero](#) (Zhao et al., 2025) · [Intelligence per Watt](#) (Saad-Falcon et al., 2025)

一句话: 自我改进回路想一直转下去, 卡在三处: 推理链越训越同质、只核对终答案的奖励管不住过程、训练题目还得靠专家挑——三篇论文各拆一处 (多 agent 保多样性、元验证、模型自己出题)。回路转得越多推理账单越大, 所以后半讲问的是: 每瓦特能买到多少智能, 有多少流量其实可以留在本地。

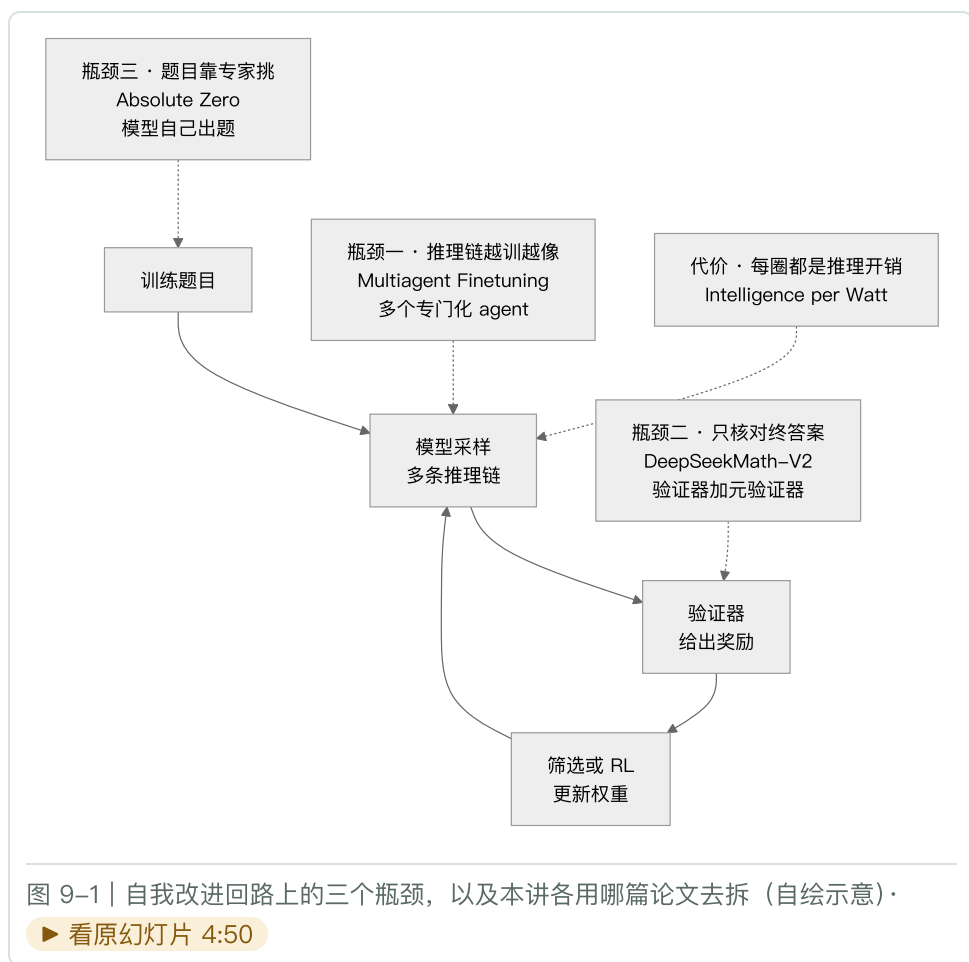
时间轴

时间	内容
0:06	学期回顾: verifier 驱动回路 → 开放式进化 → 工具与 workflow → 检索与记忆 → 规划与评测 → 客座
2:44	重申 agent 的定义; workflow 仍多是手写的, coding agent 是例外
4:19	本讲结构: 自我改进回路的三个瓶颈 + 智能的效率
6:54	论文一 Multiagent Finetuning: 单模型自训为什么几轮就停
9:00	生成 agent、评审 agent 与多轮辩论; 微调数据怎么取
12:39	结果: 多样性曲线、多轮微调、MATH → GSM8K 零样本
14:45	论文二 DeepSeekMath-V2: 终答案奖励为什么不够, LLM judge 为什么不行
17:20	验证器 + 元验证器; 生成器与验证器互相抬升; 标注自动化
19:57	结果: IMO Shortlist 2024 上的 Pass@1 / Best@32
21:31	推广到其他领域的三条配方与局限
23:02	论文三 Absolute Zero: 训练题目靠专家挑, 这条路越走越窄
25:09	出题—解题回路; 演绎 / 溯因 / 归纳三类任务

时间	内容
26:44	出题奖励、任务校验、缓冲区与课程学习
29:49	结果与涌现行为；只在代码上自训，数学也涨；大模型收益更大
31:52	Azalia：与 SWiRL 的发现互相印证
32:54	三个瓶颈小结；研究题：只靠可验证领域能把模型推多远
35:01	不可验证的两类：验证太慢、真正主观；学生追问现在怎么做
40:19	Intelligence per Watt：mainframe 时代与推理需求爆炸
43:26	两个趋势：77% 的请求不需要前沿模型；本地加速器内存涨 126 倍
46:37	IPW 的定义与实验规模
48:38	三个发现：88.7%、M4 Max 对 B200、 $5.3 = 3.1 \times 1.7$
51:53	开放方向一：test-time scaling 的原理、蒸馏、continual learning、推理 infra
56:40	开放方向二：本地—云混合推理、低能耗架构与 kernel、把能耗当优化目标
58:53	问答：continual learning 靠记忆系统还是改权重；无限上下文与 Cartridges
1:04:05	问答：agent 能不能自己造环境
1:05:42	结课致辞

核心内容

1. 学期回顾，和这一讲的两条线



- **学期主线：**verifier 驱动回路（跑验证器拿奖励，再用 RL 或搜索在数学、代码上爬坡；第 1—3、6 讲）→ 不靠单一奖励的开放式进化（AlphaEvolve；课表第 7 次课，视频未公开）→ 带工具的端到端工作流（第 4、5、7 讲）→ 检索与记忆 → 多步规划与 agent 评测（第 8 讲）→ 几场客座。Azalia 补充：客座讲的符号方法也属于工具使用，还能用来造合成数据。
- **agent 的现状：**定义没变（有目标、与环境交互、拿反馈纠偏）。LLM 单独还撑不起完整目标，多数系统仍是手写的 workflow，编排 LLM、verifier、judge、工具和并行搜索；coding agent 是开始能自己驱动流程的例外。
- **本讲两条线：**前半讲回路自身的三个瓶颈（图 9-1）——现在的 test-time / train-time scaling 基本只在数学、代码这类窄领域见效；后半讲效率，即“每单位什么，换来多少智能”。

2. 方向一：推理链的多样性——Multiagent Finetuning

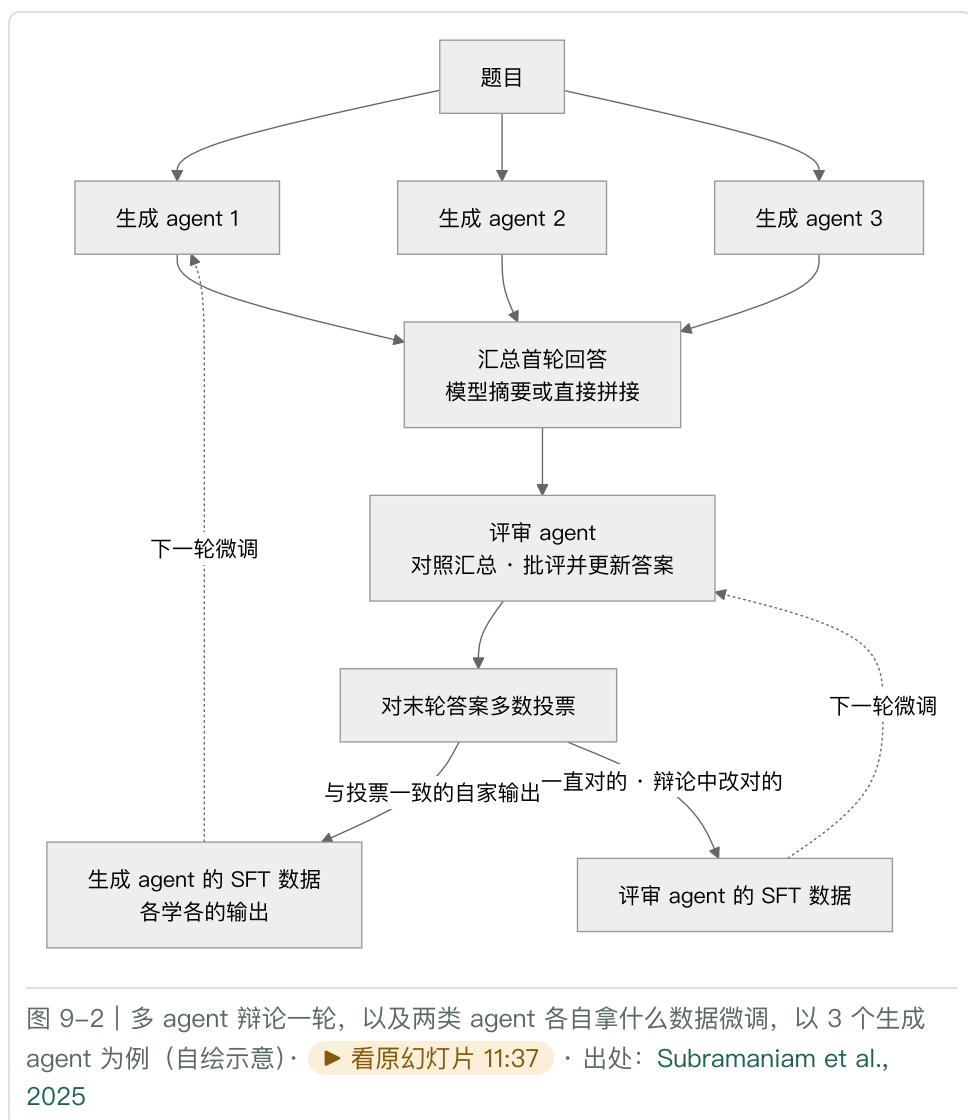


图 9-2 | 多 agent 辩论一轮，以及两类 agent 各自拿什么数据微调，以 3 个生成 agent 为例（自绘示意）· [▶ 看原幻灯片 11:37](#) · 出处：Subramaniam et al., 2025

- **问题：**预训练受限于互联网数据，指令微调受限于人类偏好数据。替代办法是拿模型自己生成的数据迭代微调：采样、滤掉错的（rejection sampling）、只在好样本上 SFT；第 6 讲的 STaR 是带推理链的变体。
- **为什么不够：**数据出自同一个模型，解法彼此相似，调高 temperature 也救不回来，提升通常几轮到几十轮就停。预训练数据则是很多人在很长时间里写出来的，多样性是天然的。
- **做法：**从同一个基座分化出一组生成 agent 和一组评审 agent。生成 agent 各自作答；之后每一轮，评审 agent 对着“所有 agent 回答的汇总”做批评并更新答案；最后对末轮答案多数投票。生成 agent 只学自己那些与投票一致的输出；评审 agent 学两类轨迹的混合——一开始就对的，和辩论中才被改对的，以学会分辨对错。与普通 self-critique 的差别：批评对象是多个 agent 的汇总，生成端先有了多样

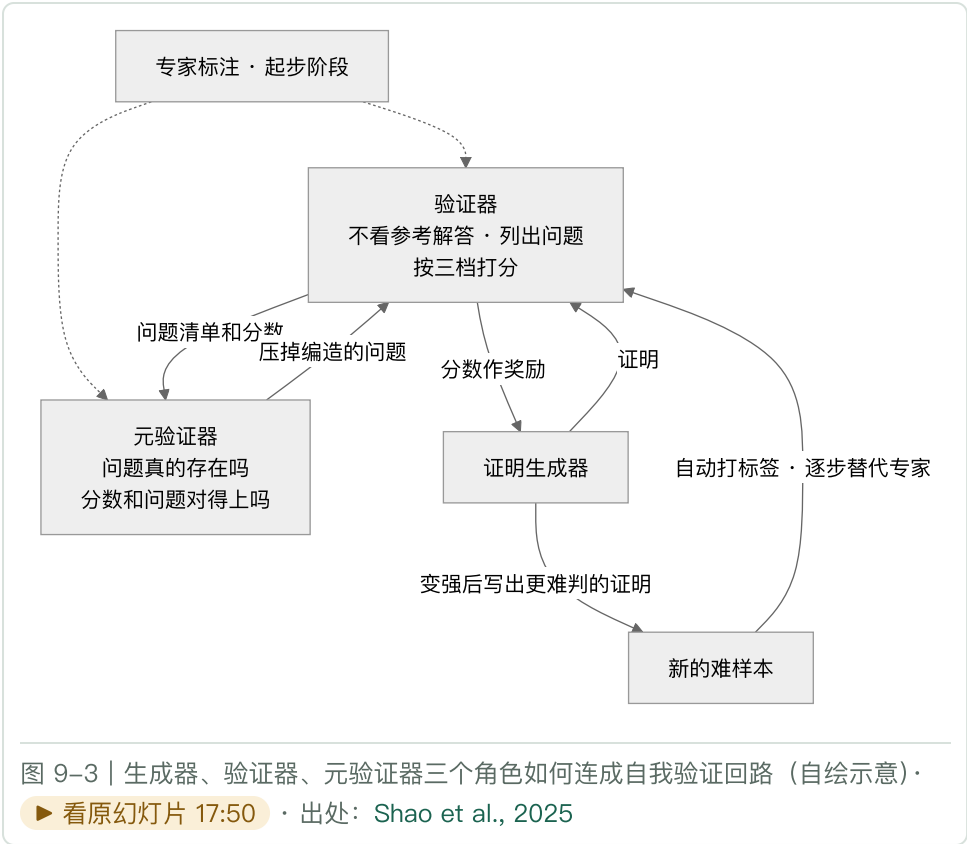
性，多数投票（第 2 讲）也顺带有了。穷人版是直接几个不同的模型和 prompt 各生成一份，实践里很常见。

- **结果：**随微调轮次增加，多 agent 微调下各回答之间的 embedding dissimilarity 一直保持在高位，准确率多轮后还在涨；单 agent 微调则停滞甚至塌掉。MATH 上三个开源模型都如此；只在 MATH 上训、拿到 GSM8K 上零样本测也更好。

小注：按论文，那张双图的两个指标（NLL、embedding dissimilarity）量的都是多样性，准确率随轮次的变化是另一张图；课上把 NLL 说成性能的代理，以论文为准。论文默认 3 个 agent、辩论 2 轮，除 Phi-3、Mistral-7B、LLaMA-3-8B 外也微调了 GPT-3.5。

- **落点：**想让回路持续转，喂回去的推理链必须保持多样；怎么得到多样性，整体仍是开放问题。

3. 方向二：不靠终答案的验证——DeepSeekMath-V2



- **问题：**现在的 RL 只要终答案对上标准答案就给奖励，这足以刷满 AIME 一类榜单；但答案对不等于推理对（第 3 讲的 false positive），定理证明更是没有终答案可对，要的是每一步都严格。
- **为什么不够：**PRM 难造（第 3 讲）；拿 LLM 当 judge 也不行——模型主要在算数值答案的题上训出来，写的证明常不成立，检查时又会

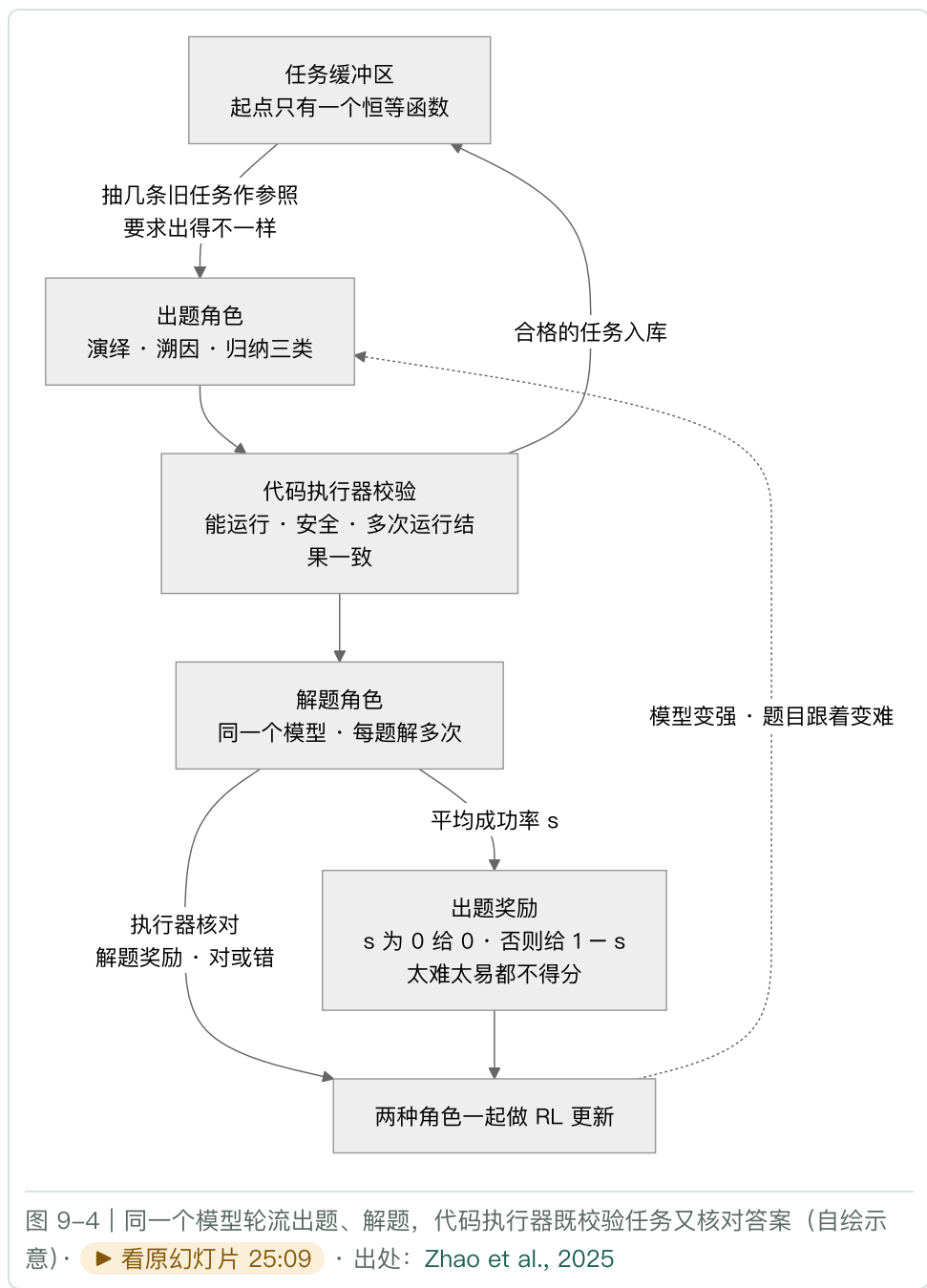
把错的判成对的。人类专家却不需要参考解答，就能看出哪一步接不上。

- **做法**：让专家在不看参考解答的条件下给证明挑问题，据此训练验证器：输入证明，输出问题清单和一个三档分数。验证器也会错，典型的是编造并不存在的问题。于是再叠一层元验证器，专门审验证器的分析：问题真的存在吗，分数是从这些问题推出来的吗；专家再给这层评估打标。
- **回路**：验证器的分数用来训练生成器；生成器变强后写出更难判的证明，又成为验证器的新训练材料。起步靠专家，元验证器学成之后，找错这一步的标注可以自动化。
- **结果**：IMO Shortlist 2024 上，迭代数加到 8，Pass@1 的证明得分一路上升，Best@32 接近 42%；评测还包括 IMO 题和 CNML 难度的题。Gemini 的对应工作是 Thang Luong 客座讲的，这一篇是开源版本。

小注：按论文，三档分数是 0 / 0.5 / 1（字幕丢了 0）；RL 用第 6 讲讲过的 GRPO（字幕作 TRPO），基座是 DeepSeek-V3.2-Exp-Base；那张图的横轴是测试时的顺序自我修正轮数（1 = 不修正，8 = 最多再改 7 次），不是训练迭代。论文另报告：元验证把验证器分析的质量分从 0.85 提到 0.96；IMO 2025、CMO 2024 达金牌线，Putnam 2024 为 118/120。

- **推广配方**：①让 LLM 验证器在没有参考解答时也能指出问题；②加元验证，压低编造问题的概率；③给生成器加一条激励，靠更审慎的推理把质量做高。局限：仍只适用于验证相对容易的领域。对照第 3 讲：Weaver 不训练新验证器，这里是把验证器训到能给自己造标签。

4. 方向三：让模型自己出题——Absolute Zero



- **问题：** SFT 用人工整理的推理轨迹，RLVR 要专家整理"题目一答案"对——数学要数学专家，IMO 级要 IMO 级专家。模型越接近、越超过人，能出题的人和可用的题就越少。讲者评价这个想法很新、还没怎么被用起来。
- **做法：** 不要任何外部题目，同一个模型兼任出题者和解题者，落在可以执行的代码领域。三类任务都围绕"程序、输入、输出"三元组：

任务	出题者给出	解题者要做的（按论文补）
deductio n 演绎	程序 + 输入，环境执行得到输出	给程序和输入，预测输出

任务	出题者给出	解题者要做的（按论文补）
abduction 溯因	同上	给程序和输出，反推一个可行的输入
induction 归纳	抽一个已有程序，生成一批新输入，外加一段描述函数的自然语言	给部分输入输出样例和描述，写出程序并通过隐藏样例

- **出题奖励：**每道题让解题者做多次。成功率为 0 给 0 分，否则给"1 - 平均成功率"，于是偏好难度适中的题——有时对、有时错，才有东西可学；模型变强，出题者也得出更难的问题。
- **校验与课程：**任务进训练前先过三关——程序能跑通、通过安全检查、多次运行输出完全一致。种子只有一个恒等函数；合格任务连同成败记录入库，出题时抽旧任务作参照并被要求出得不一样，既保多样性，也形成一套随时间演化的课程（curriculum）。
- **结果：**prompt 侧没有任何人工整理的数据，代码基准上仍做到 SOTA，超过用几万条专家样例训练的模型。训练中任务的复杂度指标、程序和答案的多样性都在上升；讲者说有点像博弈，出题者和解题者略带对抗却互相成就。更意外的是：只在自己出的代码题上爬坡，数学基准也明显变好；模型越大收益越大。

小注：课上只讲了出题侧，上表"解题者要做的"一列按论文补。论文口径：3B / 7B / 14B 的 Coder 模型总体分别 +5.7 / +10.2 / +13.2 分，7B Coder 代码 +5.0、数学 +15.2；"SOTA"是和同为 zero 设定（从基座直接做 RL）的模型相比。

- **落点：**选题不该被人类数据卡住，前提是有验证手段和课程机制。思路上接开放式进化那次课（课表第 7 次，视频未公开）：让系统自己生成下一步该学的东西。

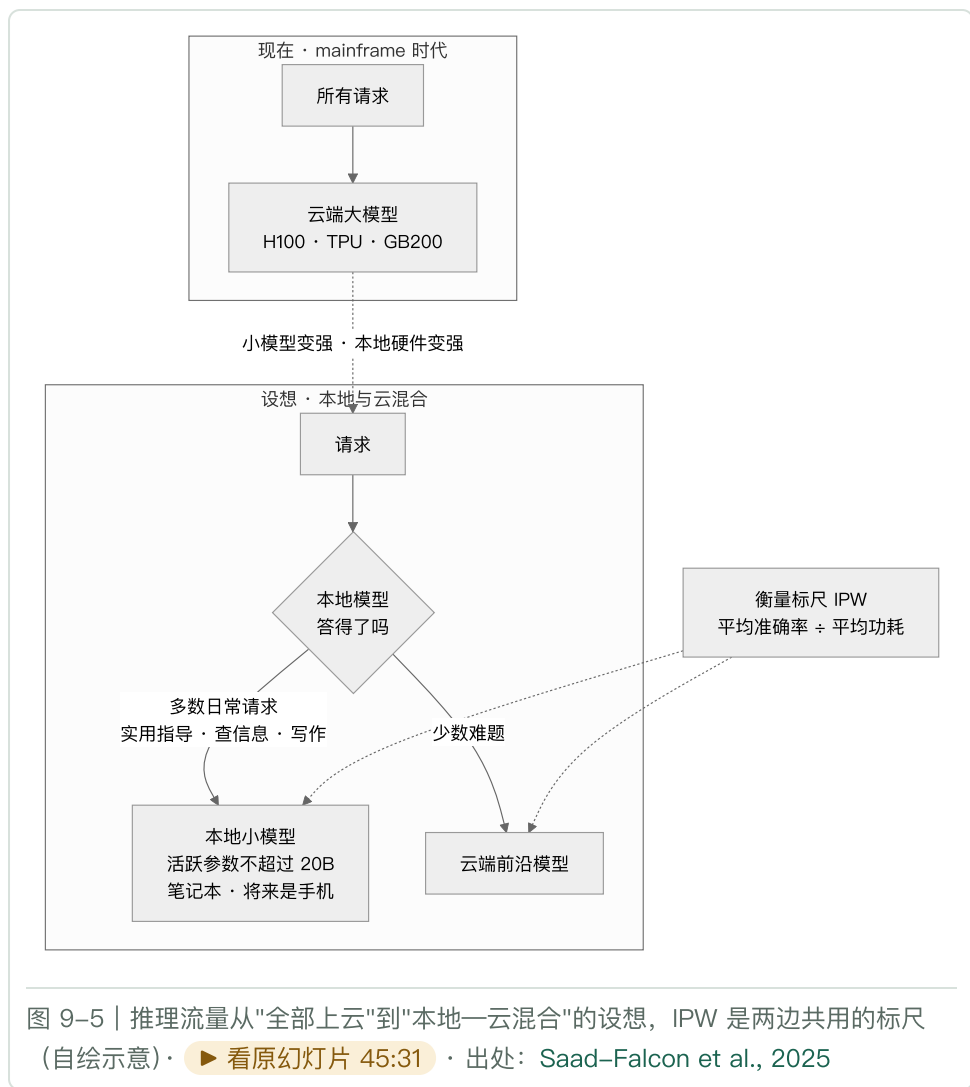
5. 小结与讨论：飞轮能迁移多远，不可验证的领域怎么办

- **Azalia 的旁证：**AZR 的发现和第 5 讲的 SWiRL（她是作者之一）对得上——在带检索的多跳问答上造合成数据、做逐步 RL，调用 Python 解数学题也变好，反之亦然；大模型更能吃下这种数据飞轮，在 RL 优化一侧尤其明显。模型自产数据带来跨任务迁移，是反复出现的现象。

小注：SWiRL 摘要的口径：只在 HotPotQA 上训练，GSM8K 零样本相对提升 16.9%。

- **小结与追加的研究题：**Aakanksha 把前半讲收成三点——多样性（怎么得到仍开放）、验证（越不依赖人类专家越好）、数据（人能整理的 prompt 有限）。Azalia 追加：只在可验证、能自动造数据的任务上推前沿，能带动多少没有自动验证的领域，从而减少甚至免掉那里的标注？很吃算力。
- **"不可验证"有两类：**一类能验证但太慢——科学发现、芯片设计里一跑几天的仿真、湿实验。RL 微调要成百上千步，test-time scaling 也要验证器几乎即时，等几分钟到一小时还行，等几天或等人打分不行。另一类真正主观，比如创意写作，写不出精确的奖励函数；可以学一个模型去近似，但稍有偏差，agent 就会 reward hacking。
- **学生追问现在怎么做：**Azalia——离线攒大量"输入—仿真结果"数据，训练一个预测仿真结果的 reward model 放进 RL 回路；它的泛化取决于数据量，预测不准会出问题。Aakanksha 以 KernelBench（第 2 讲提过）为例——单个 kernel 对不对、能不能编译有现成信号，可一旦许多 kernel 拼成大程序、要读性能剖析去定位该改哪段，就难得多（有课程项目在做）；常见办法是把问题拆到模型够得着的粒度，分块优化，再配一个放参考解的知识库。

6. 方向四：智能的效率——Intelligence per Watt



这是 Azalia 实验室与 Christopher Ré、John Hennessy 等人的合作。

- **背景:** 现在是 LLM 的 mainframe 时代, 大模型 (包括开源的) 几乎全在云上跑, 推理需求爆炸。讲者给的量级: Google Cloud 的服务量 20 个月涨约 1200 倍, NVIDIA 同比约 10 倍, 需要约 250 GW 的数据中心, 能源供给成了问题。

小注: 讲者口述 Google 每月处理的 token 数是"去年 2 月 160 trillion、今年 10 月 1.3 billion", 后者显然是口误。Google 公开的数字是 2024 年 4 月 9.7 万亿、2025 年 10 月 1.3 quadrillion (即 1.3 千万亿, 18 个月约 130 倍); 字幕里的起点无论和 1200 倍还是和公开数字都对不上, 引用前请对照幻灯片。

- **两个趋势:** ①大规模 ChatGPT 使用数据里, 约 77% 的请求是实用指导、查信息、写作, 并不需要前沿模型; 问题会随模型变强而变难, 但绝大多数仍在简单一侧。②2012 年以来本地 GPU 内存涨了约 126 倍, MacBook 可配到 100 GB 级内存, 量化后能装下相当大的模型。

- **指标与实验**：IPW = 平均任务准确率 ÷ 完成任务的平均功耗；"本地模型"指活跃参数不超过 20B，任务是单轮聊天与推理请求。覆盖 20 多个本地模型（Qwen、gpt-oss、Gemma 3 等）、企业级与本地两类加速器、约 100 万条请求（真实 ChatGPT 对话加 NaturalReasoning、MMLU-Pro、SuperGPQA），记录准确率、能耗、延迟、计算量，数据开源。
- **三个发现**：①本地模型已能答对 88.7% 的请求，能答对的比例两年涨了 3.1 倍；②本地加速器效率落后：同样的模型，B200 的 IPW 约是 Apple M4 Max 的 1.5 倍——B200 为语言模型负载做了极致优化，本地芯片要兼顾别的负载，设计时也没预期 LLM 会在本地跑；③两年里 IPW 提升 5.3 倍，3.1 倍来自模型、1.7 倍来自硬件。两条曲线一起走，越来越多的流量可以由笔记本、将来是手机承担。

小注：论文的口径是 B200 的 IPW 为 M4 Max 的 1.40 倍（摘要写作本地加速器"至少低 1.4 倍"）；3.1 倍对应可本地回答的比例从 23.2% 升到 71.3%。聊天请求来自 WildChat；77% 出自 Chatterji et al. 《How People Use ChatGPT》。论文估算理想路由可省约 80% 能耗。

7. 讲者列出的开放方向

方向	问题是什么	现状为什么不够	课上点到的工作
test-time scaling 的基本原理	同一个问题反复问，为什么正确答案会冒出来？说明了模型的什么性质？	只会用（采样、RL），不理解为什么有效	—
把成功轨迹蒸馏回模型	合成数据飞轮的最佳实践是什么	目前就是"收集数据再微调 / 做 RL"（第 5、6 讲的做法），没有公认的最佳实践	—
continual learning	人边做边长本事；模型是先攒一批 agent 经验，隔一段时间再离线微调	异步的"造数据—微调"范式；负面经验怎么学也没有答案	ICL、Cartridges（见本文第 8 节）
面向 test-time scaling 的推理 infra	重复采样、反复修改上一版、工具调用，负载形态和单轮聊天完全不同	主流 serving 为聊天优化，而这些方法正在变成主流	Hydragen、Tokasaurus
本地—云混合的推理引擎	按请求的需要和复杂度在本地与云之间平滑路由	现在几乎所有流量都上云	IPW 的测量数据
面向本地加速器的新架构与 kernel	低能耗推理	研究和芯片设计的注意力都在云端加速器上	—
把能耗当一等指标	能源会是今后最稀缺的资源	很少有工作把功耗当优化目标，怎么量都还没做好	IPW 这类指标

讲者用幻灯片底部的"预训练—后训练—test-time"示意收束：课程很少碰预训练；合成数据飞轮和 continual learning 落在微调、在线学习与 test-time scaling 的交界处，那里还有很多没解决的问题。

8. 最后的问答

- **continual learning 靠记忆系统，还是靠模型本身**：Aakanksha 说，把它对应到长期记忆系统是照着人来类比；在那之前还有更根本的缺口——"学会如何学习"，也就是在多步推理里从成败中学，把技能留进权重或其中一部分。她的例子是机器人什么时候能看视频学会一件事，而不是靠大量示范。Azalia 给了不改权重的思路：假如上下文无限长、且处处能被完美利用，把正负经验全放进去就够了；现实是到了百万级 token，模型对上下文内容的推理能力就下降。被追问工程上哪个容易，Aakanksha 的回答是看应用（下表）；机器人的跨本体泛化属于技能迁移，加多少记忆系统也得不到。

经验写进哪里	怎么做	适合什么	课上指出的限制
权重	微调 / RL，目前是离线、异步的	新领域的推理、技能迁移	做不到边做边学
外部记忆	更新一个模型会去查的数据库	知识更新，工程上最容易	教不会新领域的推理
上下文 / KV cache	ICL；Cartridges 不动权重、把知识放进 KV cache	长上下文、长期记忆	到百万级 token 后推理能力下降

- **有没有让 agent 自己造环境的论文**（提问者注意到 AZR 里环境扮演了后训练数据的角色）：回答（应为 Aakanksha）没有接"谁来造"这个问法。窄智能时代用游戏仿真当环境，是因为游戏是有限空间、容易用代码表示；今天环境重要，是因为它是真实任务的代理。用 agent 造还是用软件造都不难，难的是它是不是"模型与真实世界交互、拿到反馈"的合理代理。
- **结课**：Aakanksha——领域变得快，现在教的内容到下次开课就成了历史，但基本技术会被后来的工作反复用到；Azalia——这门课的备课本身也是边学边做。

关键图表速查（点时间戳跳到原幻灯片）

图	看什么	跳转	出处
多样性随微调轮次	横轴是微调轮次；右图 embedding dissimilarity 越高越多样——多 agent 一直在高位，单 agent	12:39	Subramani am et al.

图	看什么	跳转	出处
	往下掉		
多轮微调准确率与零样本泛化	多 agent 的线一轮轮往上，单 agent 塌掉或不再涨；紧接着一页是只在 MATH 上训、GSM8K 上测	13:11	同上
DeepSeekMath-V2 三角色结构	熟悉的"生成器 + 验证器"之外多出来的元验证器，以及箭头怎样连成回路	17:50	Shao et al.
IMO Shortlist 2024 得分曲线	Pass@1 和 Best@32 两条线都随迭代数（到 8）上升；Best@32 接近 42%	20:30	同上
AZR 总览图	出题阶段（三类任务 + 出题奖励）和解题阶段（校验 + 准确率奖励）怎样汇成一次联合更新	25:09	Zhao et al.
出题奖励公式	成功率为 0 给 0，否则 $1 - \text{平均成功率}$ ；太难和太易都拿不到分	27:14	同上
AZR 结果表	零人工题目，对比用几万条专家样例训练的模型；数学列也在涨；模型越大涨得越多	29:49	同上
Google 每月处理 token 数	曲线有多陡——讲者称之为史上增长最快的算力需求之一	42:55	—
ChatGPT 请求类别随时间变化	实用指导、查信息、写作合计约 77%；复杂请求在增加，简单请求仍占绝大多数	43:26	应出自 Chatterji et al.
本地加速器内存增长	2012 年至今约 126 倍	45:01	Saad-Falcon et al.
IPW 的三个发现	88.7% 的请求本地可答；B200 的 IPW 约为 M4 Max 的 1.5 倍；5.3 倍 = 模型 3.1 倍 $\times$ 硬件 1.7 倍	48:38	同上
预训练 / 后训练 / test-time 示意	幻灯片底部：合成数据飞轮和 continual learning 落在微调、在线学习与 test-time scaling 的交界	56:07	—

提到的工作

名称	在本讲里的作用
Multiagent Finetuning: Self Improvement with Diverse Reasoning Chains (Subramaniam et al., 2025)	方向一：用一组分化的生成 / 评审 agent 保住自训数据的多样性
DeepSeekMath-V2: Towards Self-Verifiable Mathematical Reasoning (Shao et al., 2025)	方向二：无参考解答的证明验证器 + 元验证器
Absolute Zero: Reinforced Self-play Reasoning with Zero Data (Zhao et al., 2025)	方向三：同一模型自己出题、自己解题，代码执行器当环境
Intelligence per Watt (Saad-Falcon, Narayan et al., 2025)	方向四：IPW 指标与本地推理可行性的大规模测量

名称	在本讲里的作用
SWiRL (Goldie et al., 2025)	Azalia 的旁证：合成数据 + 逐步 RL 带来跨任务迁移，大模型受益更多
STaR (Zelikman et al., 2022)	带推理链的迭代自训，单模型自我改进的代表
AlphaEvolve (Novikov et al., 2025)	学期回顾里开放式进化的代表
KernelBench (Ouyang et al., 2025)	例子："对不对"可验证，"大程序的性能瓶颈在哪"难验证
Hydragen (Juravsky et al., 2024) · Tokasaurus	Azalia 实验室面向高吞吐采样的推理系统工作
Cartridges (Eyuboglu et al., 2025)	不改权重、把知识放进 KV cache 的长上下文方案
How People Use ChatGPT (Chatterji et al., 2025)	77% 这个数字的出处（应为；IPW 论文引用）
Gemini 的 IMO 金牌工作 (Thang Luong 客座)	DeepSeekMath-V2 的闭源对照
MATH / GSM8K / AIME / IMO Shortlist 2024 / CNML 难度题集	前三篇论文涉及的评测
NaturalReasoning / MMLU-Pro / SuperGPQA	IPW 的请求来源与评测

术语对照

English	中文
self-improvement loop	自我改进回路
reasoning chain	推理链
rejection sampling	拒绝采样
synthetic data flywheel	合成数据飞轮
multiagent debate	多智能体辩论
generation agent / critic agent	生成 agent / 评审 agent
embedding dissimilarity	嵌入不相似度
theorem proving	定理证明
self-verification	自我验证
meta-verification, meta-verifier	元验证，元验证器
reference solution	参考解答
hallucinated / fabricated issue	编造出来的问题
RL with verifiable rewards (RLVR)	可验证奖励的强化学习
self-play	自博弈
proposer / solver	出题者 / 解题者

English	中文
deduction / abduction / induction	演绎 / 溯因 / 归纳
curriculum learning	课程学习
task buffer	任务缓冲区
non-verifiable domain	不可验证领域
reward hacking	奖励投机
performance profile	性能剖析
intelligence per watt (IPW)	每瓦智能
local inference / local accelerator	本地推理 / 本地加速器
active parameters	活跃参数
quantization	量化
power draw	功耗
hybrid serving, routing	混合推理服务, 路由
continual learning	持续学习
in-context learning (ICL)	上下文学习
cross-embodiment generalization	跨本体泛化
skill transfer	技能迁移

字幕勘误

"Azalea" → Azalia; "Akanksha" → Aakanksha; "alpha evolve" → AlphaEvolve; "Amy" → AIME; "GSM 8-k" → GSM8K; "TRPO" → 应为 GRPO; "a scale of 0.5 and 1" → 0 / 0.5 / 1 三档; "just past one" → Pass@1; "best at 32" → Best@32; "absolute zero paper" → Absolute Zero (AZR); "Professor Ray" → Christopher Ré; "1.3 billion" → 应为 1.3 quadrillion; "GP200 and GP300s" → GB200 / GB300; "MLU pro" → MMLU-Pro; "super GPQA" → SuperGPQA; "hardware sub-trees" → hardware substrates (推断); "hydrogen token SRS" → Hydragen、Tokasaurus; "vet lab" → wet lab; "EICL" → ICL; "KB caches" → KV caches; "kernel bench" → KernelBench; 提问里的 "the alarms themselves" → the LLMs themselves。"CNML" 不是错字，是论文里内部题集的难度参照 (Chinese National High School Mathematics League)。

带走的问题

1. 只在可验证领域（数学、代码）里转飞轮，能把不可验证领域的能力带到多高？AZR 的"代码 → 数学"和 SWiRL 的"问答 → 数学"是同一种迁移吗，边界在哪？
2. 元验证器由谁来验证？在没有专家起步标注、也没有 IMO 评分标准那种共识的领域，这层叠加还成立吗？标注自动化之后，生成器和验证器会不会把彼此的错误越放越大？
3. 同一个基座分化出来的 agent，多样性的上限在哪？和直接用几个不同基座（穷人版）相比哪种更划算？它和 RL 把分布压尖（第 3 讲最后的讨论）之间怎么取舍？
4. 出题奖励只要求"难度适中"。离开代码执行器这种便宜又确定的环境，任务的有效性靠什么校验？"难度刚好、却毫无价值"的题怎么排除？
5. 慢验证领域用学出来的 reward model 代替仿真或实验：误差到多大，RL 就开始钻空子？"便宜的代理 + 偶尔一次真实验证"能不能做成有保证的方案？
6. 经验该写进权重、外部记忆，还是 KV cache？哪类能力非改权重不可，有没有判据？如果以每瓦智能为目标重新设计模型、kernel 和路由，路由器自己的判断成本和判错成本怎么计入？